Importación del dataset

Existen dos alternativas para importar el conjunto de datos (dataset). La primera consiste en utilizar la librería ucimlrepo, aunque este método puede presentar inconvenientes en entornos locales debido a errores en la validación de los certificados SSL. La segunda opción es realizar la descarga mediante wget. En el notebook hay celdas que deben ejecutarse únicamente cuando se utiliza un método de importación determinado. Por esta razón, es importante leer atentamente las indicaciones que aparecen en los textos para identificar cuáles celdas corresponden a cada alternativa.

Importación de dataset usando ucimlrepo

In [1]:
```
!pip install ucimlrepo
```

```
Requirement already satisfied: ucimlrepo in .\.venv\Lib\site-packages (0.0.7)
Requirement already satisfied: pandas>=1.0.0 in .\.venv\Lib\site-packages (from ucim
lrepo) (3.1.1)
Requirement already satisfied: certifi>=2020.12.5 in .\.venv\Lib\site-packages (from
ucimlrepo) (2026.10.23)
Requirement already satisfied: numpy>=2.0.2 in .\.venv\Lib\site-packages (from panda
s>=1.0.0->ucimlrepo) (2.5.4)
Requirement already satisfied: python-dateutil>=2.9.0 in .\.venv\Lib\site-packages
(from pandas>=1.0.0->ucimlrepo) (4.0.0)
Requirement already satisfied: tzdata in .\.venv\Lib\site-packages (from pandas>=1.
0.0->ucimlrepo) (2026.6)
```

In [2]:
```
import numpy as np
import pandas as pd
from ucimlrepo import fetch_ucirepo
```

In [3]:
```
# fetch dataset
online_shoppers_purchasing_intention_dataset = fetch_ucirepo(id=468)

# data (as pandas dataframes)
X = online_shoppers_purchasing_intention_dataset.data.features
y = online_shoppers_purchasing_intention_dataset.data.targets.copy()

online_shoppers_purchasing_intention_dataset['variables']
```

Out[3]:

	name	role	type	demographic	description	units	missing_
0	Administrative	Feature	Integer	None	None	None	
1	Administrative_Duration	Feature	Integer	None	None	None	
2	Informational	Feature	Integer	None	None	None	
3	Informational_Duration	Feature	Integer	None	None	None	
4	ProductRelated	Feature	Integer	None	None	None	
5	ProductRelated_Duration	Feature	Continuous	None	None	None	
6	BounceRates	Feature	Continuous	None	None	None	
7	ExitRates	Feature	Continuous	None	None	None	
8	PageValues	Feature	Integer	None	None	None	
9	SpecialDay	Feature	Integer	None	None	None	
10	Month	Feature	Categorical	None	None	None	
11	OperatingSystems	Feature	Integer	None	None	None	
12	Browser	Feature	Integer	None	None	None	
13	Region	Feature	Integer	None	None	None	
14	TrafficType	Feature	Integer	None	None	None	
15	VisitorType	Feature	Categorical	None	None	None	
16	Weekend	Feature	Binary	None	None	None	
17	Revenue	Target	Binary	None	None	None	

In [4]:
```
print(X.shape, y.shape)
print(y["Revenue"].value_counts())
```

```
(12330, 17) (12330, 1)
Revenue
False    10422
True      1908
Name: count, dtype: int64
```

Validación del archivo `online_shoppers.csv` del repositorio frente a UCI

El análisis de este notebook usa los datos descargados de UCI con `fetch_ucirepo(id=468)`. La siguiente celda compara ese conjunto con el archivo `online_shoppers.csv` del repositorio para dejar evidencia de si ambos coinciden: dimensiones, columnas, nulos, distribución de `Revenue`, valores fila a fila y etiquetas

distintas. Si hay diferencias, se guardan en `diferencias_csv_vs_uci.csv`. Esta comprobación no modifica `X` ni `y`.

In [5]:
```
import hashlib, urllib.request
import numpy as np
import pandas as pd

URL_CSV = "https://raw.githubusercontent.com/Jennyaorozco/online-shoppers-intention

try:
    from ucimlrepo import fetch_ucirepo
except ImportError:
    !pip install -q ucimlrepo
    from ucimlrepo import fetch_ucirepo

_uci = fetch_ucirepo(id=468)
uci = pd.concat([_uci.data.features.copy(), _uci.data.targets.copy()], axis=1)

csv = pd.read_csv(URL_CSV)
with urllib.request.urlopen(URL_CSV) as r:
    print("SHA-256 del CSV del repositorio:", hashlib.sha256(r.read()).hexdigest())

def normalizar(d):
    d = d.copy()
    d["Revenue"] = d["Revenue"].astype(int)
    d["Weekend"] = d["Weekend"].astype(int)
    d["Month"] = d["Month"].replace({"June": "Jun"})   # UCI usa 'June' y el CSV 'J
    return d

uci_n, csv_n = normalizar(uci), normalizar(csv)

print("\n1) Dimensiones      UCI:", uci_n.shape, "| CSV:", csv_n.shape)
print("2) Mismas columnas  :", set(uci_n.columns) == set(csv_n.columns))
print("3) Nulos           UCI:", int(uci_n.isna().sum().sum()), "| CSV:", int(csv_

cols = list(uci_n.columns)
csv_n = csv_n[cols]
print("\n4) Distribución de Revenue (1 = compra)")
print(pd.DataFrame({"UCI": uci_n["Revenue"].value_counts().sort_index(),
                    "CSV": csv_n["Revenue"].value_counts().sort_index()}))

if len(uci_n) == len(csv_n):
    features = [c for c in cols if c != "Revenue"]
    dif_col = {}
    for c in features:
        a, b = uci_n[c], csv_n[c]
        if pd.api.types.is_numeric_dtype(a) and pd.api.types.is_numeric_dtype(b):
            dif_col[c] = int((~np.isclose(a.to_numpy(float), b.to_numpy(float), rto
        else:
            dif_col[c] = int((a.astype(str).to_numpy() != b.astype(str).to_numpy())
    n_feat_dif = sum(dif_col.values())
    filas_etq = np.where(uci_n["Revenue"].values != csv_n["Revenue"].values)[0]
    print("\n5) Valores distintos en variables predictoras (total):", n_feat_dif)
    if n_feat_dif:
        print({k: v for k, v in dif_col.items() if v})
```

```
    print("6) Filas con etiqueta Revenue distinta:", len(filas_etq))
    if len(filas_etq):
        diferencias = pd.DataFrame({
            "fila": filas_etq,
            "Revenue_UCI": uci_n["Revenue"].values[filas_etq],
            "Revenue_CSV": csv_n["Revenue"].values[filas_etq],
            "Month": uci_n["Month"].values[filas_etq],
            "PageValues": uci_n["PageValues"].values[filas_etq],
        })
        print(diferencias.groupby(["Revenue_UCI", "Revenue_CSV"]).size().rename("fi
        diferencias.to_csv("diferencias_csv_vs_uci.csv", index=False)
        display(diferencias.head(10))
    ok = (n_feat_dif == 0) and (len(filas_etq) == 0)
else:
    ok = False
    print("\nLos conjuntos tienen distinto número de filas; no se puede comparar fi

print("\nVEREDICTO:", "el CSV del repositorio es idéntico al dataset de UCI."
      if ok else "el CSV del repositorio NO es idéntico al dataset de UCI (ver dife
```

```
SHA-256 del CSV del repositorio: 170b9d87aa8a31e288aa8f1850ebb539cb56755a0fe16220165
ad97da2aa6d3b

1) Dimensiones      UCI: (12330, 18) | CSV: (12330, 18)
2) Mismas columnas  : True
3) Nulos            UCI: 0 | CSV: 0

4) Distribución de Revenue (1 = compra)
           UCI    CSV
Revenue
0        10422  10422
1         1908   1908

5) Valores distintos en variables predictoras (total): 0
6) Filas con etiqueta Revenue distinta: 0

VEREDICTO: el CSV del repositorio es idéntico al dataset de UCI.
```

Análisis variables

De acuerdo a la descripción del dataset disponible en UC Irvine, se concluye que las siguientes variables son categóricas.

- Revenue
- Weekend
- VisitorType
- TrafficType
- Region
- Browser
- OperatingSystems
- Month

In [6]:
```
vars_cat = ["Weekend","VisitorType","TrafficType","Region","Browser","OperatingSyst
print("Revenue: ",np.unique(y["Revenue"]))
for i in vars_cat:
    print(i+": ", np.unique(X[i]))
```

```
Revenue:  [False  True]
Weekend:  [False  True]
VisitorType:  ['New_Visitor' 'Other' 'Returning_Visitor']
TrafficType:  [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
Region:  [1 2 3 4 5 6 7 8 9]
Browser:  [ 1  2  3  4  5  6  7  8  9 10 11 12 13]
OperatingSystems:  [1 2 3 4 5 6 7 8]
Month:  ['Aug' 'Dec' 'Feb' 'Jul' 'June' 'Mar' 'May' 'Nov' 'Oct' 'Sep']
```

Descripción de las variables numéricas

In [7]:
```
num_cols = ['Administrative','Administrative_Duration','Informational',
            'Informational_Duration','ProductRelated','ProductRelated_Duration',
            'BounceRates','ExitRates','PageValues','SpecialDay']
```

In [8]:
```
for i in num_cols:
    print(i+": ", X[i].dtype)
```

```
Administrative:  int64
Administrative_Duration:  float64
Informational:  int64
Informational_Duration:  float64
ProductRelated:  int64
ProductRelated_Duration:  float64
BounceRates:  float64
ExitRates:  float64
PageValues:  float64
SpecialDay:  float64
```

In [9]:
```
X[num_cols].describe()
```

Out[9]:

	Administrative	Administrative_Duration	Informational	Informational_Duration	Proc
count	12330.000000	12330.000000	12330.000000	12330.000000	12
mean	2.315166	80.818611	0.503569	34.472398	
std	3.321784	176.779107	1.270156	140.749294	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	
50%	1.000000	7.500000	0.000000	0.000000	
75%	4.000000	93.256250	0.000000	0.000000	
max	27.000000	3398.750000	24.000000	2549.375000	

Preparación encoding variables

One-hot Encoding

In [10]:
```
X = online_shoppers_purchasing_intention_dataset.data.features.copy()

for col in ['OperatingSystems', 'Browser', 'Region', 'TrafficType', 'VisitorType',
    X = pd.get_dummies(X, columns=[col], prefix=col, dtype=int)
```

Convertir valores tipos Boolean a int(0 y 1)

In [11]:
```
X['Weekend'] = X['Weekend'].astype(int)
```

In [12]:
```
y['Revenue'] = y['Revenue'].astype(int)
```

Nuevos shapes

In [13]:
```
X.shape, y.shape
```

Out[13]:
```
((12330, 74), (12330, 1))
```

Separación de test y train, demostración de la aplicación de SMOTE y definición de validación cruzada

Dividimos el dataset en test y train, asignando un 20% para el test

In [14]:
```
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE

# División train/test
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

In [15]:
```
print(X_train.shape, y_train.shape)
```

```
(9864, 74) (9864, 1)
```

Aplicación de SMOTE para generar muestras artificiales de la clase minoritaria de manera demostrativa para explorar los efectos de SMOTE

Se obtiene la relación entre la clase positiva y la clase negativa

In [16]:
```
clase_negativo, clase_positivo = y_train.value_counts()
(clase_positivo)/(clase_negativo)
```

Out[16]:
```
0.1830175101942912
```

Primer porcentaje: Se aumenta la proporción de un 20% a un 22%

In [17]:
```
# Aplicar Imputación y SMOTE de forma segura mediante un Pipeline secuencial
from sklearn.impute import SimpleImputer
from imblearn.pipeline import Pipeline

pipeline_balanceo_demo = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),  # Corrige los NaN causados por
    ('smote', SMOTE(random_state=42, sampling_strategy=0.22))
])

# Ajustar e imputar/balancear el conjunto de entrenamiento sin romper el flujo
X_train_resampled, y_train_resampled = pipeline_balanceo_demo.fit_resample(
    X_train,
    y_train
)
```

In [18]:
```
from sklearn.impute import SimpleImputer
from imblearn.pipeline import Pipeline

# Definir el pipeline con imputación de nulos (usando la estrategia de la mediana c
pipeline_balanceo = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('smote', SMOTE(random_state=42, sampling_strategy=0.22))
])

# Ajustar e imputar/balancear el conjunto de entrenamiento de forma segura
X_train_resampled, y_train_resampled = pipeline_balanceo.fit_resample(X_train, y_tr

# Verificar dimensiones y nulos resultantes
print("Nulos en X_train_resampled:", np.isnan(X_train_resampled).sum().sum())
print("Forma de X_train_resampled:", X_train_resampled.shape)
print("Distribución de clases después de SMOTE:")
print(y_train_resampled.value_counts())
```

```
Nulos en X_train_resampled: 0
Forma de X_train_resampled: (10172, 74)
Distribución de clases después de SMOTE:
Revenue
0          8338
1          1834
Name: count, dtype: int64
```

In [19]:
```
print(y_train.value_counts())
print(y_train_resampled.value_counts())
```

```
Revenue
0          8338
1          1526
Name: count, dtype: int64
Revenue
0          8338
1          1834
Name: count, dtype: int64
```

Segundo porcentaje: Se aumenta la proporción de un 20% a un 30%

In [20]:
```
from sklearn.impute import SimpleImputer
from imblearn.pipeline import Pipeline

# Definir el pipeline secuencial de forma segura
pipeline_balanceo1 = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('smote', SMOTE(random_state=42, sampling_strategy=0.3))
])

# Ajustar e imputar/balancear el conjunto de entrenamiento sin data leakage
X_train_resampled1, y_train_resampled1 = pipeline_balanceo1.fit_resample(X_train, y
```

In [21]:
```
print(y_train.value_counts())
print(y_train_resampled1.value_counts())
```

```
Revenue
0          8338
1          1526
Name: count, dtype: int64
Revenue
0          8338
1          2501
Name: count, dtype: int64
```

Tercer porcentaje: Se aumenta la proporción de un 20% a un 40%

In [22]:
```
# Aplicar Imputación y SMOTE de forma segura mediante un Pipeline secuencial
from sklearn.impute import SimpleImputer
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE

# Definir el pipeline con imputación de nulos y SMOTE al 40%
pipeline_balanceo2 = Pipeline([
```

```
    ('imputer', SimpleImputer(strategy='median')),  # Corrige los NaN causados por
    ('smote', SMOTE(random_state=42, sampling_strategy=0.4))
])

# Ajustar e imputar/balancear el conjunto de entrenamiento sin romper el flujo
X_train_resampled2, y_train_resampled2 = pipeline_balanceo2.fit_resample(
    X_train,
    y_train
)
```

In [23]:
```
print(y_train.value_counts())
print(y_train_resampled2.value_counts())
```

```
Revenue
0          8338
1          1526
Name: count, dtype: int64
Revenue
0          8338
1          3335
Name: count, dtype: int64
```

Definición de metodología de validación

In [24]:
```
from sklearn.model_selection import StratifiedKFold

cv_strategy = StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=42
)
```

Entrenamiento de modelos usando pipeline con normalización, SMOTE y malla de hiperparametros

In [25]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, StratifiedKFold, learning_curve,

from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.svm import SVC

from sklearn.metrics import roc_auc_score, classification_report, confusion_matrix,

from sklearn.utils.parallel import Parallel, delayed
```

```
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Transformación de y train para asegurar su compatibilidad respecto a dimensiones

In [26]:
```
y_train = y_train.values.ravel()
```

Random Forest

Definición de pipeline y malla de hiperparametros

In [27]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.metrics import roc_auc_score, classification_report, confusion_matrix
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
import numpy as np
```

In [28]:
```
# PIPELINE  (scaler → SMOTE → RandomForest)
# El StandardScaler no es estrictamente necesario para RF, pero se mantiene para se

pipeline_rf = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42, sampling_strategy=0.3)),   # valor base; GridS
    ('classifier', RandomForestClassifier(
        random_state=42,
        n_jobs=-1,          # usar todos los núcleos disponibles
        class_weight='balanced'  # refuerzo adicional para clase minoritaria (Reven
    ))
])
```

In [29]:
```
# Parámetros clave para RF:
#   n_estimators   → número de árboles; más árboles = más estable, más lento
#   max_depth      → profundidad máxima; None = crece hasta hojas puras (riesgo ove
#   min_samples_split → muestras mínimas para dividir un nodo (regularización)
#   min_samples_leaf  → muestras mínimas en hoja (regularización)
#   max_features   → features evaluados en cada split; 'sqrt' es estándar para clas
#   sampling_strategy → proporción SMOTE (clase_positiva / clase_negativa)

param_grid_rf = {
    # SMOTE (evaluar los 3 porcentajes)
    'smote__sampling_strategy': [0.22, 0.3, 0.4],
    'smote__k_neighbors': [3, 5, 7],

    # RF — solo los más influyentes, valores fijos para el resto
    'classifier__n_estimators': [200],        # fijo, suficiente para estabilidad
```

```
    'classifier__max_depth': [10, 20],        # el más importante a explorar
    'classifier__min_samples_leaf': [1, 4],   # segundo más importante
    'classifier__max_features': ['sqrt'],     # fijo, es el estándar para clasifica
}
```

In [30]:
```
# GRID SEARCH CON VALIDACIÓN CRUZADA ESTRATIFICADA
# Se optimiza balanced_accuracy; ROC-AUC y F1 se reportan sobre el conjunto de prue

from sklearn.model_selection import StratifiedKFold

cv_strategy = StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=42
)

grid_rf = GridSearchCV(
    estimator=pipeline_rf,
    param_grid=param_grid_rf,
    cv=cv_strategy,         # StratifiedKFold(n_splits=10, shuffle=True, random_sta
    scoring='balanced_accuracy',      # métrica principal del proyecto
    n_jobs=-1,
    verbose=0,
    error_score='raise',
    refit=True              # refitea automáticamente con los mejores parámetros
)
```

Entrenamiento usando la malla de valores y validación cruzada

In [31]:
```
from sklearn.impute import SimpleImputer

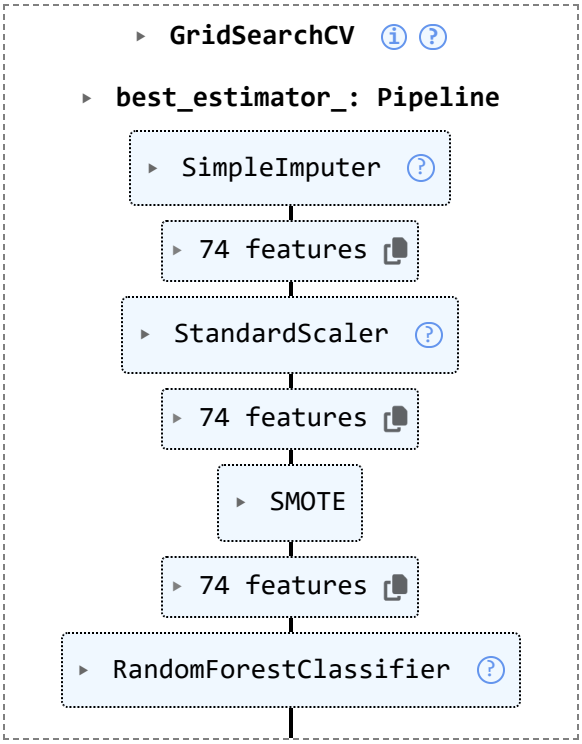
# Modificamos el pipeline original para incluir imputación y evitar problemas con N
pipeline_rf = Pipeline([
    ('imputer', SimpleImputer(strategy='median')), # Sin NaN con la codificación ac

    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42, sampling_strategy=0.3)),
    ('classifier', RandomForestClassifier(
        random_state=42,
        n_jobs=-1,
        class_weight='balanced'
    ))
])

# Actualizamos el estimador en grid_rf con el nuevo pipeline
grid_rf.estimator = pipeline_rf

# Entrenamiento seguro
grid_rf.fit(X_train, y_train)
```

Out[31]:



Resultados de los mejores 3 pipelines

In [32]:

```python
# OBTENER LOS 3 MEJORES PIPELINES DE LA VALIDACIÓN CRUZADA
from sklearn.impute import SimpleImputer
cv_results = grid_rf.cv_results_
n_top = 3
top_indices = np.argsort(cv_results['mean_test_score'])[-n_top:][::-1]

print("\n" + "=" * 80)
print("TOP 3 PIPELINES SEGÚN BALANCED_ACCURACY (CV)")
print("=" * 80)

for i, idx in enumerate(top_indices, 1):
    # Obtener hiperparámetros
    params = cv_results['params'][idx]
    balanced_acc_cv_mean = cv_results['mean_test_score'][idx]
    balanced_acc_cv_std = cv_results['std_test_score'][idx]

    # Reconstruir pipeline con estos parámetros e incluir Imputación para evitar er
    pipeline_temp = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42, sampling_strategy=params['smote__sampling_
        ('classifier', RandomForestClassifier(
            random_state=42, n_jobs=-1, class_weight='balanced',
            n_estimators=params['classifier__n_estimators'],
            max_depth=params['classifier__max_depth'],
            min_samples_leaf=params['classifier__min_samples_leaf'],
            max_features=params['classifier__max_features']
        ))
    ])
```

```python
    # Entrenar y evaluar
    pipeline_temp.fit(X_train, y_train)
    y_train_pred = pipeline_temp.predict(X_train)
    y_test_pred = pipeline_temp.predict(X_test)
    y_test_prob = pipeline_temp.predict_proba(X_test)[:, 1]

    # Métricas
    from sklearn.metrics import balanced_accuracy_score, roc_auc_score, f1_score

    balanced_acc_train = balanced_accuracy_score(y_train, y_train_pred)
    auc_test = roc_auc_score(y_test, y_test_prob)
    f1_test = f1_score(y_test, y_test_pred)

    print(f"\n{'='*80}")
    print(f"PIPELINE #{i}")
    print(f"{'='*80}")
    print("HIPERPARÁMETROS:")
    for param, value in params.items():
        print(f"  {param}: {value}")

    print(f"\nBALANCED_ACCURACY:")
    print(f"  Train: {balanced_acc_train:.4f}")
    print(f"  Validation (CV mean): {balanced_acc_cv_mean:.4f} (±{balanced_acc_cv_s

    print(f"\nTEST:")
    print(f"  ROC-AUC: {auc_test:.4f}")
    print(f"  F1-Score: {f1_test:.4f}")
    print()
```

```
================================================================================
TOP 3 PIPELINES SEGÚN BALANCED_ACCURACY (CV)
================================================================================

================================================================================
PIPELINE #1
================================================================================
HIPERPARÁMETROS:
  classifier__max_depth: 10
  classifier__max_features: sqrt
  classifier__min_samples_leaf: 4
  classifier__n_estimators: 200
  smote__k_neighbors: 7
  smote__sampling_strategy: 0.22

BALANCED_ACCURACY:
  Train: 0.8867
  Validation (CV mean): 0.8552 (±0.0134)

TEST:
  ROC-AUC: 0.9145
  F1-Score: 0.6498


================================================================================
PIPELINE #2
================================================================================
HIPERPARÁMETROS:
  classifier__max_depth: 10
  classifier__max_features: sqrt
  classifier__min_samples_leaf: 4
  classifier__n_estimators: 200
  smote__k_neighbors: 3
  smote__sampling_strategy: 0.22

BALANCED_ACCURACY:
  Train: 0.8867
  Validation (CV mean): 0.8549 (±0.0138)

TEST:
  ROC-AUC: 0.9145
  F1-Score: 0.6498


================================================================================
PIPELINE #3
================================================================================
HIPERPARÁMETROS:
  classifier__max_depth: 10
  classifier__max_features: sqrt
  classifier__min_samples_leaf: 4
  classifier__n_estimators: 200
  smote__k_neighbors: 5
  smote__sampling_strategy: 0.22

BALANCED_ACCURACY:
```

```
  Train: 0.8867
  Validation (CV mean): 0.8547 (±0.0097)

TEST:
  ROC-AUC: 0.9145
  F1-Score: 0.6498
```

Resultados

In [33]:
```python
# RESULTADOS DEL GRID SEARCH
print("=" * 60)
print("MEJORES HIPERPARÁMETROS ENCONTRADOS:")
print("=" * 60)
for param, value in grid_rf.best_params_.items():
    print(f"  {param}: {value}")

print(f"\nMejor exactitud balanceada (CV en train): {grid_rf.best_score_:.4f}")
```

```
============================================================
MEJORES HIPERPARÁMETROS ENCONTRADOS:
============================================================
  classifier__max_depth: 10
  classifier__max_features: sqrt
  classifier__min_samples_leaf: 4
  classifier__n_estimators: 200
  smote__k_neighbors: 7
  smote__sampling_strategy: 0.22

Mejor exactitud balanceada (CV en train): 0.8552
```

In [34]:
```python
# VALIDACIÓN CRUZADA FINAL CON EL MEJOR PIPELINE
scores_rf = cross_val_score(
    grid_rf.best_estimator_,
    X_train,
    y_train,
    cv=cv_strategy,
    scoring='roc_auc'
)

print(f"\nROC-AUC por fold (CV):")
print(np.round(scores_rf, 4))
print(f"Media:  {scores_rf.mean():.4f}")
print(f"Std:    {scores_rf.std():.4f}")
```

```
ROC-AUC por fold (CV):
[0.921  0.9287 0.9074 0.9312 0.906  0.9347 0.9361 0.9316 0.9325 0.9106]
Media:  0.9240
Std:    0.0112
```

In [35]:
```python
# EVALUACIÓN FINAL EN TEST
best_rf = grid_rf.best_estimator_
best_rf.fit(X_train, y_train)

y_pred_prob_rf = best_rf.predict_proba(X_test)[:, 1]
y_pred_rf = best_rf.predict(X_test)
```

```
roc_auc_test = roc_auc_score(y_test, y_pred_prob_rf)

print("\n" + "=" * 60)
print(f"ROC-AUC EN TEST: {roc_auc_test:.4f}")
print("=" * 60)

print("\nReporte de clasificación (threshold=0.5):")
print(classification_report(y_test, y_pred_rf, target_names=['No compra (0)', 'Comp

print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred_rf))
```

```
============================================================
ROC-AUC EN TEST: 0.9127
============================================================

Reporte de clasificación (threshold=0.5):
              precision    recall  f1-score   support

No compra (0)       0.96      0.87      0.91      2084
   Compra (1)       0.54      0.81      0.64       382

     accuracy                           0.86      2466
    macro avg       0.75      0.84      0.78      2466
 weighted avg       0.90      0.86      0.87      2466

Matriz de confusión:
[[1816  268]
 [  73  309]]
```

In [36]:
```
for v in ['X_train', 'y_train', 'cv_strategy', 'pipeline_svc', 'param_grid_svc', 'g
    print(v, v in globals())
```

```
X_train True
y_train True
cv_strategy True
pipeline_svc False
param_grid_svc False
grid_svc False
```

In [37]:
```
# === INTERVALOS DE CONFIANZA BOOTSTRAP para test ===
from sklearn.utils import resample

def bootstrap_ci(y_true, y_pred, y_prob, n_bootstrap=1000, alpha=0.05):
    """Calcula IC 95% por bootstrap para ROC-AUC y F1."""
    rng = np.random.RandomState(42)
    aucs, f1s = [], []
    for _ in range(n_bootstrap):
        idx = resample(range(len(y_true)), random_state=rng)
        if len(np.unique(np.array(y_true)[idx])) < 2:
            continue
        aucs.append(roc_auc_score(np.array(y_true)[idx], np.array(y_prob)[idx]))
        f1s.append(f1_score(np.array(y_true)[idx], np.array(y_pred)[idx]))
    lo, hi = alpha/2, 1 - alpha/2
    return {
        'ROC-AUC': (np.percentile(aucs, lo*100), np.mean(aucs), np.percentile(aucs,
```

```
        'F1':      (np.percentile(f1s,  lo*100), np.mean(f1s),  np.percentile(f1s,
    }

from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

if 'grid_svc' in globals() and hasattr(grid_svc, 'best_estimator_'):
    best_svc = grid_svc.best_estimator_          # modelo de la búsqueda
else:
    # Respaldo: mejores hiperparámetros de GridSearchCV (núcleo lineal, C=1, SMOTE
    best_svc = ImbPipeline([
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42, sampling_strategy=0.3)),
        ('svc', SVC(kernel='linear', C=1, probability=True, random_state=42))
    ]).fit(X_train, y_train)

y_pred_svc = best_svc.predict(X_test)
y_pred_prob_svc = best_svc.predict_proba(X_test)[:, 1]

ci = bootstrap_ci(y_test, y_pred_svc, y_pred_prob_svc)

print(f"ROC-AUC test: {ci['ROC-AUC'][1]:.4f} "
      f"(IC 95%: [{ci['ROC-AUC'][0]:.4f}, {ci['ROC-AUC'][2]:.4f}])")
print(f"F1 test:      {ci['F1'][1]:.4f} "
      f"(IC 95%: [{ci['F1'][0]:.4f}, {ci['F1'][2]:.4f}])")
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
ROC-AUC test: 0.8904 (IC 95%: [0.8715, 0.9087])
F1 test:      0.5722 (IC 95%: [0.5254, 0.6163])
```

Curva de Aprendizaje

In [38]:

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import learning_curve

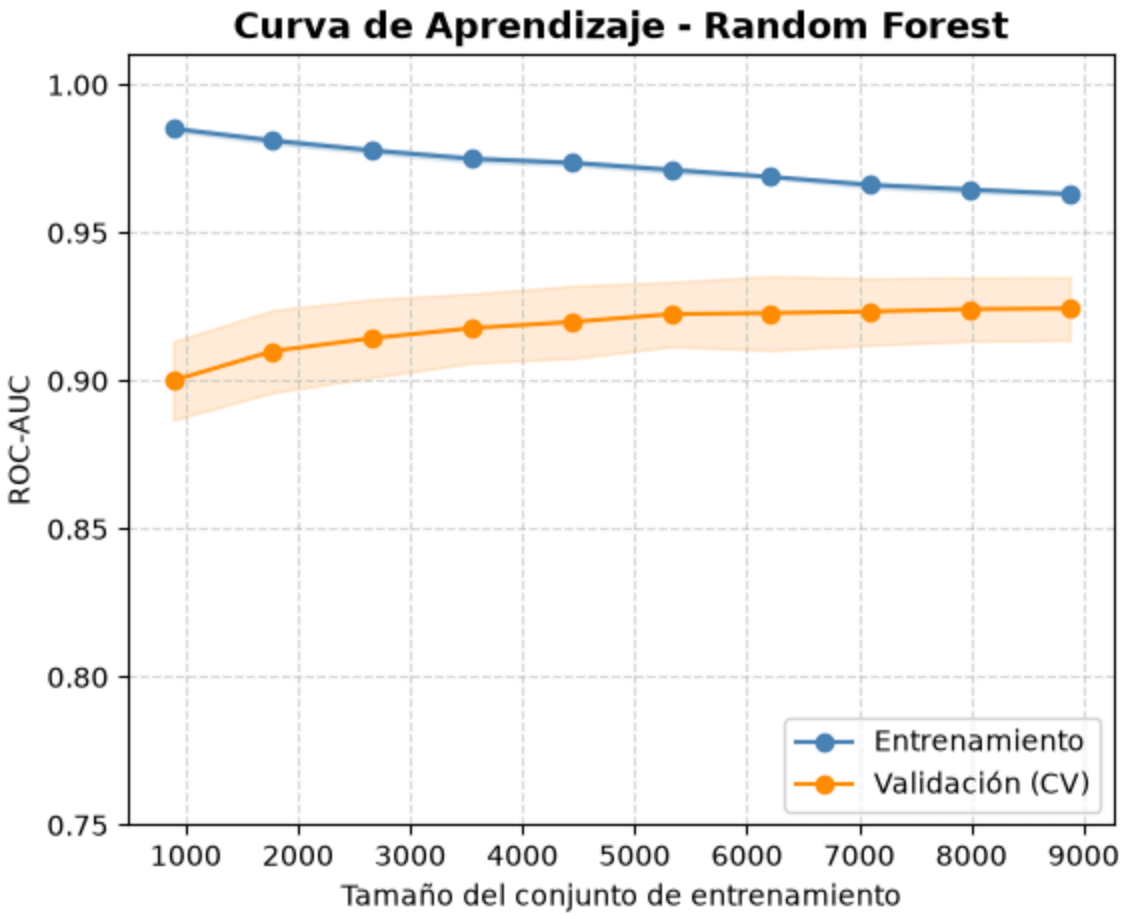
fig = plt.figure(figsize=(14, 5))
gs = gridspec.GridSpec(1, 2, figure=fig)

train_sizes, train_scores, val_scores = learning_curve(
    grid_rf.best_estimator_,
    X_train, y_train,
    cv=cv_strategy,
    scoring='roc_auc',
    train_sizes=np.linspace(0.1, 1.0, 10),
    n_jobs=-1
)
```

```
train_mean = train_scores.mean(axis=1)
train_std  = train_scores.std(axis=1)
val_mean   = val_scores.mean(axis=1)
val_std    = val_scores.std(axis=1)

ax1 = fig.add_subplot(gs[0])
ax1.plot(train_sizes, train_mean, 'o-', color='steelblue', label='Entrenamiento')
ax1.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha
ax1.plot(train_sizes, val_mean, 'o-', color='darkorange', label='Validación (CV)')
ax1.fill_between(train_sizes, val_mean - val_std, val_mean + val_std, alpha=0.15, c
ax1.set_title('Curva de Aprendizaje - Random Forest', fontsize=13, fontweight='bold
ax1.set_xlabel('Tamaño del conjunto de entrenamiento')
ax1.set_ylabel('ROC-AUC')
ax1.legend(loc='lower right')
ax1.grid(True, linestyle='--', alpha=0.5)
ax1.set_ylim(0.75, 1.01)
```

Out[38]: (0.75, 1.01)



Matriz de confusión

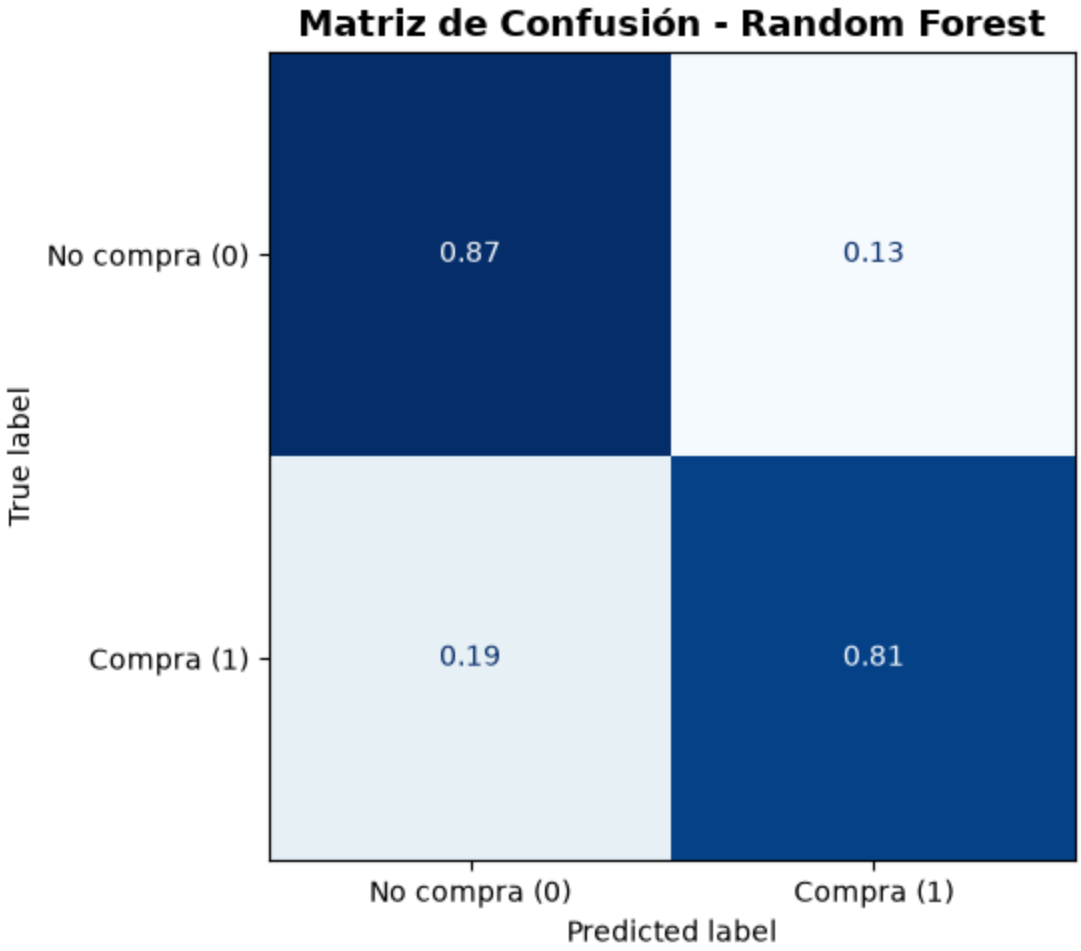
In [39]:
```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import learning_curve
```

```
fig = plt.figure(figsize=(6, 5))
gs = gridspec.GridSpec(1, 1, figure=fig)

cm = confusion_matrix(y_test, y_pred_rf,normalize="true")
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No compra (0)',

ax2 = fig.add_subplot(gs[0])
disp.plot(ax=ax2, colorbar=False, cmap='Blues')
ax2.set_title('Matriz de Confusión - Random Forest', fontsize=13, fontweight='bold'

plt.tight_layout()
plt.savefig('rf_curva_confusion.png', dpi=150, bbox_inches='tight')
plt.show()
```



XGBoost

In [40]:
```
from xgboost import XGBClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.metrics import roc_auc_score, classification_report, confusion_matrix
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
import numpy as np
```

In [41]:
```
pipeline_xgb = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42, sampling_strategy=0.3)),
    ('classifier', XGBClassifier(
        random_state=42,
        n_jobs=-1,
        eval_metric='logloss',   # evita warning interno de XGBoost
        verbosity=0              # silencia logs innecesarios
    ))
])
```

MALLA DE HIPERPARÁMETROS

In [42]:
```
param_grid_xgb = {
    # SMOTE — los 3 porcentajes requeridos
    'smote__sampling_strategy': [0.22, 0.3, 0.4],
    'smote__k_neighbors': [3, 5, 7],

    # XGBoost — parámetros más influyentes
    'classifier__n_estimators': [200],
    'classifier__max_depth': [3, 6],          # profundidad del árbol base
    'classifier__learning_rate': [0.05, 0.1], # tasa de aprendizaje
    'classifier__subsample': [0.8],           # fracción de muestras por árbol
    'classifier__colsample_bytree': [0.8],    # fracción de features por árbol
}
```

GRID SEARCH

In [43]:
```
grid_xgb = GridSearchCV(
    estimator=pipeline_xgb,
    param_grid=param_grid_xgb,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=0,
    refit=True
)
```

Entrenamiento usando la malla de valores y validación cruzada

In [44]:
```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier

# 1. Definir el pipeline seguro que incluye la imputación de NaNs
pipeline_xgb = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),  # Corrige los NaN causados por
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42, sampling_strategy=0.3)),
    ('classifier', XGBClassifier(
```

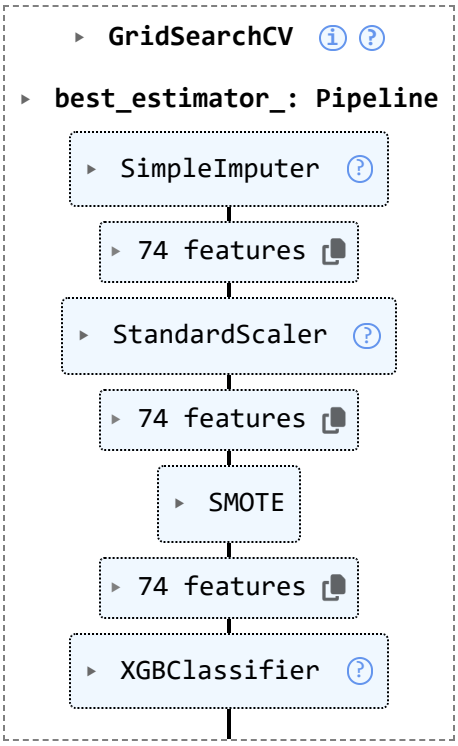
```
        random_state=42,
        n_jobs=-1,
        eval_metric='logloss',
        verbosity=0
    ))
])

# 2. Definir la malla de parámetros usando la configuración previa
param_grid_xgb = {
    'smote__sampling_strategy': [0.22, 0.3, 0.4],
    'smote__k_neighbors': [3, 5, 7],
    'classifier__n_estimators': [200],
    'classifier__max_depth': [3, 6],
    'classifier__learning_rate': [0.05, 0.1],
    'classifier__subsample': [0.8],
    'classifier__colsample_bytree': [0.8],
}

# 3. Inicializar el GridSearchCV
grid_xgb = GridSearchCV(
    estimator=pipeline_xgb,
    param_grid=param_grid_xgb,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=0,
    refit=True
)

# 4. Ajustar el modelo de forma segura
grid_xgb.fit(X_train, y_train)
```

Out[44]:



Resultados de los mejores 3 pipelines

In [45]:

```
from sklearn.impute import SimpleImputer

n_top = 3
# OBTENER LOS 3 MEJORES PIPELINES DE XGBOOST
cv_results_xgb = grid_xgb.cv_results_
top_indices_xgb = np.argsort(cv_results_xgb['mean_test_score'])[-n_top:][::-1]

print("\n" + "=" * 80)
print("TOP 3 PIPELINES XGBOOST SEGÚN BALANCED_ACCURACY (CV)")
print("=" * 80)

for i, idx in enumerate(top_indices_xgb, 1):
    params = cv_results_xgb['params'][idx]
    balanced_acc_cv_mean = cv_results_xgb['mean_test_score'][idx]
    balanced_acc_cv_std = cv_results_xgb['std_test_score'][idx]

    # Reconstruir pipeline con estos parámetros (incluyendo SimpleImputer para reso
    pipeline_temp_xgb = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42,sampling_strategy=params['smote__sampling_s
        ('classifier', XGBClassifier(
            random_state=42,
            n_jobs=-1,
            eval_metric='logloss',
            verbosity=0,
            n_estimators=params['classifier__n_estimators'],
            max_depth=params['classifier__max_depth'],
            learning_rate=params['classifier__learning_rate'],
            subsample=params['classifier__subsample'],
            colsample_bytree=params['classifier__colsample_bytree']
        ))
    ])

    # Entrenar y evaluar
    pipeline_temp_xgb.fit(X_train, y_train)
    y_train_pred = pipeline_temp_xgb.predict(X_train)
    y_test_pred = pipeline_temp_xgb.predict(X_test)
    y_test_prob = pipeline_temp_xgb.predict_proba(X_test)[:, 1]

    balanced_acc_train = balanced_accuracy_score(y_train, y_train_pred)
    auc_test = roc_auc_score(y_test, y_test_prob)
    f1_test = f1_score(y_test, y_test_pred)

    print(f"\n{'='*80}")
    print(f"PIPELINE XGBOOST #{i}")
    print(f"{'='*80}")
    print("HIPERPARÁMETROS:")
    for param, value in params.items():
        print(f"  {param}: {value}")

    print(f"\nBALANCED_ACCURACY:")
    print(f"  Train: {balanced_acc_train:.4f}")
```

```
    print(f"  Validation (CV mean): {balanced_acc_cv_mean:.4f} (±{balanced_acc_cv_s

    print(f"\nTEST:")
    print(f"  ROC-AUC: {auc_test:.4f}")
    print(f"  F1-Score: {f1_test:.4f}")
    print()
```

```
================================================================================
TOP 3 PIPELINES XGBOOST SEGÚN BALANCED_ACCURACY (CV)
================================================================================

================================================================================
PIPELINE XGBOOST #1
================================================================================
HIPERPARÁMETROS:
  classifier__colsample_bytree: 0.8
  classifier__learning_rate: 0.05
  classifier__max_depth: 3
  classifier__n_estimators: 200
  classifier__subsample: 0.8
  smote__k_neighbors: 3
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train: 0.8360
  Validation (CV mean): 0.8203 (±0.0180)

TEST:
  ROC-AUC: 0.9308
  F1-Score: 0.6753


================================================================================
PIPELINE XGBOOST #2
================================================================================
HIPERPARÁMETROS:
  classifier__colsample_bytree: 0.8
  classifier__learning_rate: 0.05
  classifier__max_depth: 3
  classifier__n_estimators: 200
  classifier__subsample: 0.8
  smote__k_neighbors: 5
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train: 0.8345
  Validation (CV mean): 0.8198 (±0.0194)

TEST:
  ROC-AUC: 0.9309
  F1-Score: 0.6632


================================================================================
PIPELINE XGBOOST #3
================================================================================
HIPERPARÁMETROS:
  classifier__colsample_bytree: 0.8
  classifier__learning_rate: 0.05
  classifier__max_depth: 3
  classifier__n_estimators: 200
  classifier__subsample: 0.8
  smote__k_neighbors: 7
```

```
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train: 0.8307
  Validation (CV mean): 0.8192 (±0.0199)

TEST:
  ROC-AUC: 0.9303
  F1-Score: 0.6727
```

RESULTADOS

In [46]:
```python
print("=" * 60)
print("MEJORES HIPERPARÁMETROS:")
print("=" * 60)
for param, value in grid_xgb.best_params_.items():
    print(f"  {param}: {value}")

print(f"\nMejor exactitud balanceada (CV en train): {grid_xgb.best_score_:.4f}")
```

```
============================================================
MEJORES HIPERPARÁMETROS:
============================================================
  classifier__colsample_bytree: 0.8
  classifier__learning_rate: 0.05
  classifier__max_depth: 3
  classifier__n_estimators: 200
  classifier__subsample: 0.8
  smote__k_neighbors: 3
  smote__sampling_strategy: 0.4

Mejor exactitud balanceada (CV en train): 0.8203
```

EVALUACIÓN FINAL EN TEST

In [47]:
```python
best_xgb = grid_xgb.best_estimator_
best_xgb.fit(X_train, y_train)

y_pred_prob_xgb = best_xgb.predict_proba(X_test)[:, 1]
y_pred_xgb = best_xgb.predict(X_test)

roc_auc_test = roc_auc_score(y_test, y_pred_prob_xgb)

print("\n" + "=" * 60)
print(f"ROC-AUC EN TEST: {roc_auc_test:.4f}")
print("=" * 60)

print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred_xgb, target_names=['No compra (0)', 'Com

print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred_xgb))
```

```
============================================================
ROC-AUC EN TEST: 0.9308
============================================================

Reporte de clasificación:
              precision    recall  f1-score   support

No compra (0)       0.94      0.94      0.94      2084
   Compra (1)       0.66      0.69      0.68       382

     accuracy                           0.90      2466
    macro avg       0.80      0.81      0.81      2466
 weighted avg       0.90      0.90      0.90      2466

Matriz de confusión:
[[1952  132]
 [ 120  262]]
```

Intervalos de confianza

In [48]:
```python
from sklearn.model_selection import cross_val_score
scores_xgb = cross_val_score(
    grid_xgb.best_estimator_,
    X_train,
    y_train,
    cv=cv_strategy,
    scoring='roc_auc'
)
print(f"ROC-AUC por fold Cross-Validation:")
print(np.round(scores_xgb, 4))
print(f"Media:  {scores_xgb.mean():.4f}")
print(f"Std:    {scores_xgb.std():.4f}")
```

```
ROC-AUC por fold Cross-Validation:
[0.9254 0.9346 0.9294 0.94   0.9185 0.9409 0.941  0.9393 0.942  0.9279]
Media:  0.9339
Std:    0.0077
```

In [49]:
```python
# === INTERVALOS DE CONFIANZA BOOTSTRAP para test ===
from sklearn.utils import resample

def bootstrap_ci(y_true, y_pred, y_prob, n_bootstrap=1000, alpha=0.05):
    """Calcula IC 95% por bootstrap para ROC-AUC y F1."""
    rng = np.random.RandomState(42)
    aucs, f1s = [], []
    for _ in range(n_bootstrap):
        idx = resample(range(len(y_true)), random_state=rng)
        if len(np.unique(np.array(y_true)[idx])) < 2:
            continue
        aucs.append(roc_auc_score(np.array(y_true)[idx], np.array(y_prob)[idx]))
        f1s.append(f1_score(np.array(y_true)[idx], np.array(y_pred)[idx]))
    lo, hi = alpha/2, 1 - alpha/2
    return {
        'ROC-AUC': (np.percentile(aucs, lo*100), np.mean(aucs), np.percentile(aucs,
        'F1':      (np.percentile(f1s,  lo*100), np.mean(f1s),  np.percentile(f1s,
    }
```

```
ci = bootstrap_ci(y_test, y_pred_xgb, y_pred_prob_xgb)
print(f"ROC-AUC test: {ci['ROC-AUC'][1]:.4f} "
      f"(IC 95%: [{ci['ROC-AUC'][0]:.4f}, {ci['ROC-AUC'][2]:.4f}])")
print(f"F1 test:      {ci['F1'][1]:.4f} "
      f"(IC 95%: [{ci['F1'][0]:.4f}, {ci['F1'][2]:.4f}])")
```

```
ROC-AUC test: 0.9308 (IC 95%: [0.9190, 0.9420])
F1 test:      0.6742 (IC 95%: [0.6354, 0.7107])
```

Curva de Aprendizaje

In [50]:
```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import learning_curve

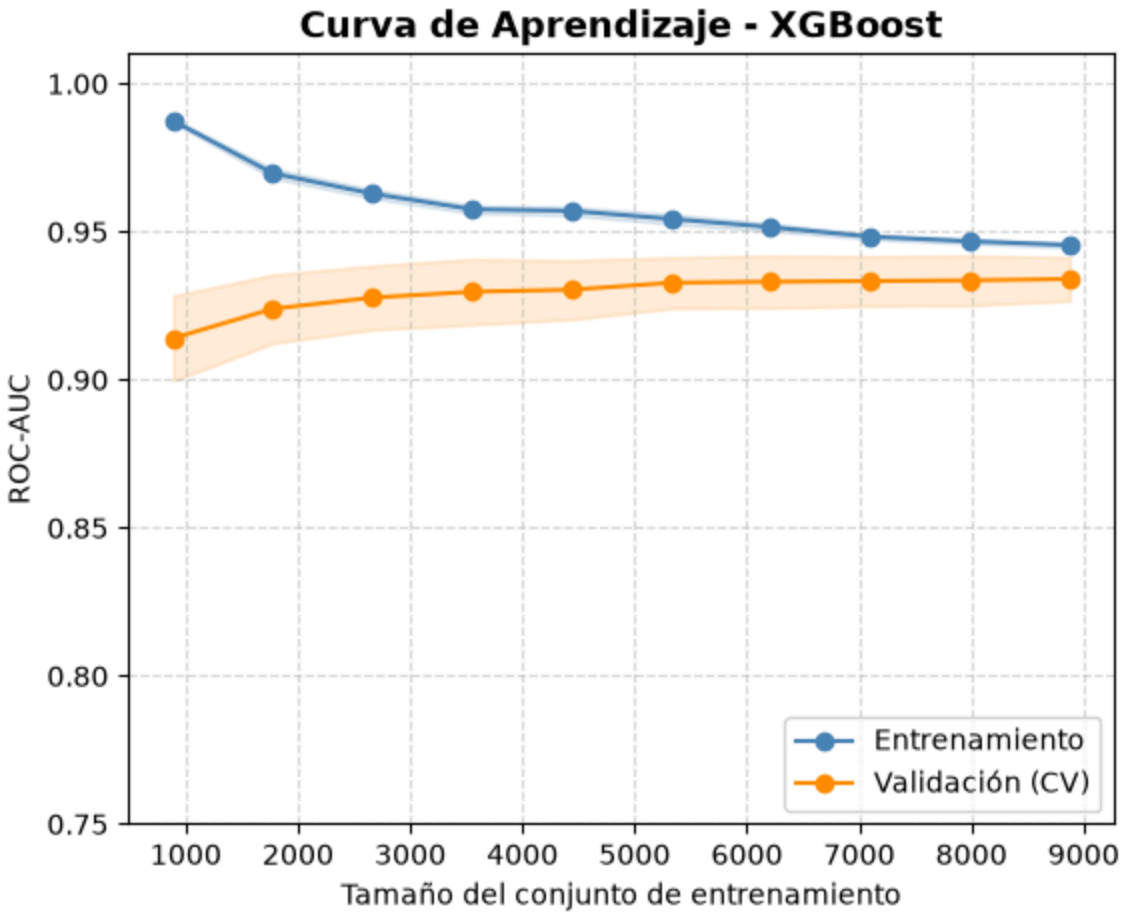
fig = plt.figure(figsize=(14, 5))
gs = gridspec.GridSpec(1, 2, figure=fig)

train_sizes, train_scores, val_scores = learning_curve(
    grid_xgb.best_estimator_,
    X_train, y_train,
    cv=cv_strategy,
    scoring='roc_auc',
    train_sizes=np.linspace(0.1, 1.0, 10),
    n_jobs=-1
)

train_mean = train_scores.mean(axis=1)
train_std  = train_scores.std(axis=1)
val_mean   = val_scores.mean(axis=1)
val_std    = val_scores.std(axis=1)

ax1 = fig.add_subplot(gs[0])
ax1.plot(train_sizes, train_mean, 'o-', color='steelblue', label='Entrenamiento')
ax1.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha
ax1.plot(train_sizes, val_mean, 'o-', color='darkorange', label='Validación (CV)')
ax1.fill_between(train_sizes, val_mean - val_std, val_mean + val_std, alpha=0.15, c
ax1.set_title('Curva de Aprendizaje - XGBoost', fontsize=13, fontweight='bold')
ax1.set_xlabel('Tamaño del conjunto de entrenamiento')
ax1.set_ylabel('ROC-AUC')
ax1.legend(loc='lower right')
ax1.grid(True, linestyle='--', alpha=0.5)
ax1.set_ylim(0.75, 1.01)
```

Out[50]:
```
(0.75, 1.01)
```



MATRIZ DE CONFUSIÓN

In [51]:

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import learning_curve

fig = plt.figure(figsize=(6, 5))
gs = gridspec.GridSpec(1, 1, figure=fig)

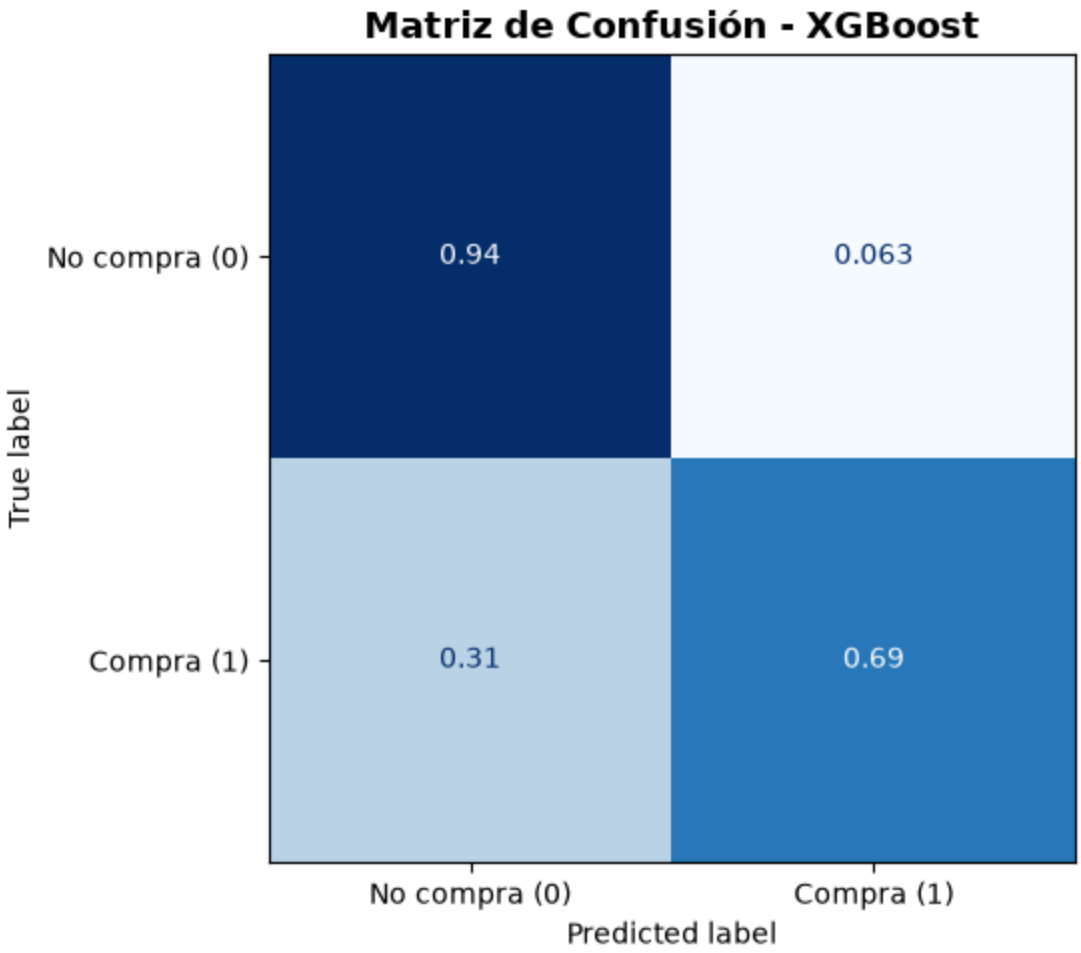
cm = confusion_matrix(y_test, y_pred_xgb, normalize="true")
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No compra (0)',

ax2 = fig.add_subplot(gs[0])
disp.plot(ax=ax2, colorbar=False, cmap='Blues')
ax2.set_title('Matriz de Confusión - XGBoost', fontsize=13, fontweight='bold')

plt.tight_layout()
plt.savefig('xgb_curva_confusion.png', dpi=150, bbox_inches='tight')
plt.show()

# ------------------------------------------------------------
# 3. RESUMEN NUMÉRICO
# ------------------------------------------------------------
print(f"\nROC-AUC promedio CV : {val_mean[-1]:.4f} ± {val_std[-1]:.4f}")
print(f"ROC-AUC en test     : {roc_auc_score(y_test, y_pred_prob_xgb):.4f}")
```

```
print(f"\nVerdaderos Negativos : {cm[0,0]}  |  Falsos Positivos : {cm[0,1]}")
print(f"Falsos Negativos     : {cm[1,0]}  |  Verdaderos Positivos: {cm[1,1]}")
```




```
ROC-AUC promedio CV : 0.9340 ± 0.0074
ROC-AUC en test     : 0.9308

Verdaderos Negativos : 0.9366602687140115  |  Falsos Positivos : 0.06333973128598848
Falsos Negativos     : 0.31413612565445026  |  Verdaderos Positivos: 0.6858638743455
497
```

MLP

In [52]:
```
pipeline_mlp = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('classifier', MLPClassifier(
        max_iter=1000,
        random_state=42
    ))
])
```

In [53]:
```
param_grid_mlp = {

    'smote__sampling_strategy':[0.22,0.3],
    'smote__k_neighbors':[5],
```

```
    'classifier__hidden_layer_sizes':[
        (50,),
        (100,)
    ],

    'classifier__activation':[
        'relu'
    ],

    'classifier__alpha':[
        0.0001,
        0.001
    ],

    'classifier__learning_rate_init':[
        0.001,
        0.01
    ]
}
```

In [54]:
```
grid_mlp = GridSearchCV(
    estimator=pipeline_mlp,
    param_grid=param_grid_mlp,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=0
)
```

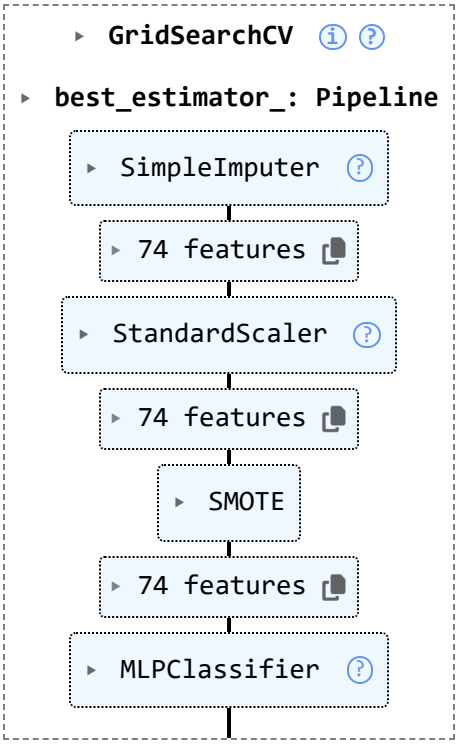
In [55]:
```
from sklearn.impute import SimpleImputer
from imblearn.pipeline import Pipeline

# Redefinimos el pipeline de MLP para incluir imputación y evitar problemas con NaN
pipeline_mlp = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),  # Corrige los NaN causados por
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('classifier', MLPClassifier(
        max_iter=1000,
        random_state=42
    ))
])

# Actualizamos el estimador en grid_mlp
grid_mlp.estimator = pipeline_mlp

# Ajustar el modelo de forma segura
grid_mlp.fit(X_train, y_train)
```

Out[55]:



Resultados de los 3 mejores pipelines

In [56]:

```python
# OBTENER LOS 3 MEJORES PIPELINES DE MLP INCLUYENDO IMPUTADOR
from sklearn.impute import SimpleImputer
from sklearn.metrics import balanced_accuracy_score, roc_auc_score, f1_score
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
import numpy as np

# Asegurar existencia y ajuste del grid_mlp en caso de pérdida de estado del kernel
if 'grid_mlp' not in globals() or not hasattr(grid_mlp, 'cv_results_'):
    print("Re-entrenando grid_mlp para recuperar el estado...")
    pipeline_mlp = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42)),
        ('classifier', MLPClassifier(max_iter=1000, random_state=42))
    ])
    param_grid_mlp = {
        'smote__sampling_strategy': [0.22, 0.3],
        'smote__k_neighbors': [5],
        'classifier__hidden_layer_sizes': [(50,), (100,)],
        'classifier__activation': ['relu'],
        'classifier__alpha': [0.0001, 0.001],
        'classifier__learning_rate_init': [0.001, 0.01]
    }
    grid_mlp = GridSearchCV(
        estimator=pipeline_mlp,
```

```
        param_grid=param_grid_mlp,
        cv=cv_strategy,
        scoring='balanced_accuracy',
        n_jobs=-1,
        verbose=0
    )
    grid_mlp.fit(X_train, y_train)

n_top = 3
cv_results_mlp = grid_mlp.cv_results_
top_indices_mlp = np.argsort(cv_results_mlp['mean_test_score'])[-n_top:][::-1]

print("\n" + "=" * 80)
print("TOP 3 PIPELINES MLP SEGÚN BALANCED_ACCURACY (CV)")
print("=" * 80)

for i, idx in enumerate(top_indices_mlp, 1):
    params = cv_results_mlp['params'][idx]
    balanced_acc_cv_mean = cv_results_mlp['mean_test_score'][idx]
    balanced_acc_cv_std = cv_results_mlp['std_test_score'][idx]

    # Reconstruir pipeline con estos parámetros e incluir Imputación para evitar er
    pipeline_temp_mlp = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42, sampling_strategy=params['smote__sampling_
        ('classifier', MLPClassifier(
            max_iter=1000,
            random_state=42,
            hidden_layer_sizes=params['classifier__hidden_layer_sizes'],
            activation=params['classifier__activation'],
            alpha=params['classifier__alpha'],
            learning_rate_init=params['classifier__learning_rate_init']
        ))
    ])

    # Entrenar y evaluar de forma segura
    pipeline_temp_mlp.fit(X_train, y_train)
    y_train_pred = pipeline_temp_mlp.predict(X_train)
    y_test_pred = pipeline_temp_mlp.predict(X_test)
    y_test_prob = pipeline_temp_mlp.predict_proba(X_test)[:, 1]

    balanced_acc_train = balanced_accuracy_score(y_train, y_train_pred)
    auc_test = roc_auc_score(y_test, y_test_prob)
    f1_test = f1_score(y_test, y_test_pred)

    print(f"\n{'='*80}")
    print(f"PIPELINE MLP #{i}")
    print(f"{'='*80}")
    print("HIPERPARÁMETROS:")
    for param, value in params.items():
        print(f"  {param}: {value}")

    print(f"\nBALANCED_ACCURACY:")
    print(f"  Train: {balanced_acc_train:.4f}")
    print(f"  Validation (CV mean): {balanced_acc_cv_mean:.4f} (±{balanced_acc_cv_s
```

```
    print(f"\nTEST:")
    print(f"  ROC-AUC: {auc_test:.4f}")
    print(f"  F1-Score: {f1_test:.4f}")
    print()
```

```
================================================================================
TOP 3 PIPELINES MLP SEGÚN BALANCED_ACCURACY (CV)
================================================================================

================================================================================
PIPELINE MLP #1
================================================================================
HIPERPARÁMETROS:
  classifier__activation: relu
  classifier__alpha: 0.001
  classifier__hidden_layer_sizes: (50,)
  classifier__learning_rate_init: 0.001
  smote__k_neighbors: 5
  smote__sampling_strategy: 0.3

BALANCED_ACCURACY:
  Train: 0.9533
  Validation (CV mean): 0.7506 (±0.0207)

TEST:
  ROC-AUC: 0.8458
  F1-Score: 0.5459


================================================================================
PIPELINE MLP #2
================================================================================
HIPERPARÁMETROS:
  classifier__activation: relu
  classifier__alpha: 0.001
  classifier__hidden_layer_sizes: (100,)
  classifier__learning_rate_init: 0.01
  smote__k_neighbors: 5
  smote__sampling_strategy: 0.3

BALANCED_ACCURACY:
  Train: 0.9823
  Validation (CV mean): 0.7496 (±0.0211)

TEST:
  ROC-AUC: 0.8514
  F1-Score: 0.5654


================================================================================
PIPELINE MLP #3
================================================================================
HIPERPARÁMETROS:
  classifier__activation: relu
  classifier__alpha: 0.0001
  classifier__hidden_layer_sizes: (50,)
  classifier__learning_rate_init: 0.001
  smote__k_neighbors: 5
  smote__sampling_strategy: 0.3

BALANCED_ACCURACY:
```

```
  Train: 0.9679
  Validation (CV mean): 0.7457 (±0.0164)

TEST:
  ROC-AUC: 0.8444
  F1-Score: 0.5467
```

Mejores hiperparámetros

In [57]:
```
grid_mlp.best_score_
```

Out[57]:
```
np.float64(0.7506051186724847)
```

In [58]:
```
print(grid_mlp.best_params_)
```

```
{'classifier__activation': 'relu', 'classifier__alpha': 0.001, 'classifier__hidden_l
ayer_sizes': (50,), 'classifier__learning_rate_init': 0.001, 'smote__k_neighbors':
5, 'smote__sampling_strategy': 0.3}
```

Intervalos de confianza del Cross-Validation

In [59]:
```
from sklearn.model_selection import cross_val_score, GridSearchCV, StratifiedKFold,
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
import pandas as pd
import numpy as np

# 1. Asegurar la existencia de X_train e y_train recreando el flujo si es necesario
if 'X_train' not in globals() or 'y_train' not in globals():
    print("X_train o y_train no definidos. Re-importando y procesando el dataset de
    raw_url = "https://raw.githubusercontent.com/Jennyaorozco/online-shoppers-inten
    df = pd.read_csv(raw_url)

    X = df.drop(columns=["Revenue"])
    y = df[["Revenue"]].copy()

    # Codificación
    X = pd.get_dummies(X, columns=['OperatingSystems'], prefix="OperatingSytem_", d
    X = pd.get_dummies(X, columns=['Browser'], prefix="Browser_", dtype=int)
    X = pd.get_dummies(X, columns=['Region'], prefix="Region_", dtype=int)
    X = pd.get_dummies(X, columns=['TrafficType'], prefix="TrafficType_", dtype=int
    X = pd.get_dummies(X, columns=['VisitorType'], prefix="VisitorType_", dtype=int

    X['Month'] = X['Month'].map({
        'Feb': 1, 'Mar': 2, 'May': 3, 'June': 4, 'Jul': 5,
        'Aug': 6, 'Sep': 7, 'Oct': 8, 'Nov': 9, 'Dec': 10
    })
    X['Weekend'] = X['Weekend'].astype(int)
    y['Revenue'] = y['Revenue'].astype(int)

    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.2, stratify=y, random_state=42
    )
    y_train = y_train.values.ravel()

# 2. Asegurar que cv_strategy esté definido
if 'cv_strategy' not in globals():
    cv_strategy = StratifiedKFold(
        n_splits=10,
        shuffle=True,
        random_state=42
    )

# 3. Asegurar que grid_mlp esté definido y entrenado
if 'grid_mlp' not in globals() or not hasattr(grid_mlp, 'best_estimator_'):
    print("grid_mlp no estaba definido o entrenado. Entrenando el modelo MLP...")
    pipeline_mlp = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42)),
        ('classifier', MLPClassifier(max_iter=1000, random_state=42))
    ])
    param_grid_mlp = {
        'smote__sampling_strategy': [0.22, 0.3],
        'smote__k_neighbors': [5],
        'classifier__hidden_layer_sizes': [(50,), (100,)],
        'classifier__activation': ['relu'],
        'classifier__alpha': [0.0001, 0.001],
        'classifier__learning_rate_init': [0.001, 0.01]
    }
    grid_mlp = GridSearchCV(
        estimator=pipeline_mlp,
        param_grid=param_grid_mlp,
        cv=cv_strategy,
        scoring='balanced_accuracy',
        n_jobs=-1,
        verbose=0
    )
    grid_mlp.fit(X_train, y_train)

# 4. Realizar la validación cruzada con el mejor estimador de MLP
scores_mlp = cross_val_score(
    grid_mlp.best_estimator_,
    X_train,
    y_train,
    cv=cv_strategy,
    scoring='roc_auc'
)

print(f"\nROC-AUC por fold Cross-Validation:")
print(np.round(scores_mlp, 4))
print(f"Media:  {scores_mlp.mean():.4f}")
print(f"Std:    {scores_mlp.std():.4f}")
```

```
ROC-AUC por fold Cross-Validation:
[0.8334 0.8334 0.8496 0.8733 0.8153 0.8718 0.8621 0.8554 0.8669 0.8658]
Media:  0.8527
Std:    0.0185
```

Resultados en test

In [60]:
```python
best_mlp = grid_mlp.best_estimator_

best_mlp.fit(X_train,y_train)

y_pred_prob_mlp = best_mlp.predict_proba(X_test)[:,1]
y_pred_mlp = best_mlp.predict(X_test)

print("ROC-AUC TEST:")
print(roc_auc_score(y_test,y_pred_prob_mlp))

print(classification_report(y_test,y_pred_mlp))
print(confusion_matrix(y_test,y_pred_mlp))
```

```
ROC-AUC TEST:
0.8457519771683533
              precision    recall  f1-score   support

           0       0.91      0.93      0.92      2084
           1       0.56      0.53      0.55       382

    accuracy                           0.86      2466
   macro avg       0.74      0.73      0.73      2466
weighted avg       0.86      0.86      0.86      2466

[[1928  156]
 [ 180  202]]
```

Resultados en train

In [61]:
```python
best_mlp.fit(X_train, y_train)
y_pred_prob_train_mlp = best_mlp.predict_proba(X_train)[:, 1]
y_pred_train_mlp = best_mlp.predict(X_train)
print("ROC-AUC TRAIN:")
print(roc_auc_score(y_train, y_pred_prob_train_mlp))
print(classification_report(y_train, y_pred_train_mlp))
```

```
ROC-AUC TRAIN:
0.9962505662621854
              precision    recall  f1-score   support

           0       0.98      0.99      0.99      8338
           1       0.95      0.92      0.93      1526

    accuracy                           0.98      9864
   macro avg       0.97      0.95      0.96      9864
weighted avg       0.98      0.98      0.98      9864
```

In [62]:
```
# === INTERVALOS DE CONFIANZA BOOTSTRAP para test ===
from sklearn.utils import resample

def bootstrap_ci(y_true, y_pred, y_prob, n_bootstrap=1000, alpha=0.05):
    """Calcula IC 95% por bootstrap para ROC-AUC y F1."""
    rng = np.random.RandomState(42)
    aucs, f1s = [], []
    for _ in range(n_bootstrap):
        idx = resample(range(len(y_true)), random_state=rng)
        if len(np.unique(np.array(y_true)[idx])) < 2:
            continue
        aucs.append(roc_auc_score(np.array(y_true)[idx], np.array(y_prob)[idx]))
        f1s.append(f1_score(np.array(y_true)[idx], np.array(y_pred)[idx]))
    lo, hi = alpha/2, 1 - alpha/2
    return {
        'ROC-AUC': (np.percentile(aucs, lo*100), np.mean(aucs), np.percentile(aucs,
        'F1':      (np.percentile(f1s,  lo*100), np.mean(f1s),  np.percentile(f1s,
    }

ci = bootstrap_ci(y_test, y_pred_mlp, y_pred_prob_mlp)
print(f"ROC-AUC test: {ci['ROC-AUC'][1]:.4f} "
      f"(IC 95%: [{ci['ROC-AUC'][0]:.4f}, {ci['ROC-AUC'][2]:.4f}])")
print(f"F1 test:      {ci['F1'][1]:.4f} "
      f"(IC 95%: [{ci['F1'][0]:.4f}, {ci['F1'][2]:.4f}])")
```

```
ROC-AUC test: 0.8454 (IC 95%: [0.8233, 0.8662])
F1 test:      0.5454 (IC 95%: [0.5015, 0.5865])
```

Nombres de clases en el classification_report

In [63]:
```
print(classification_report(
    y_test, y_pred_mlp,
    target_names=['No compra (0)', 'Compra (1)']
))
```

```
              precision    recall  f1-score   support

No compra (0)       0.91      0.93      0.92      2084
   Compra (1)       0.56      0.53      0.55       382

     accuracy                           0.86      2466
    macro avg       0.74      0.73      0.73      2466
 weighted avg       0.86      0.86      0.86      2466
```

Matriz de confusión

In [64]:
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# MATRIZ DE CONFUSIÓN NORMALIZADA EN TEST

# predicciones del mejor modelo
y_test_pred = best_mlp.predict(X_test)

# matriz de confusión normalizada
```

```
cm = confusion_matrix(
    y_test,
    y_test_pred,
    normalize='true'
)

# visualización
disp = ConfusionMatrixDisplay(confusion_matrix=cm)

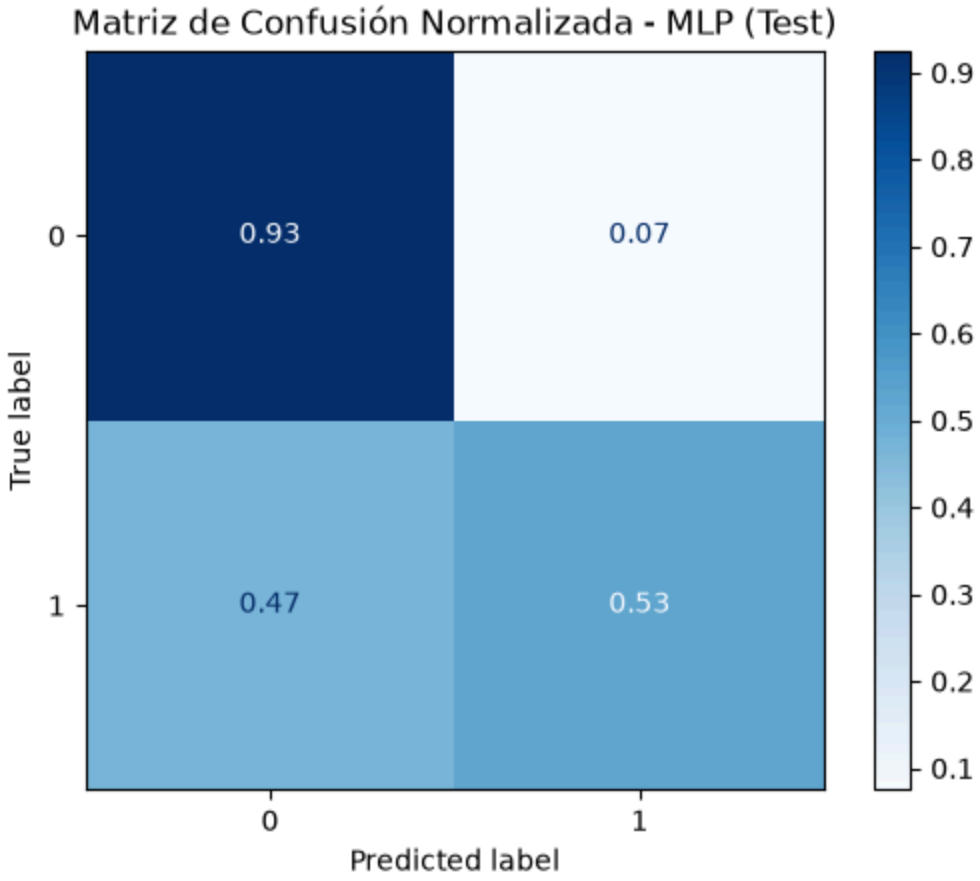
plt.figure(figsize=(6, 6))

disp.plot(
    cmap='Blues',
    values_format='.2f'
)

plt.title('Matriz de Confusión Normalizada - MLP (Test)')

plt.show()
```

```
<Figure size 600x600 with 0 Axes>
```



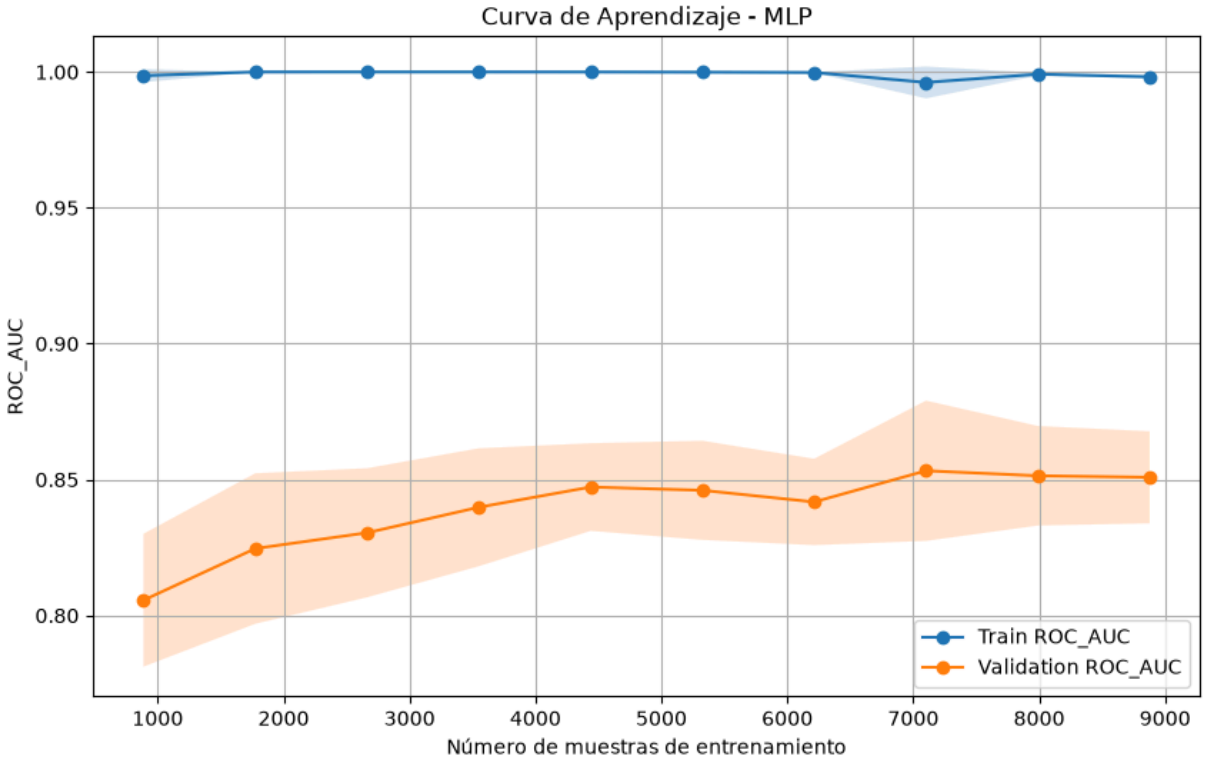
Curva de aprendizaje

In [65]:
```
train_sizes, train_scores, val_scores = learning_curve(
    estimator=best_mlp,
    X=X_train,
    y=y_train,
    cv=cv_strategy,
```

```
    scoring='roc_auc',
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10)
)

train_mean = train_scores.mean(axis=1)
train_std  = train_scores.std(axis=1)
val_mean   = val_scores.mean(axis=1)
val_std    = val_scores.std(axis=1)

plt.figure(figsize=(10, 6))
plt.plot(
    train_sizes,
    train_mean,
    marker='o',
    label='Train ROC_AUC'
)
plt.plot(
    train_sizes,
    val_mean,
    marker='o',
    label='Validation ROC_AUC'
)
plt.fill_between(
    train_sizes,
    train_mean - train_std,
    train_mean + train_std,
    alpha=0.2
)
plt.fill_between(
    train_sizes,
    val_mean - val_std,
    val_mean + val_std,
    alpha=0.2
)
plt.xlabel('Número de muestras de entrenamiento')
plt.ylabel('ROC_AUC')
plt.title('Curva de Aprendizaje - MLP')
plt.legend()
plt.grid(True)
plt.show()
```



K-means

Implementación de K-Means como modelo de clasificación mediante asignación por clase mayoritaria

In [66]:
```
from sklearn.cluster import KMeans
from sklearn.base import BaseEstimator, ClassifierMixin
import numpy as np

class KMeansClassifier(ClassifierMixin, BaseEstimator):

    _estimator_type = "classifier"

    def __init__(self, n_clusters=2, random_state=42):
        self.n_clusters = n_clusters
        self.random_state = random_state

    def fit(self, X, y):

        y = np.asarray(y).astype(int)

        self.kmeans_ = KMeans(
            n_clusters=self.n_clusters,
            random_state=self.random_state,
            n_init=10
        )

        clusters = self.kmeans_.fit_predict(X)

        self.cluster_labels_ = {}
```

```
        for c in np.unique(clusters):

            mask = (clusters == c)

            self.cluster_labels_[c] = np.bincount(
                y[mask]
            ).argmax()

        self.classes_ = np.array([0,1])

        return self

    def predict(self, X):

        clusters = self.kmeans_.predict(X)

        return np.array([
            self.cluster_labels_[c]
            for c in clusters
        ])

    def predict_proba(self, X):
        """Probabilidad basada en distancia inversa al centroide."""
        distances = self.kmeans_.transform(X)  # shape (n, n_clusters)
        # Convertir distancias a scores por clase
        probs = np.zeros((len(X), 2))
        for c in np.unique(list(self.cluster_labels_.values())):
            # Sumar 1/distancia de todos los clusters asignados a esta clase
            for cluster_id, label in self.cluster_labels_.items():
                if label == c:
                    inv_dist = 1.0 / (distances[:, cluster_id] + 1e-8)
                    probs[:, c] += inv_dist
        # Normalizar
        row_sums = probs.sum(axis=1, keepdims=True)
        probs = probs / (row_sums + 1e-8)
        return probs
```

In [67]:
```
pipeline_kmeans = Pipeline([

    ('scaler', StandardScaler()),

    ('smote', SMOTE(random_state=42)),

    ('classifier', KMeansClassifier())
])
```

In [68]:
```
param_grid_kmeans = {

    'smote__sampling_strategy': [
        0.22,
        0.3,
        0.4
    ],
```

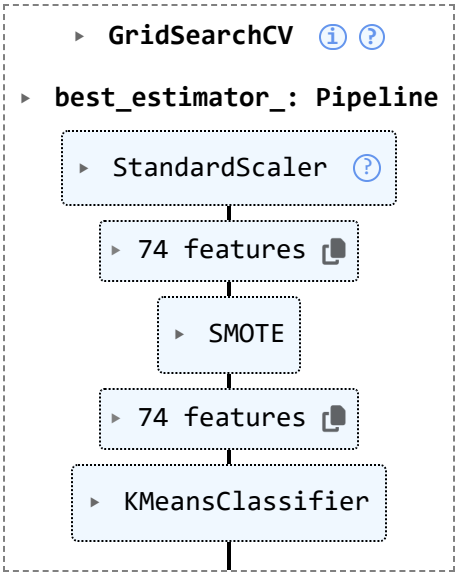
```
    'smote__k_neighbors': [
        3,
        5,
        7
    ],

    'classifier__n_clusters': [
        2,
        3,
        4,
        5
    ]
}
```

In [69]:
```
grid_kmeans = GridSearchCV(
    estimator=pipeline_kmeans,
    param_grid=param_grid_kmeans,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=0,
    error_score='raise'
)
```

In [70]:
```
grid_kmeans.fit(X_train,y_train)
```

Out[70]:



Resultados para los 3 mejores pipelines

In [71]:
```
# OBTENER LOS 3 MEJORES PIPELINES DE KMEANS
cv_results_kmeans = grid_kmeans.cv_results_
top_indices_kmeans = np.argsort(cv_results_kmeans['mean_test_score'])[-3:][::-1]

print("\n" + "=" * 80)
print("TOP 3 PIPELINES KMEANS SEGÚN BALANCED_ACCURACY (CV)")
print("=" * 80)
```

```python
for i, idx in enumerate(top_indices_kmeans, 1):
    params = cv_results_kmeans['params'][idx]
    balanced_acc_cv_mean = cv_results_kmeans['mean_test_score'][idx]
    balanced_acc_cv_std = cv_results_kmeans['std_test_score'][idx]

    # Reconstruir pipeline con estos parámetros
    pipeline_temp_kmeans = Pipeline([
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42, sampling_strategy=params['smote__sampling_
        ('classifier', KMeansClassifier(
            n_clusters=params['classifier__n_clusters'],
            random_state=42
        ))
    ])

    # Entrenar y evaluar
    pipeline_temp_kmeans.fit(X_train, y_train)
    y_train_pred = pipeline_temp_kmeans.predict(X_train)
    y_test_pred = pipeline_temp_kmeans.predict(X_test)
    y_test_prob = pipeline_temp_kmeans.predict_proba(X_test)[:, 1]

    balanced_acc_train = balanced_accuracy_score(y_train, y_train_pred)
    auc_test = roc_auc_score(y_test, y_test_prob)
    f1_test = f1_score(y_test, y_test_pred)

    print(f"\n{'='*80}")
    print(f"PIPELINE KMEANS #{i}")
    print(f"{'='*80}")
    print("HIPERPARÁMETROS:")
    for param, value in params.items():
        print(f"  {param}: {value}")

    print(f"\nBALANCED_ACCURACY:")
    print(f"  Train: {balanced_acc_train:.4f}")
    print(f"  Validation (CV): {balanced_acc_cv_mean:.4f} (±{balanced_acc_cv_std:.4

    print(f"\nTEST:")
    print(f"  ROC-AUC: {auc_test:.4f}")
    print(f"  F1-Score: {f1_test:.4f}")
    print()

# MEJOR MODELO KMEANS
print("\n" + "=" * 60)
print("MEJOR MODELO KMEANS (GridSearchCV)")
print("=" * 60)
print(f"Mejor balanced_accuracy (CV): {grid_kmeans.best_score_:.4f}")
print("Mejores hiperparámetros:")
for param, value in grid_kmeans.best_params_.items():
    print(f"  {param}: {value}")

# Evaluación del mejor modelo en test
best_kmeans = grid_kmeans.best_estimator_
y_pred_prob_kmeans = best_kmeans.predict_proba(X_test)[:, 1]
y_pred_kmeans = best_kmeans.predict(X_test)

roc_auc_test = roc_auc_score(y_test, y_pred_prob_kmeans)
```

```
print(f"\nROC-AUC en TEST: {roc_auc_test:.4f}")

print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred_kmeans, target_names=['No compra (0)', '

print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred_kmeans))
```

```
================================================================================
TOP 3 PIPELINES KMEANS SEGÚN BALANCED_ACCURACY (CV)
================================================================================

================================================================================
PIPELINE KMEANS #1
================================================================================
HIPERPARÁMETROS:
  classifier__n_clusters: 4
  smote__k_neighbors: 7
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train: 0.5000
  Validation (CV): 0.5003 (±0.0010)

TEST:
  ROC-AUC: 0.5000
  F1-Score: 0.0000


================================================================================
PIPELINE KMEANS #2
================================================================================
HIPERPARÁMETROS:
  classifier__n_clusters: 5
  smote__k_neighbors: 7
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train: 0.5000
  Validation (CV): 0.5000 (±0.0000)

TEST:
  ROC-AUC: 0.5000
  F1-Score: 0.0000


================================================================================
PIPELINE KMEANS #3
================================================================================
HIPERPARÁMETROS:
  classifier__n_clusters: 5
  smote__k_neighbors: 7
  smote__sampling_strategy: 0.22

BALANCED_ACCURACY:
  Train: 0.5000
  Validation (CV): 0.5000 (±0.0000)

TEST:
  ROC-AUC: 0.5000
  F1-Score: 0.0000


============================================================
```

```
MEJOR MODELO KMEANS (GridSearchCV)
============================================================
Mejor balanced_accuracy (CV): 0.5003
Mejores hiperparámetros:
  classifier__n_clusters: 4
  smote__k_neighbors: 7
  smote__sampling_strategy: 0.4

ROC-AUC en TEST: 0.5000

Reporte de clasificación:
               precision    recall  f1-score   support

No compra (0)       0.85      1.00      0.92      2084
   Compra (1)       0.00      0.00      0.00       382

     accuracy                           0.85      2466
    macro avg       0.42      0.50      0.46      2466
 weighted avg       0.71      0.85      0.77      2466

Matriz de confusión:
[[2084    0]
 [ 382    0]]
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

Mejores hiperparámetros

In [72]:
```
grid_kmeans.best_score_
```

Out[72]:
```
np.float64(0.5003267973856209)
```

In [73]:
```
print(grid_kmeans.best_params_)
```

```
{'classifier__n_clusters': 4, 'smote__k_neighbors': 7, 'smote__sampling_strategy': 0.4}
```

Intervalos de confianza del Cross-Validation

In [74]:
```
scores_kmeans = cross_val_score(
    grid_kmeans.best_estimator_,
    X_train,
```

```
    y_train,
    cv=cv_strategy,
    scoring='roc_auc'
)
print(f"ROC-AUC por fold Cross-Validation:")
print(np.round(scores_kmeans, 4))
print(f"Media:  {scores_kmeans.mean():.4f}")
print(f"Std:    {scores_kmeans.std():.4f}")
```

```
ROC-AUC por fold Cross-Validation:
[0.5    0.5    0.5    0.5    0.5    0.5    0.5    0.5    0.5313 0.5   ]
Media:  0.5031
Std:    0.0094
```

Resultados en train

In [75]:

```
best_kmeans = grid_kmeans.best_estimator_

best_kmeans.fit(X_train,y_train)

y_pred_prob_kmeans = best_kmeans.predict_proba(X_test)[:,1]
y_pred_kmeans = best_kmeans.predict(X_test)

print("ROC-AUC TEST:")
print(
    roc_auc_score(
        y_test,
        y_pred_prob_kmeans
    )
)

print(
    classification_report(
        y_test,
        y_pred_kmeans
    )
)

print(
    confusion_matrix(
        y_test,
        y_pred_kmeans
    )
)
```

```
ROC-AUC TEST:
0.5
              precision    recall  f1-score   support

           0       0.85      1.00      0.92      2084
           1       0.00      0.00      0.00       382

    accuracy                           0.85      2466
   macro avg       0.42      0.50      0.46      2466
weighted avg       0.71      0.85      0.77      2466

[[2084    0]
 [ 382    0]]
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

In [76]:
```
y_pred_prob_train_kmeans = best_kmeans.predict_proba(X_train)[:, 1]
y_pred_train_kmeans = best_kmeans.predict(X_train)
print("ROC-AUC TRAIN:")
print(roc_auc_score(y_train, y_pred_prob_train_kmeans))
print(classification_report(y_train, y_pred_train_kmeans,
      target_names=['No compra (0)', 'Compra (1)']))
```

```
ROC-AUC TRAIN:
0.5
               precision    recall  f1-score   support

No compra (0)       0.85      1.00      0.92      8338
   Compra (1)       0.00      0.00      0.00      1526

     accuracy                           0.85      9864
    macro avg       0.42      0.50      0.46      9864
 weighted avg       0.71      0.85      0.77      9864
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

In [77]:
```python
# === INTERVALOS DE CONFIANZA BOOTSTRAP para test ===
from sklearn.utils import resample

def bootstrap_ci(y_true, y_pred, y_prob, n_bootstrap=1000, alpha=0.05):
    """Calcula IC 95% por bootstrap para ROC-AUC y F1."""
    rng = np.random.RandomState(42)
    aucs, f1s = [], []
    for _ in range(n_bootstrap):
        idx = resample(range(len(y_true)), random_state=rng)
        if len(np.unique(np.array(y_true)[idx])) < 2:
            continue
        aucs.append(roc_auc_score(np.array(y_true)[idx], np.array(y_prob)[idx]))
        f1s.append(f1_score(np.array(y_true)[idx], np.array(y_pred)[idx]))
    lo, hi = alpha/2, 1 - alpha/2
    return {
        'ROC-AUC': (np.percentile(aucs, lo*100), np.mean(aucs), np.percentile(aucs,
        'F1':      (np.percentile(f1s,  lo*100), np.mean(f1s),  np.percentile(f1s,
    }

ci = bootstrap_ci(y_test, y_pred_kmeans, y_pred_prob_kmeans)
print(f"ROC-AUC test: {ci['ROC-AUC'][1]:.4f} "
      f"(IC 95%: [{ci['ROC-AUC'][0]:.4f}, {ci['ROC-AUC'][2]:.4f}])")
print(f"F1 test:      {ci['F1'][1]:.4f} "
      f"(IC 95%: [{ci['F1'][0]:.4f}, {ci['F1'][2]:.4f}])")
```

```
ROC-AUC test: 0.5000 (IC 95%: [0.5000, 0.5000])
F1 test:      0.0000 (IC 95%: [0.0000, 0.0000])
```

Resultados en test

Nombres de clases en el classification_report

In [78]:
```python
print(classification_report(
    y_test,
    y_pred_kmeans,
    target_names=['No compra (0)', 'Compra (1)']
))
```

```
                precision    recall  f1-score   support

No compra (0)       0.85      1.00      0.92      2084
   Compra (1)       0.00      0.00      0.00       382

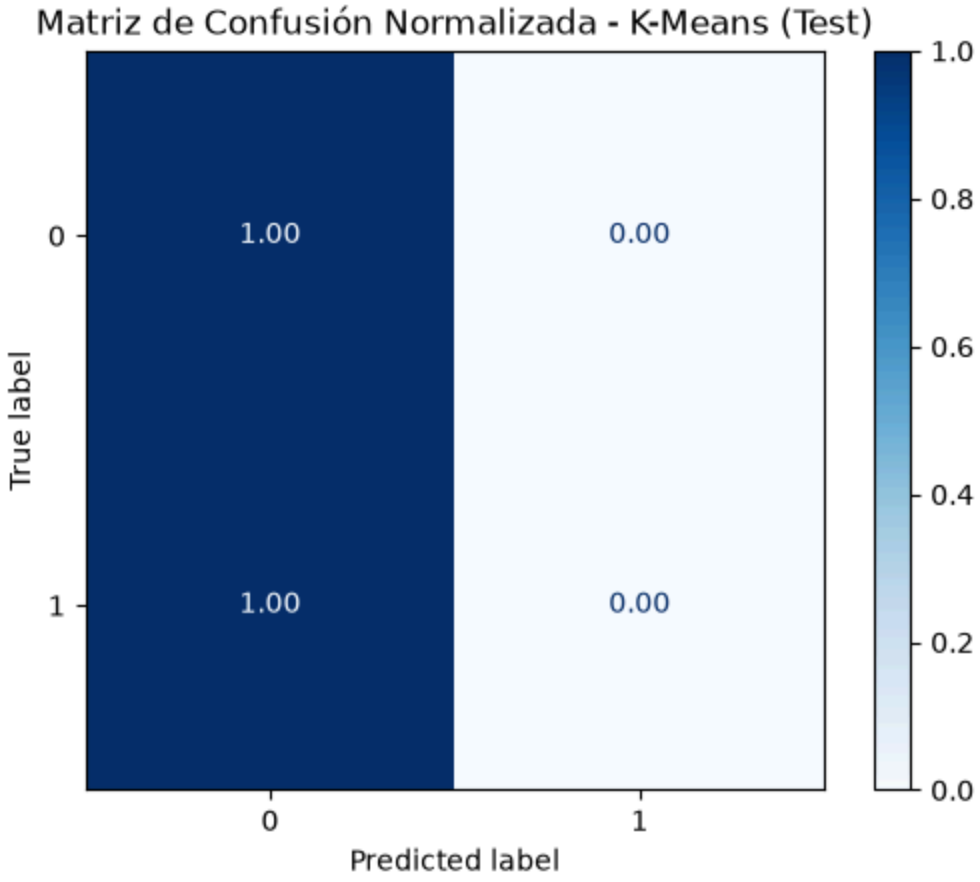
     accuracy                           0.85      2466
    macro avg       0.42      0.50      0.46      2466
 weighted avg       0.71      0.85      0.77      2466
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

Matriz de confusión

In [79]:
```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
# MATRIZ DE CONFUSIÓN NORMALIZADA EN TEST
# predicciones del mejor modelo
y_test_pred = best_kmeans.predict(X_test)
# matriz de confusión normalizada
cm = confusion_matrix(
    y_test,
    y_test_pred,
    normalize='true'
)
# visualización
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
plt.figure(figsize=(6, 6))
disp.plot(
    cmap='Blues',
    values_format='.2f'
)
plt.title('Matriz de Confusión Normalizada - K-Means (Test)')
plt.show()
```

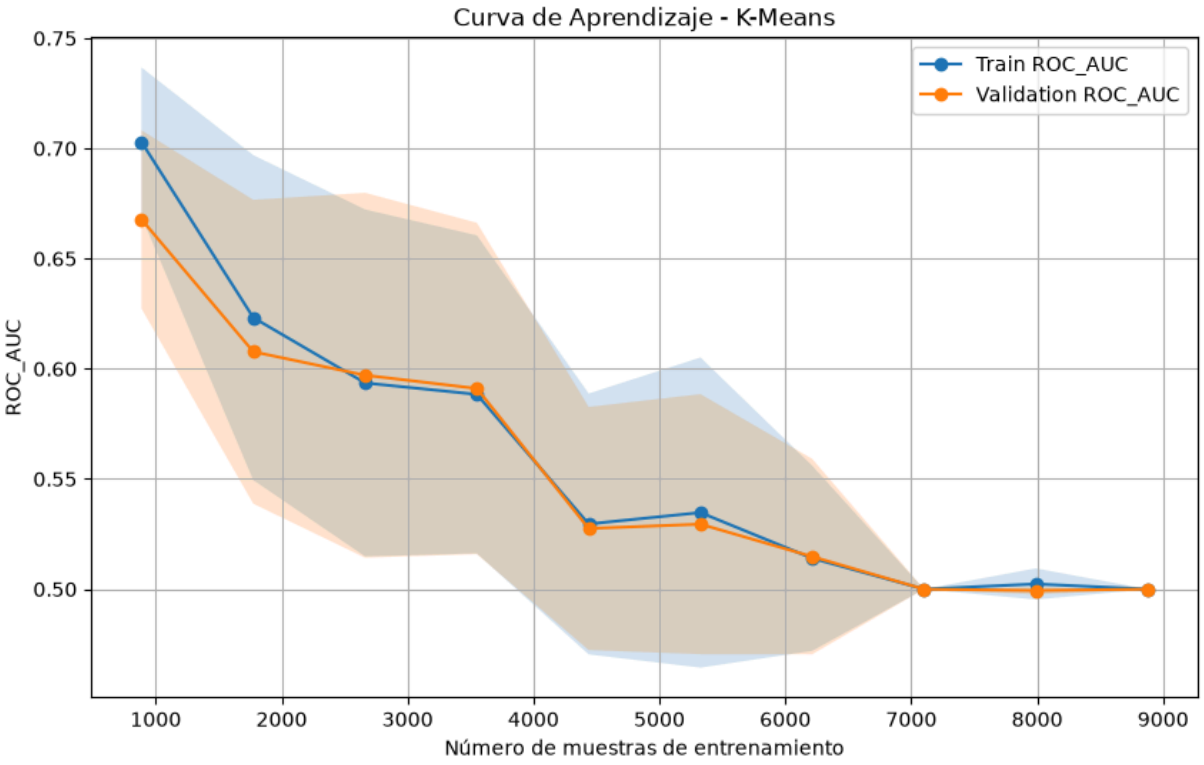
```
<Figure size 600x600 with 0 Axes>
```



Curva de aprendizaje

In [80]:
```
train_sizes, train_scores, val_scores = learning_curve(
    estimator=best_kmeans,
    X=X_train,
    y=y_train,
    cv=cv_strategy,
    scoring='roc_auc',
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10)
)
# medias
train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
val_mean = val_scores.mean(axis=1)
val_std = val_scores.std(axis=1)
plt.figure(figsize=(10, 6))
plt.plot(
    train_sizes,
    train_mean,
    marker='o',
    label='Train ROC_AUC'
)
plt.plot(
    train_sizes,
    val_mean,
    marker='o',
```

```
    label='Validation ROC_AUC'
)
plt.fill_between(
    train_sizes,
    train_mean - train_std,
    train_mean + train_std,
    alpha=0.2
)
plt.fill_between(
    train_sizes,
    val_mean - val_std,
    val_mean + val_std,
    alpha=0.2
)
plt.xlabel('Número de muestras de entrenamiento')
plt.ylabel('ROC_AUC')
plt.title('Curva de Aprendizaje - K-Means')
plt.legend()
plt.grid(True)
plt.show()
```



Regresión lineal

In [81]:
```
from sklearn.linear_model import LogisticRegression
from imblearn.pipeline import Pipeline

pipeline_lr = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('classifier', LogisticRegression(
        max_iter=1000,
```

```
        random_state=42,
        n_jobs=-1
    ))
])
```

In [82]:
```
param_grid_lr = {
    'smote__sampling_strategy': [0.22, 0.30, 0.40],
    'smote__k_neighbors': [3, 5],
    'classifier__C': [0.01, 0.1, 1, 10],
    'classifier__penalty': ['l2'],
    'classifier__solver': ['lbfgs']
}
```

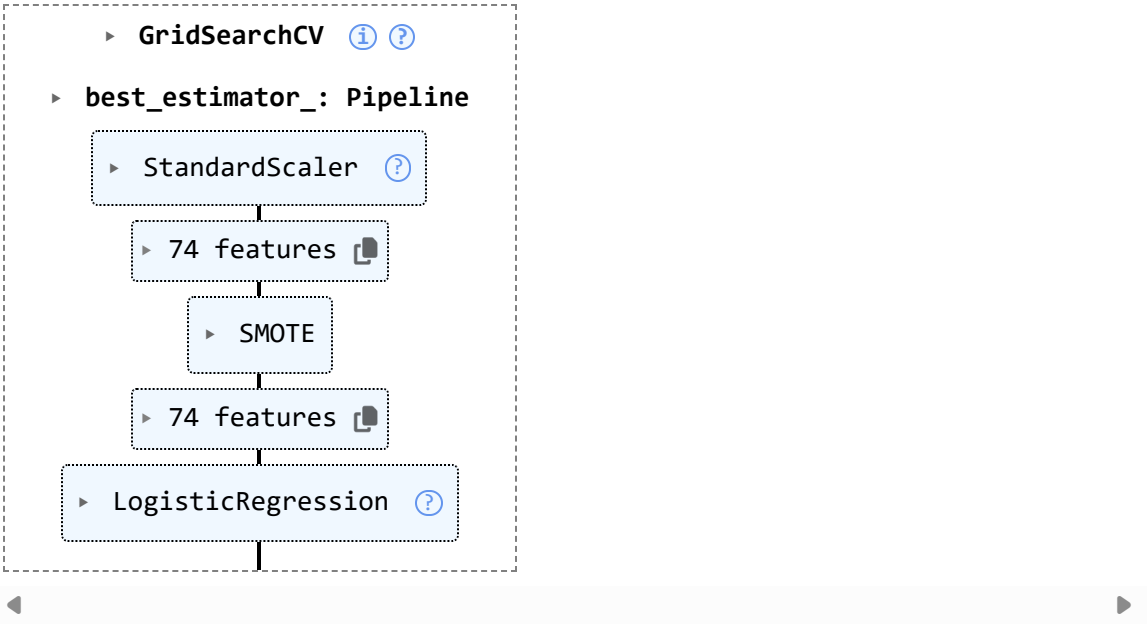
In [83]:
```
grid_lr = GridSearchCV(
    estimator=pipeline_lr,
    param_grid=param_grid_lr,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=0,
    refit=True
)
```

Entrenamiento

In [84]:
```
grid_lr.fit(X_train, y_train)
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:1381: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid this warning, leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of penalty='elasticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:1435: FutureWarning: 'n_jobs' has no effect since 1.8 and will be removed in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
```

Out[84]:



Resultados de los mejores 3 pipelines

In [85]:

```
cv_results_lr = grid_lr.cv_results_
top_indices_lr = np.argsort(cv_results_lr['mean_test_score'])[-3:][::-1]

print("\n" + "=" * 80)
print("TOP 3 PIPELINES LOGISTIC REGRESSION SEGÚN BALANCED_ACCURACY (CV)")
print("=" * 80)

for i, idx in enumerate(top_indices_lr, 1):
    params = cv_results_lr['params'][idx]
    balanced_acc_cv_mean = cv_results_lr['mean_test_score'][idx]
    balanced_acc_cv_std = cv_results_lr['std_test_score'][idx]

    pipeline_temp_lr = Pipeline([
        ('scaler', StandardScaler()),
        ('smote', SMOTE(
            random_state=42,
            sampling_strategy=params['smote__sampling_strategy'],
            k_neighbors=params['smote__k_neighbors']
        )),
        ('classifier', LogisticRegression(
            max_iter=1000,
            random_state=42,
            n_jobs=-1,
            C=params['classifier__C'],
            penalty=params['classifier__penalty'],
            solver=params['classifier__solver']
        ))
    ])

    pipeline_temp_lr.fit(X_train, y_train)
    y_train_pred = pipeline_temp_lr.predict(X_train)
    y_test_pred  = pipeline_temp_lr.predict(X_test)
    y_test_prob  = pipeline_temp_lr.predict_proba(X_test)[:, 1]
```

```
    balanced_acc_train = balanced_accuracy_score(y_train, y_train_pred)
    auc_test  = roc_auc_score(y_test, y_test_prob)
    f1_test   = f1_score(y_test, y_test_pred)

    print(f"\n{'='*80}")
    print(f"PIPELINE LR #{i}")
    print(f"{'='*80}")
    print("HIPERPARÁMETROS:")
    for param, value in params.items():
        print(f"  {param}: {value}")
    print(f"\nBALANCED_ACCURACY:")
    print(f"  Train:           {balanced_acc_train:.4f}")
    print(f"  Validation (CV): {balanced_acc_cv_mean:.4f} (±{balanced_acc_cv_std:.4
    print(f"\nTEST:")
    print(f"  ROC-AUC:  {auc_test:.4f}")
    print(f"  F1-Score: {f1_test:.4f}")
    print()
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_mod
el\_logistic.py:1381: FutureWarning: 'penalty' was deprecated in version 1.8 and wil
l be removed in 1.10. To avoid this warning, leave 'penalty' set to its default valu
e and use 'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_rati
o=1 instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of pena
lty='elasticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_mod
el\_logistic.py:1435: FutureWarning: 'n_jobs' has no effect since 1.8 and will be re
moved in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
================================================================================
TOP 3 PIPELINES LOGISTIC REGRESSION SEGÚN BALANCED_ACCURACY (CV)
================================================================================

================================================================================
PIPELINE LR #1
================================================================================
HIPERPARÁMETROS:
  classifier__C: 1
  classifier__penalty: l2
  classifier__solver: lbfgs
  smote__k_neighbors: 3
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train:           0.7649
  Validation (CV): 0.7599 (±0.0227)

TEST:
  ROC-AUC:  0.8852
  F1-Score: 0.6169
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:1381: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid this warning, leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of penalty='elasticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:1435: FutureWarning: 'n_jobs' has no effect since 1.8 and will be removed in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
================================================================================
PIPELINE LR #2
================================================================================
HIPERPARÁMETROS:
  classifier__C: 10
  classifier__penalty: l2
  classifier__solver: lbfgs
  smote__k_neighbors: 5
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train:           0.7610
  Validation (CV): 0.7599 (±0.0220)

TEST:
  ROC-AUC:  0.8819
  F1-Score: 0.6107


================================================================================
PIPELINE LR #3
================================================================================
HIPERPARÁMETROS:
  classifier__C: 10
  classifier__penalty: l2
  classifier__solver: lbfgs
  smote__k_neighbors: 3
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train:           0.7652
  Validation (CV): 0.7596 (±0.0235)

TEST:
  ROC-AUC:  0.8852
  F1-Score: 0.6169
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:1381: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid this warning, leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of penalty='elasticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:1435: FutureWarning: 'n_jobs' has no effect since 1.8 and will be removed in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
```

Resultados

In [86]:
```python
best_lr = grid_lr.best_estimator_
best_lr.fit(X_train, y_train)

y_pred_prob_lr = best_lr.predict_proba(X_test)[:, 1]
y_pred_lr      = best_lr.predict(X_test)

print("\n" + "=" * 60)
print(f"ROC-AUC EN TEST: {roc_auc_score(y_test, y_pred_prob_lr):.4f}")
print("=" * 60)
print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred_lr,
      target_names=['No compra (0)', 'Compra (1)']))
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:1381: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid this warning, leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of penalty='elasticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:1435: FutureWarning: 'n_jobs' has no effect since 1.8 and will be removed in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
============================================================
ROC-AUC EN TEST: 0.8852
============================================================

Reporte de clasificación:
              precision    recall  f1-score   support

No compra (0)       0.92      0.95      0.94      2084
   Compra (1)       0.68      0.56      0.62       382

     accuracy                           0.89      2466
    macro avg       0.80      0.76      0.78      2466
 weighted avg       0.89      0.89      0.89      2466
```

In [87]:
```python
# === INTERVALOS DE CONFIANZA BOOTSTRAP para test ===
from sklearn.utils import resample
```

```
def bootstrap_ci(y_true, y_pred, y_prob, n_bootstrap=1000, alpha=0.05):
    """Calcula IC 95% por bootstrap para ROC-AUC y F1."""
    rng = np.random.RandomState(42)
    aucs, f1s = [], []
    for _ in range(n_bootstrap):
        idx = resample(range(len(y_true)), random_state=rng)
        if len(np.unique(np.array(y_true)[idx])) < 2:
            continue
        aucs.append(roc_auc_score(np.array(y_true)[idx], np.array(y_prob)[idx]))
        f1s.append(f1_score(np.array(y_true)[idx], np.array(y_pred)[idx]))
    lo, hi = alpha/2, 1 - alpha/2
    return {
        'ROC-AUC': (np.percentile(aucs, lo*100), np.mean(aucs), np.percentile(aucs,
        'F1':      (np.percentile(f1s,  lo*100), np.mean(f1s),  np.percentile(f1s,
    }

ci = bootstrap_ci(y_test, y_pred_lr, y_pred_prob_lr)
print(f"ROC-AUC test: {ci['ROC-AUC'][1]:.4f} "
      f"(IC 95%: [{ci['ROC-AUC'][0]:.4f}, {ci['ROC-AUC'][2]:.4f}])")
print(f"F1 test:      {ci['F1'][1]:.4f} "
      f"(IC 95%: [{ci['F1'][0]:.4f}, {ci['F1'][2]:.4f}])")
```

```
ROC-AUC test: 0.8852 (IC 95%: [0.8667, 0.9041])
F1 test:      0.6173 (IC 95%: [0.5731, 0.6597])
```

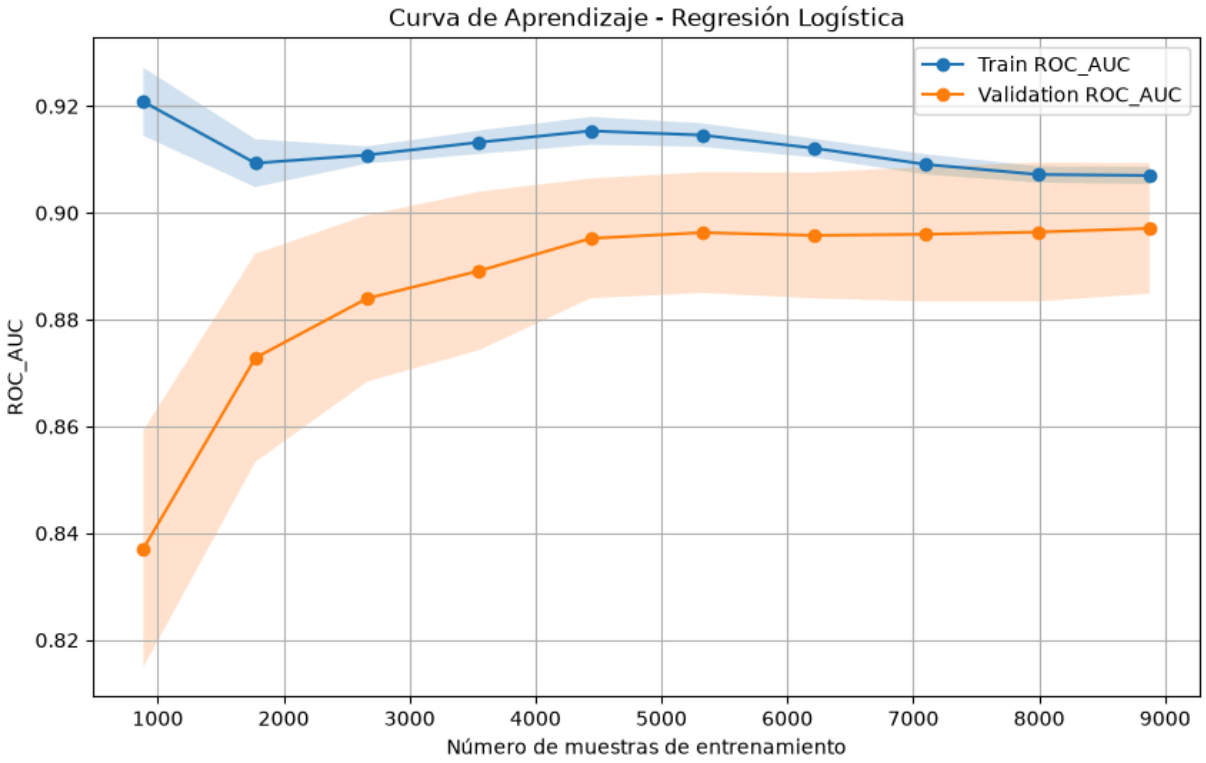
Curva de aprendizaje

In [88]:

```
train_sizes, train_scores, val_scores = learning_curve(
    estimator=best_lr,
    X=X_train, y=y_train,
    cv=cv_strategy,
    scoring='roc_auc',
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10)
)

train_mean = train_scores.mean(axis=1)
train_std  = train_scores.std(axis=1)
val_mean   = val_scores.mean(axis=1)
val_std    = val_scores.std(axis=1)

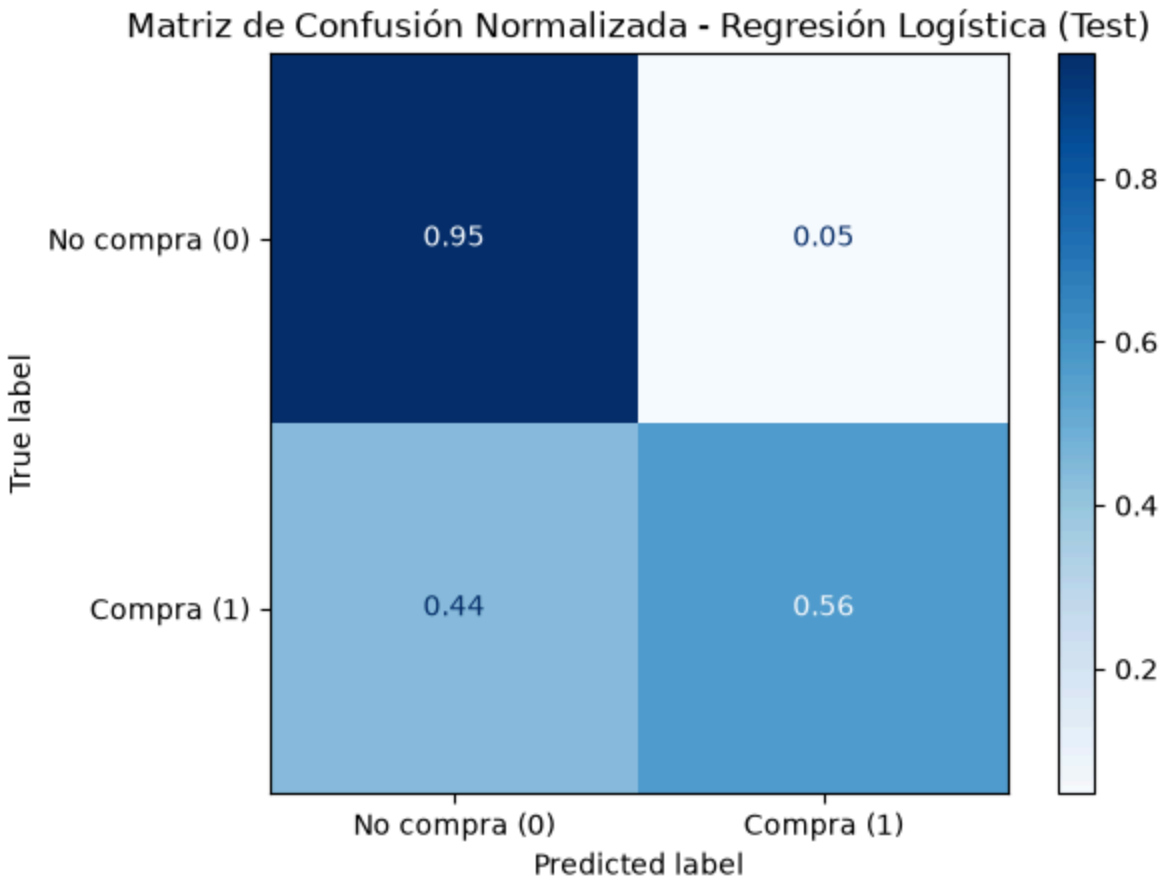
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, 'o-', label='Train ROC_AUC')
plt.plot(train_sizes, val_mean,   'o-', label='Validation ROC_AUC')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha
plt.fill_between(train_sizes, val_mean - val_std,     val_mean + val_std,     alpha
plt.xlabel('Número de muestras de entrenamiento')
plt.ylabel('ROC_AUC')
plt.title('Curva de Aprendizaje - Regresión Logística')
plt.legend()
plt.grid(True)
plt.savefig('lr_curva_aprendizaje.png', dpi=150, bbox_inches='tight')
plt.show()
```



Matriz de confusión

In [89]:
```
cm_lr = confusion_matrix(y_test, y_pred_lr, normalize='true')
disp  = ConfusionMatrixDisplay(confusion_matrix=cm_lr,
                               display_labels=['No compra (0)', 'Compra (1)'])
plt.figure(figsize=(6, 6))
disp.plot(cmap='Blues', values_format='.2f')
plt.title('Matriz de Confusión Normalizada - Regresión Logística (Test)')
plt.savefig('lr_confusion.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
<Figure size 600x600 with 0 Axes>
```



KNN

In [90]:
```
from sklearn.neighbors import KNeighborsClassifier

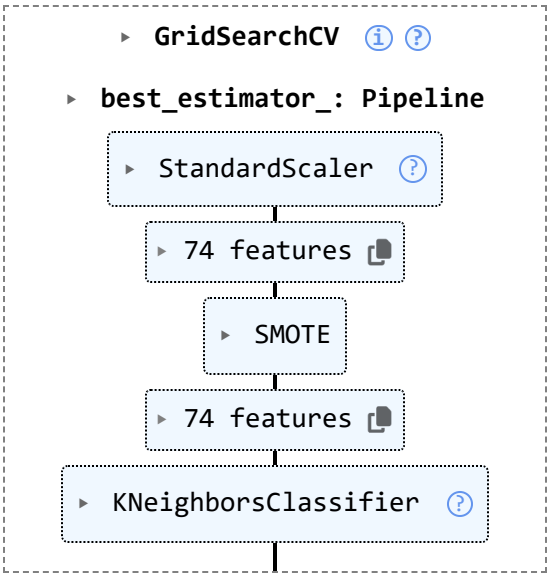
pipeline_knn = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42, k_neighbors=5)),
    ('classifier', KNeighborsClassifier())
])
```

In [91]:
```
param_grid_knn = {
    'smote__sampling_strategy': [0.22, 0.3, 0.4],
    'classifier__n_neighbors': [3, 5, 11],
    'classifier__weights': ['uniform', 'distance'],
    'classifier__metric': ['euclidean', 'manhattan']
}
```

In [92]:
```
grid_knn = GridSearchCV(
    estimator=pipeline_knn,
    param_grid=param_grid_knn,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=0
)
```

```
grid_knn.fit(X_train, y_train)
```

Out[92]:



Resultados de los 3 mejores pipelines

In [93]:

```
# OBTENER LOS 3 MEJORES PIPELINES DE KNN
cv_results_knn = grid_knn.cv_results_
top_indices_knn = np.argsort(cv_results_knn['mean_test_score'])[-3:][::-1]

print("\n" + "=" * 80)
print("TOP 3 PIPELINES KNN SEGÚN BALANCED_ACCURACY (CV)")
print("=" * 80)

for i, idx in enumerate(top_indices_knn, 1):
    params = cv_results_knn['params'][idx]
    balanced_acc_cv_mean = cv_results_knn['mean_test_score'][idx]
    balanced_acc_cv_std = cv_results_knn['std_test_score'][idx]

    # Reconstruir pipeline con estos parámetros
    pipeline_temp_knn = Pipeline([
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42,
                        sampling_strategy=params['smote__sampling_strategy'],
                        k_neighbors=5)),  # k_neighbors fijo para SMOTE
        ('classifier', KNeighborsClassifier(
            n_neighbors=params['classifier__n_neighbors'],
            weights=params['classifier__weights'],
            metric=params['classifier__metric']
        ))
    ])

    # Entrenar y evaluar
    pipeline_temp_knn.fit(X_train, y_train)
    y_train_pred = pipeline_temp_knn.predict(X_train)
    y_test_pred = pipeline_temp_knn.predict(X_test)
    y_test_prob = pipeline_temp_knn.predict_proba(X_test)[:, 1]
```

```
    balanced_acc_train = balanced_accuracy_score(y_train, y_train_pred)
    auc_test = roc_auc_score(y_test, y_test_prob)
    f1_test = f1_score(y_test, y_test_pred)

    print(f"\n{'='*80}")
    print(f"PIPELINE KNN #{i}")
    print(f"{'='*80}")
    print("HIPERPARÁMETROS:")
    for param, value in params.items():
        print(f"  {param}: {value}")

    print(f"\nBALANCED_ACCURACY:")
    print(f"  Train: {balanced_acc_train:.4f}")
    print(f"  Validation (CV): {balanced_acc_cv_mean:.4f} (±{balanced_acc_cv_std:.4

    print(f"\nTEST:")
    print(f"  ROC-AUC: {auc_test:.4f}")
    print(f"  F1-Score: {f1_test:.4f}")
    print()

# MEJOR MODELO KNN
print("\n" + "=" * 60)
print("MEJOR MODELO KNN (GridSearchCV)")
print("=" * 60)
print(f"Mejor balanced_accuracy (CV): {grid_knn.best_score_:.4f}")
print("Mejores hiperparámetros:")
for param, value in grid_knn.best_params_.items():
    print(f"  {param}: {value}")

# Evaluación del mejor modelo en test
best_knn = grid_knn.best_estimator_
y_pred_prob_knn = best_knn.predict_proba(X_test)[:, 1]
y_pred_knn = best_knn.predict(X_test)

roc_auc_test = roc_auc_score(y_test, y_pred_prob_knn)
print(f"\nROC-AUC en TEST: {roc_auc_test:.4f}")

print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred_knn, target_names=['No compra (0)', 'Com

print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred_knn))
```

```
================================================================================
TOP 3 PIPELINES KNN SEGÚN BALANCED_ACCURACY (CV)
================================================================================

================================================================================
PIPELINE KNN #1
================================================================================
HIPERPARÁMETROS:
  classifier__metric: euclidean
  classifier__n_neighbors: 11
  classifier__weights: distance
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train: 1.0000
  Validation (CV): 0.6537 (±0.0190)

TEST:
  ROC-AUC: 0.7475
  F1-Score: 0.4185


================================================================================
PIPELINE KNN #2
================================================================================
HIPERPARÁMETROS:
  classifier__metric: euclidean
  classifier__n_neighbors: 11
  classifier__weights: uniform
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train: 0.7516
  Validation (CV): 0.6494 (±0.0154)

TEST:
  ROC-AUC: 0.7424
  F1-Score: 0.4239


================================================================================
PIPELINE KNN #3
================================================================================
HIPERPARÁMETROS:
  classifier__metric: euclidean
  classifier__n_neighbors: 5
  classifier__weights: distance
  smote__sampling_strategy: 0.4

BALANCED_ACCURACY:
  Train: 1.0000
  Validation (CV): 0.6468 (±0.0201)

TEST:
  ROC-AUC: 0.6982
  F1-Score: 0.3884
```

```
============================================================
MEJOR MODELO KNN (GridSearchCV)
============================================================
Mejor balanced_accuracy (CV): 0.6537
Mejores hiperparámetros:
  classifier__metric: euclidean
  classifier__n_neighbors: 11
  classifier__weights: distance
  smote__sampling_strategy: 0.4

ROC-AUC en TEST: 0.7475

Reporte de clasificación:
               precision    recall  f1-score   support

No compra (0)       0.89      0.90      0.89      2084
   Compra (1)       0.42      0.41      0.42       382

     accuracy                           0.82      2466
    macro avg       0.66      0.66      0.66      2466
 weighted avg       0.82      0.82      0.82      2466

Matriz de confusión:
[[1869  215]
 [ 224  158]]
```

Mejores hiperparámetros

In [94]:
```
grid_knn.best_score_
```

Out[94]:
```
np.float64(0.6536764055615936)
```

In [95]:
```
print(grid_knn.best_params_)
```

```
{'classifier__metric': 'euclidean', 'classifier__n_neighbors': 11, 'classifier__weig
hts': 'distance', 'smote__sampling_strategy': 0.4}
```

Intervalos de confianza del Cross-Validation

In [96]:
```
scores_knn = cross_val_score(
    grid_knn.best_estimator_,
    X_train,
    y_train,
    cv=cv_strategy,
    scoring='roc_auc'
)
print(f"ROC-AUC por fold Cross-Validation:")
print(np.round(scores_knn, 4))
print(f"Media:  {scores_knn.mean():.4f}")
print(f"Std:    {scores_knn.std():.4f}")
```

```
ROC-AUC por fold Cross-Validation:
[0.7222 0.7493 0.7425 0.7515 0.7177 0.7697 0.7855 0.7638 0.7539 0.7322]
Media:  0.7488
Std:    0.0201
```

Resultados en train

In [97]:
```python
best_knn = grid_knn.best_estimator_
best_knn.fit(X_train, y_train)
y_pred_prob_train_knn = best_knn.predict_proba(X_train)[:, 1]
y_pred_train_knn = best_knn.predict(X_train)
print("ROC-AUC TRAIN:")
print(roc_auc_score(y_train, y_pred_prob_train_knn))
print(classification_report(y_train, y_pred_train_knn,
      target_names=['No compra (0)', 'Compra (1)']))
```

```
ROC-AUC TRAIN:
1.0
              precision    recall  f1-score   support

No compra (0)       1.00      1.00      1.00      8338
   Compra (1)       1.00      1.00      1.00      1526

     accuracy                           1.00      9864
    macro avg       1.00      1.00      1.00      9864
 weighted avg       1.00      1.00      1.00      9864
```

Resultados en test

In [98]:
```python
y_pred_prob_knn = best_knn.predict_proba(X_test)[:, 1]
y_pred_knn = best_knn.predict(X_test)
print("ROC-AUC TEST:")
print(roc_auc_score(y_test, y_pred_prob_knn))
print(classification_report(y_test, y_pred_knn,
      target_names=['No compra (0)', 'Compra (1)']))
```

```
ROC-AUC TEST:
0.7474594517189054
              precision    recall  f1-score   support

No compra (0)       0.89      0.90      0.89      2084
   Compra (1)       0.42      0.41      0.42       382

     accuracy                           0.82      2466
    macro avg       0.66      0.66      0.66      2466
 weighted avg       0.82      0.82      0.82      2466
```

In [99]:
```python
# === INTERVALOS DE CONFIANZA BOOTSTRAP para test ===
from sklearn.utils import resample

def bootstrap_ci(y_true, y_pred, y_prob, n_bootstrap=1000, alpha=0.05):
    """Calcula IC 95% por bootstrap para ROC-AUC y F1."""
    rng = np.random.RandomState(42)
    aucs, f1s = [], []
```

```
    for _ in range(n_bootstrap):
        idx = resample(range(len(y_true)), random_state=rng)
        if len(np.unique(np.array(y_true)[idx])) < 2:
            continue
        aucs.append(roc_auc_score(np.array(y_true)[idx], np.array(y_prob)[idx]))
        f1s.append(f1_score(np.array(y_true)[idx], np.array(y_pred)[idx]))
    lo, hi = alpha/2, 1 - alpha/2
    return {
        'ROC-AUC': (np.percentile(aucs, lo*100), np.mean(aucs), np.percentile(aucs,
        'F1':      (np.percentile(f1s,  lo*100), np.mean(f1s),  np.percentile(f1s,
    }

ci = bootstrap_ci(y_test, y_pred_knn, y_pred_prob_knn)
print(f"ROC-AUC test: {ci['ROC-AUC'][1]:.4f} "
      f"(IC 95%: [{ci['ROC-AUC'][0]:.4f}, {ci['ROC-AUC'][2]:.4f}])")
print(f"F1 test:      {ci['F1'][1]:.4f} "
      f"(IC 95%: [{ci['F1'][0]:.4f}, {ci['F1'][2]:.4f}])")
```

```
ROC-AUC test: 0.7472 (IC 95%: [0.7192, 0.7747])
F1 test:      0.4188 (IC 95%: [0.3753, 0.4617])
```

Nombres de clases en el classification_report

In [100…

```
print(classification_report(
    y_test,
    y_pred_knn,
    target_names=['No compra (0)', 'Compra (1)']
))
```

```
              precision    recall  f1-score   support

No compra (0)       0.89      0.90      0.89      2084
   Compra (1)       0.42      0.41      0.42       382

     accuracy                           0.82      2466
    macro avg       0.66      0.66      0.66      2466
 weighted avg       0.82      0.82      0.82      2466
```

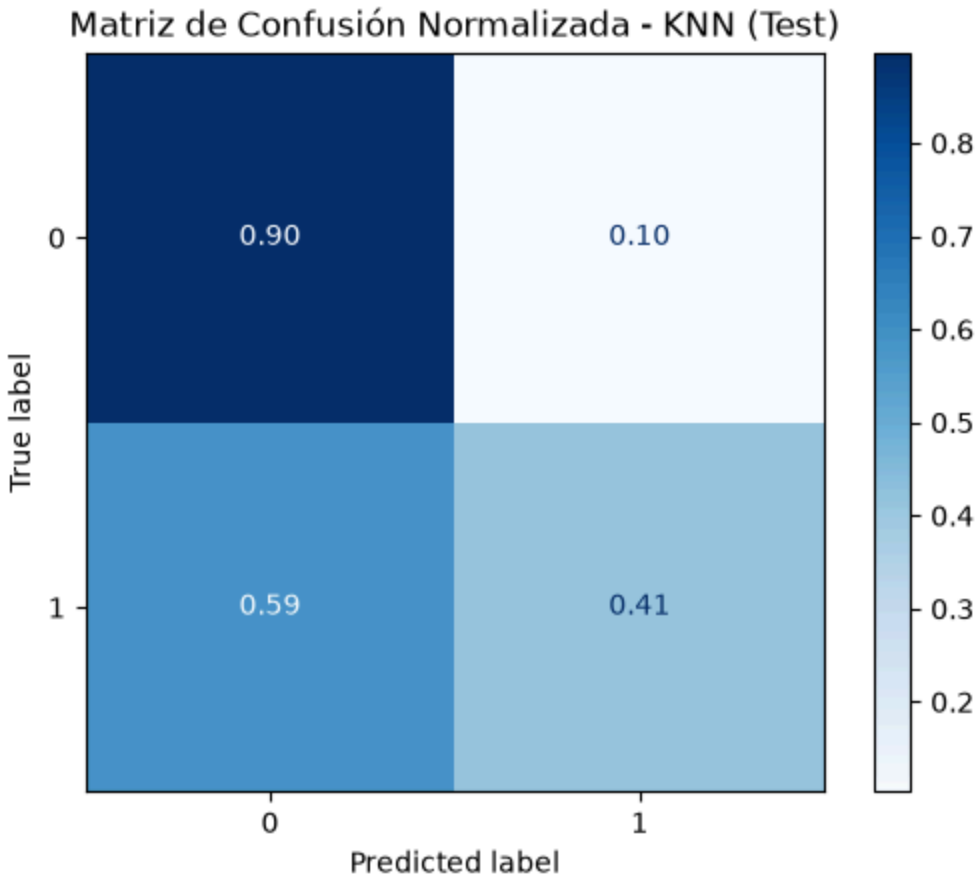
Matriz de confusión

In [101…

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
# MATRIZ DE CONFUSIÓN NORMALIZADA EN TEST
# predicciones del mejor modelo
y_test_pred = best_knn.predict(X_test)
# matriz de confusión normalizada
cm = confusion_matrix(
    y_test,
    y_test_pred,
    normalize='true'
)
# visualización
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
plt.figure(figsize=(6, 6))
disp.plot(
    cmap='Blues',
```

```
    values_format='.2f'
)
plt.title('Matriz de Confusión Normalizada - KNN (Test)')
plt.show()
```

```
<Figure size 600x600 with 0 Axes>
```

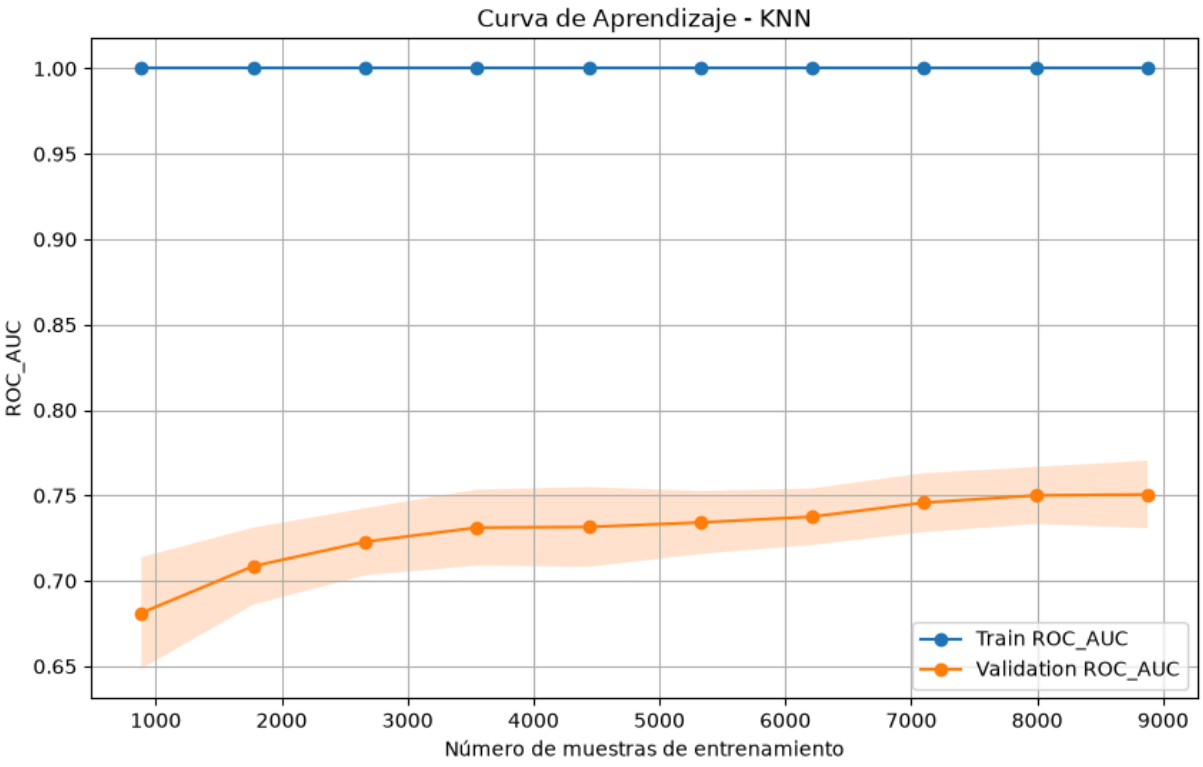


Curva de aprendizaje

In [102…

```
train_sizes, train_scores, val_scores = learning_curve(
    estimator=best_knn,
    X=X_train,
    y=y_train,
    cv=cv_strategy,
    scoring='roc_auc',
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10)
)
# medias
train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
val_mean = val_scores.mean(axis=1)
val_std = val_scores.std(axis=1)
plt.figure(figsize=(10, 6))
plt.plot(
    train_sizes,
    train_mean,
    marker='o',
    label='Train ROC_AUC'
```

```
)
plt.plot(
    train_sizes,
    val_mean,
    marker='o',
    label='Validation ROC_AUC'
)
plt.fill_between(
    train_sizes,
    train_mean - train_std,
    train_mean + train_std,
    alpha=0.2
)
plt.fill_between(
    train_sizes,
    val_mean - val_std,
    val_mean + val_std,
    alpha=0.2
)
plt.xlabel('Número de muestras de entrenamiento')
plt.ylabel('ROC_AUC')
plt.title('Curva de Aprendizaje - KNN')
plt.legend()
plt.grid(True)
plt.show()
```



SVM

Definición de pipeline y malla de hiperparametros

In [103…
```
pipeline_svc = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('svc', SVC(probability=True, random_state=42))
])
```

In [104…
```
param_grid_svc = [
  {
      'smote__sampling_strategy': [0.22, 0.3],
      'svc__kernel': ['linear'],
      'svc__C': [0.01, 0.1, 1]
  },
  {
      'smote__sampling_strategy': [0.22, 0.3],
      'svc__kernel': ['rbf'],
      'svc__C': [0.01, 0.1, 1],
      'svc__gamma': ['scale', 'auto', 0.01, 0.1],
  }

]
```

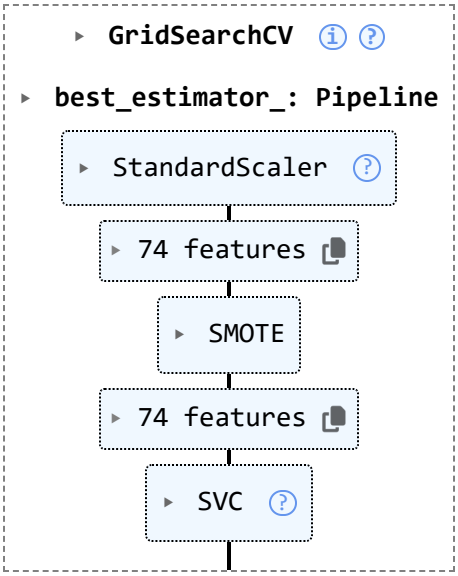
In [105…
```
grid_svc = GridSearchCV(
    estimator=pipeline_svc,
    param_grid=param_grid_svc,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=0,
    return_train_score=True
)
```

Entrenamiento usando la malla de hiperparametros

In [106…
```
grid_svc.fit(X_train,y_train)
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead of `SVC(probability=True)`
  warnings.warn(
```

Out[106…



Resultados para los 3 mejores pipelines

In [107…

```
results = pd.DataFrame(grid_svc.cv_results_)

results_sorted = results.sort_values(
    by='mean_test_score',
    ascending=False
).reset_index(drop=True)
```

In [108…

```
top_3_results = []

for i in range(3):

    row = results_sorted.iloc[i]

    params = row['params']

    # reconstruir pipeline
    model = Pipeline([
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42)),
        ('svc', SVC(probability=True, random_state=42))
    ])

    model.set_params(**params)

    # entrenar
    model.fit(X_train, y_train)

    # probabilidades
    y_train_proba = model.predict_proba(X_train)[:, 1]
    y_test_proba = model.predict_proba(X_test)[:, 1]

    # métricas
    train_auc = roc_auc_score(y_train, y_train_proba)
    test_auc = roc_auc_score(y_test, y_test_proba)
```

```
    top_3_results.append({
        'rank': i + 1,
        'params': params,
        'train_roc_auc': train_auc,
        'validation_roc_auc': row['mean_test_score'],
        'test_roc_auc': test_auc
    })
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
```

In [109…

```
top_3_df = pd.DataFrame(top_3_results)

print("\n===== TOP 3 MODELOS =====\n")
print(top_3_df)
```

```
===== TOP 3 MODELOS =====

   rank                                             params  train_roc_auc  \
0     1  {'smote__sampling_strategy': 0.3, 'svc__C': 1,...       0.900228
1     2  {'smote__sampling_strategy': 0.3, 'svc__C': 0....       0.903083
2     3  {'smote__sampling_strategy': 0.3, 'svc__C': 0....       0.896924

   validation_roc_auc  test_roc_auc
0            0.747894      0.890549
1            0.747152      0.885395
2            0.739811      0.877868
```

In [110…

```
# OBTENER LOS 3 MEJORES PIPELINES DE SVC
print("\n" + "=" * 80)
print("TOP 3 PIPELINES SVC SEGÚN BALANCED_ACCURACY (CV)")
print("=" * 80)

for i in range(3):
    row = results_sorted.iloc[i]
    params = row['params']
    balanced_acc_cv_mean = row['mean_test_score']
    balanced_acc_cv_std = row['std_test_score']

    # Reconstruir pipeline según el kernel
    if params['svc__kernel'] == 'linear':
        pipeline_temp_svc = Pipeline([
            ('scaler', StandardScaler()),
```

```
            ('smote', SMOTE(random_state=42, sampling_strategy=params['smote__sampl
            ('svc', SVC(
                probability=True,
                random_state=42,
                kernel='linear',
                C=params['svc__C']
            ))
        ])
    else:  # rbf
        pipeline_temp_svc = Pipeline([
            ('scaler', StandardScaler()),
            ('smote', SMOTE(random_state=42, sampling_strategy=params['smote__sampl
            ('svc', SVC(
                probability=True,
                random_state=42,
                kernel='rbf',
                C=params['svc__C'],
                gamma=params['svc__gamma']
            ))
        ])

    # Entrenar y evaluar
    pipeline_temp_svc.fit(X_train, y_train)
    y_train_pred = pipeline_temp_svc.predict(X_train)
    y_test_pred = pipeline_temp_svc.predict(X_test)
    y_test_prob = pipeline_temp_svc.predict_proba(X_test)[:, 1]

    balanced_acc_train = balanced_accuracy_score(y_train, y_train_pred)
    auc_test = roc_auc_score(y_test, y_test_prob)
    f1_test = f1_score(y_test, y_test_pred)

    print(f"\n{'='*80}")
    print(f"PIPELINE SVC #{i+1}")
    print(f"{'='*80}")
    print("HIPERPARÁMETROS:")
    for param, value in params.items():
        print(f"  {param}: {value}")

    print(f"\nBALANCED_ACCURACY:")
    print(f"  Train: {balanced_acc_train:.4f}")
    print(f"  Validation (CV): {balanced_acc_cv_mean:.4f} (±{balanced_acc_cv_std:.4

    print(f"\nTEST:")
    print(f"  ROC-AUC: {auc_test:.4f}")
    print(f"  F1-Score: {f1_test:.4f}")
    print()
```

```
================================================================================
TOP 3 PIPELINES SVC SEGÚN BALANCED_ACCURACY (CV)
================================================================================
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
```

```
================================================================================
PIPELINE SVC #1
================================================================================
HIPERPARÁMETROS:
  smote__sampling_strategy: 0.3
  svc__C: 1
  svc__kernel: linear

BALANCED_ACCURACY:
  Train: 0.7509
  Validation (CV): 0.7479 (±0.0208)

TEST:
  ROC-AUC: 0.8905
  F1-Score: 0.5723
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead of `SVC(probability=True)`
  warnings.warn(
```

```
================================================================================
PIPELINE SVC #2
================================================================================
HIPERPARÁMETROS:
  smote__sampling_strategy: 0.3
  svc__C: 0.1
  svc__kernel: linear

BALANCED_ACCURACY:
  Train: 0.7500
  Validation (CV): 0.7472 (±0.0205)

TEST:
  ROC-AUC: 0.8854
  F1-Score: 0.5680
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead of `SVC(probability=True)`
  warnings.warn(
```

```
================================================================================
PIPELINE SVC #3
================================================================================
HIPERPARÁMETROS:
  smote__sampling_strategy: 0.3
  svc__C: 0.01
  svc__kernel: linear

BALANCED_ACCURACY:
  Train: 0.7431
  Validation (CV): 0.7398 (±0.0196)

TEST:
  ROC-AUC: 0.8779
  F1-Score: 0.5505
```

Intervalos de confianza

In [111…

```python
# === INTERVALOS DE CONFIANZA BOOTSTRAP para test ===
from sklearn.utils import resample

def bootstrap_ci(y_true, y_pred, y_prob, n_bootstrap=1000, alpha=0.05):
    """Calcula IC 95% por bootstrap para ROC-AUC y F1."""
    rng = np.random.RandomState(42)
    aucs, f1s = [], []
    for _ in range(n_bootstrap):
        idx = resample(range(len(y_true)), random_state=rng)
        if len(np.unique(np.array(y_true)[idx])) < 2:
            continue
        aucs.append(roc_auc_score(np.array(y_true)[idx], np.array(y_prob)[idx]))
        f1s.append(f1_score(np.array(y_true)[idx], np.array(y_pred)[idx]))
    lo, hi = alpha/2, 1 - alpha/2
    return {
        'ROC-AUC': (np.percentile(aucs, lo*100), np.mean(aucs), np.percentile(aucs,
        'F1':      (np.percentile(f1s,  lo*100), np.mean(f1s),  np.percentile(f1s,
    }

ci = bootstrap_ci(y_test, y_test_pred, y_test_prob)
print(f"ROC-AUC test: {ci['ROC-AUC'][1]:.4f} "
      f"(IC 95%: [{ci['ROC-AUC'][0]:.4f}, {ci['ROC-AUC'][2]:.4f}])")
print(f"F1 test:      {ci['F1'][1]:.4f} "
      f"(IC 95%: [{ci['F1'][0]:.4f}, {ci['F1'][2]:.4f}])")
```

```
ROC-AUC test: 0.8775 (IC 95%: [0.8565, 0.8986])
F1 test:      0.5507 (IC 95%: [0.5046, 0.5982])
```

In [112…

```python
scores_svc = cross_val_score(
    grid_svc.best_estimator_,
    X_train,
    y_train,
    cv=cv_strategy,
    scoring='roc_auc'
)
print(f"ROC-AUC por fold Cross-Validation:")
print(np.round(scores_svc, 4))
```

```
print(f"Media:  {scores_svc.mean():.4f}")
print(f"Std:    {scores_svc.std():.4f}")
```

```
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
c:\Users\User\Desktop\Proyecto_Aula_LaTeX\.venv\Lib\site-packages\sklearn\svm\_base.
py:236: FutureWarning: The `probability` parameter was deprecated in 1.9 and will be
removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead
of `SVC(probability=True)`
  warnings.warn(
```

```
ROC-AUC por fold Cross-Validation:
[0.8824 0.8974 0.8996 0.9039 0.8907 0.9199 0.9    0.8888 0.9079 0.8982]
Media:  0.8989
Std:    0.0099
```

Curva de aprendizaje

In [113…

```python
best_model = grid_svc.best_estimator_

print("\n===== MEJOR MODELO =====\n")
print(grid_svc.best_params_)
print(f"Best Validation ROC_AUC: {grid_svc.best_score_:.4f}")

train_sizes, train_scores, val_scores = learning_curve(
    estimator=best_model,
    X=X_train,
    y=y_train,
    cv=cv_strategy,
    scoring='roc_auc',
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10)
)

# medias
train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)

val_mean = val_scores.mean(axis=1)
val_std = val_scores.std(axis=1)

plt.figure(figsize=(10, 6))

plt.plot(
    train_sizes,
    train_mean,
    marker='o',
    label='Train ROC_AUC'
)

plt.plot(
    train_sizes,
    val_mean,
    marker='o',
    label='Validation ROC_AUC'
)

plt.fill_between(
    train_sizes,
    train_mean - train_std,
    train_mean + train_std,
    alpha=0.2
)

plt.fill_between(
    train_sizes,
```

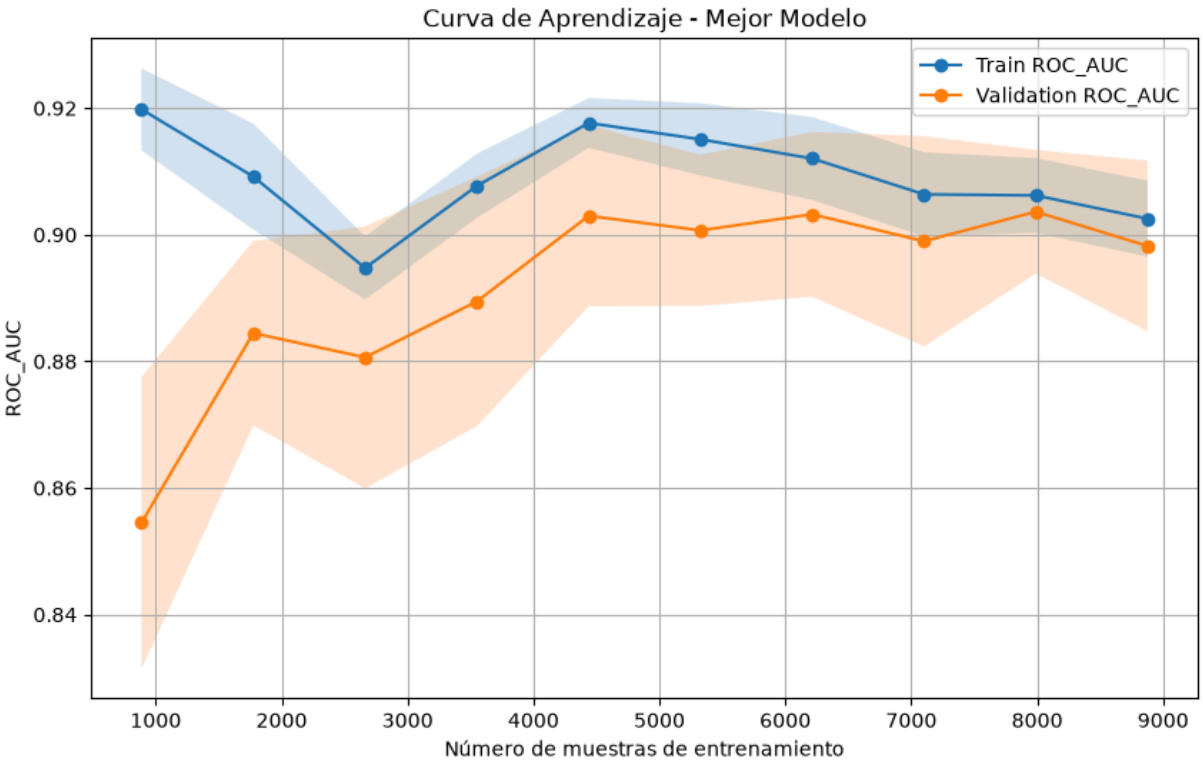
```
    val_mean - val_std,
    val_mean + val_std,
    alpha=0.2
)

plt.xlabel('Número de muestras de entrenamiento')
plt.ylabel('ROC_AUC')
plt.title('Curva de Aprendizaje - Mejor Modelo')
plt.legend()
plt.grid(True)

plt.show()
```

```
===== MEJOR MODELO =====

{'smote__sampling_strategy': 0.3, 'svc__C': 1, 'svc__kernel': 'linear'}
Best Validation ROC_AUC: 0.7479
```



Matriz de confusión

In [114…

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# ============================================================
# MATRIZ DE CONFUSIÓN NORMALIZADA EN TEST
# ============================================================

# predicciones del mejor modelo
y_test_pred = best_model.predict(X_test)

# matriz de confusión normalizada
cm = confusion_matrix(
    y_test,
    y_test_pred,
```

```
    normalize='true'
)

# visualización
disp = ConfusionMatrixDisplay(confusion_matrix=cm)

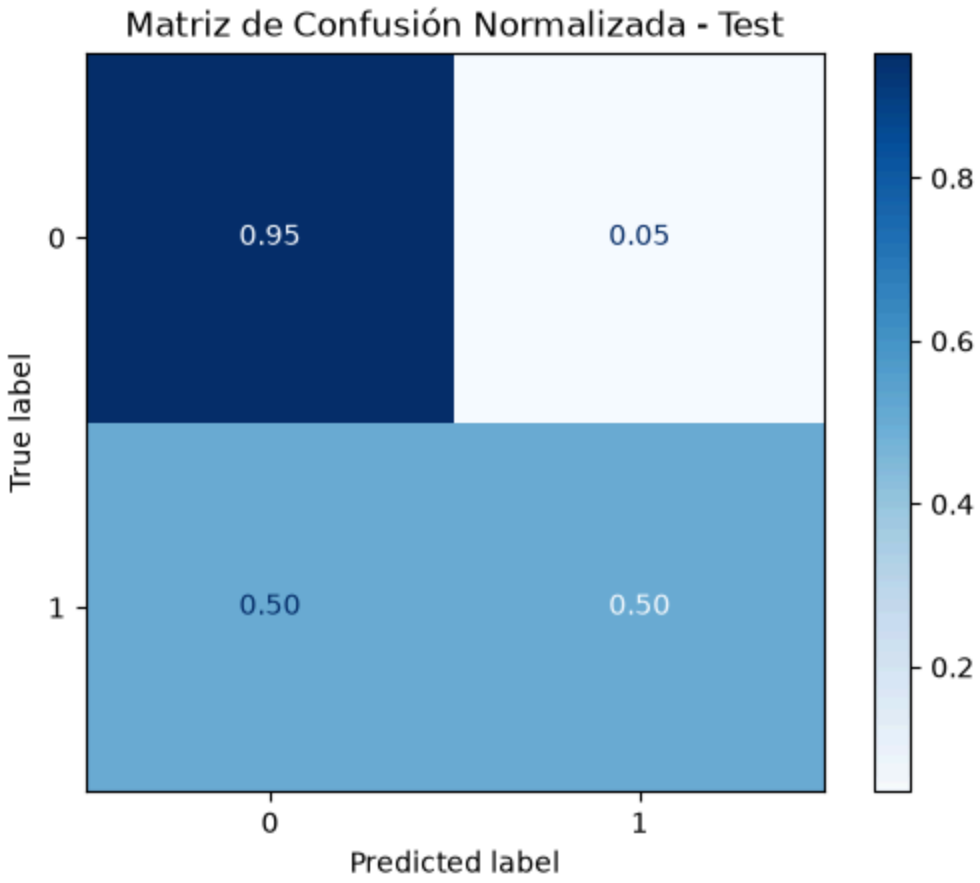
plt.figure(figsize=(6, 6))

disp.plot(
    cmap='Blues',
    values_format='.2f'
)

plt.title('Matriz de Confusión Normalizada - Test')

plt.show()
```

```
<Figure size 600x600 with 0 Axes>
```



GMM

Definición de pipeline y malla de hiperparametros

In [115…

```
import numpy as np

from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.mixture import GaussianMixture
from sklearn.utils.validation import check_X_y, check_array
```

```python
class GMMClassifier(ClassifierMixin, BaseEstimator):

    def __init__(
        self,
        n_components=1,
        covariance_type='full',
        max_iter=100,
        n_init=1,
        init_params='kmeans',
        random_state=None
    ):
        self.n_components = n_components
        self.covariance_type = covariance_type
        self.max_iter = max_iter
        self.n_init = n_init
        self.init_params = init_params
        self.random_state = random_state

    def __sklearn_tags__(self):

        tags = super().__sklearn_tags__()

        tags.estimator_type = "classifier"

        return tags

    def fit(self, X, y):

        X, y = check_X_y(X, y)

        self.classes_ = np.unique(y)

        self.models_ = {}

        self.priors_ = {}

        for c in self.classes_:

            gmm = GaussianMixture(
                n_components=self.n_components,
                covariance_type=self.covariance_type,
                max_iter=self.max_iter,
                n_init=self.n_init,
                init_params=self.init_params,
                random_state=self.random_state
            )

            X_class = X[y == c]

            gmm.fit(X_class)

            self.models_[c] = gmm

            self.priors_[c] = X_class.shape[0] / X.shape[0]
```

```
        return self

    def predict_proba(self, X):

        X = check_array(X)

        log_probs = np.zeros(
            (X.shape[0], len(self.classes_))
        )

        for idx, c in enumerate(self.classes_):

            gmm = self.models_[c]

            log_likelihood = gmm.score_samples(X)

            log_prior = np.log(self.priors_[c])

            log_probs[:, idx] = (
                log_likelihood + log_prior
            )

        log_probs -= log_probs.max(
            axis=1,
            keepdims=True
        )

        probs = np.exp(log_probs)

        probs /= probs.sum(
            axis=1,
            keepdims=True
        )

        return probs

    def predict(self, X):

        probs = self.predict_proba(X)

        return self.classes_[
            np.argmax(probs, axis=1)
        ]
```

In [116…

```
pipeline_gmm = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('gmm', GMMClassifier(
        random_state=42
    ))
])
```

In [117…

```
param_grid_gmm = {
    'smote__sampling_strategy': [0.22, 0.3,],
```

```
    'gmm__n_components': [1, 2, 3, 4],
    'gmm__covariance_type': ['full', 'diag', 'tied', 'spherical'],
    'gmm__max_iter': [100, 200],
    'gmm__n_init': [1, 2],
    'gmm__init_params': [
        'kmeans',
        'k-means++',
        'random',
        'random_from_data'
    ],
}
```

In [118…

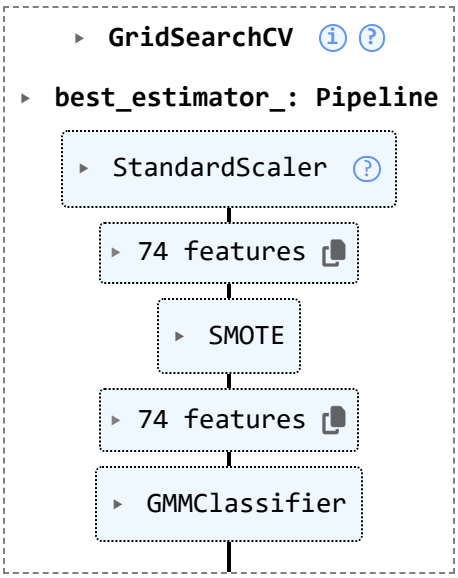
```
grid_gmm = GridSearchCV(
    estimator=pipeline_gmm,
    param_grid=param_grid_gmm,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=0,
    error_score='raise',
    return_train_score=True
)
```

Entrenamiento usando malla de hiperparametros

In [119…

```
grid_gmm.fit(X_train, y_train)
```

Out[119…



Resultado de los 3 mejores pipelines

Resultados

In [120…

```
results = pd.DataFrame(grid_gmm.cv_results_)

results_sorted = results.sort_values(
    by='mean_test_score',
```

```
    ascending=False
).reset_index(drop=True)
```

In [121…

```
top_3_results = []

for i in range(3):

    row = results_sorted.iloc[i]

    params = row['params']

    # reconstruir modelo
    model = Pipeline([
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42)),
        ('gmm', GMMClassifier(
            random_state=42
        ))
    ])

    model.set_params(**params)

    # entrenar
    model.fit(X_train, y_train)

    # probabilidades
    y_train_proba = model.predict_proba(X_train)[:, 1]
    y_test_proba = model.predict_proba(X_test)[:, 1]

    # métricas
    train_auc = roc_auc_score(y_train, y_train_proba)
    test_auc = roc_auc_score(y_test, y_test_proba)

    top_3_results.append({
        'rank': i + 1,
        'params': params,
        'train_roc_auc': train_auc,
        'validation_roc_auc': row['mean_test_score'],
        'test_roc_auc': test_auc
    })
```

In [122…

```
top_3_df = pd.DataFrame(top_3_results)

print("\n===== TOP 3 MODELOS GMM =====\n")
print(top_3_df)
```

```
===== TOP 3 MODELOS GMM =====

   rank                                             params  train_roc_auc  \
0     1  {'gmm__covariance_type': 'spherical', 'gmm__in...       0.779159
1     2  {'gmm__covariance_type': 'spherical', 'gmm__in...       0.779159
2     3  {'gmm__covariance_type': 'spherical', 'gmm__in...       0.779337

   validation_roc_auc  test_roc_auc
0            0.722054      0.777203
1            0.722054      0.777203
2            0.721934      0.777344
```

In [123…

```python
# OBTENER LOS 3 MEJORES PIPELINES DE GMM (continuación del código existente)
print("\n" + "=" * 80)
print("TOP 3 PIPELINES GMM SEGÚN BALANCED_ACCURACY (CV)")
print("=" * 80)

for i in range(3):
    row = results_sorted.iloc[i]
    params = row['params']
    balanced_acc_cv_mean = row['mean_test_score']
    balanced_acc_cv_std = row['std_test_score']

    # Reconstruir modelo
    pipeline_temp_gmm = Pipeline([
        ('scaler', StandardScaler()),
        ('smote', SMOTE(random_state=42, sampling_strategy=params['smote__sampling_
        ('gmm', GMMClassifier(
            n_components=params['gmm__n_components'],
            covariance_type=params['gmm__covariance_type'],
            max_iter=params['gmm__max_iter'],
            n_init=params['gmm__n_init'],
            init_params=params['gmm__init_params'],
            random_state=42
        ))
    ])

    # Entrenar y evaluar
    pipeline_temp_gmm.fit(X_train, y_train)
    y_train_pred = pipeline_temp_gmm.predict(X_train)
    y_test_pred = pipeline_temp_gmm.predict(X_test)
    y_test_prob = pipeline_temp_gmm.predict_proba(X_test)[:, 1]

    balanced_acc_train = balanced_accuracy_score(y_train, y_train_pred)
    auc_test = roc_auc_score(y_test, y_test_prob)
    f1_test = f1_score(y_test, y_test_pred)

    print(f"\n{'='*80}")
    print(f"PIPELINE GMM #{i+1}")
    print(f"{'='*80}")
    print("HIPERPARÁMETROS:")
    for param, value in params.items():
        print(f"  {param}: {value}")

    print(f"\nBALANCED_ACCURACY:")
    print(f"  Train: {balanced_acc_train:.4f}")
```

```
    print(f"  Validation (CV): {balanced_acc_cv_mean:.4f} (±{balanced_acc_cv_std:.4

    print(f"\nTEST:")
    print(f"  ROC-AUC: {auc_test:.4f}")
    print(f"  F1-Score: {f1_test:.4f}")
    print()

# MEJOR MODELO GMM
print("\n" + "=" * 60)
print("MEJOR MODELO GMM (GridSearchCV)")
print("=" * 60)
print(f"Mejor balanced_accuracy (CV): {grid_gmm.best_score_:.4f}")
print("Mejores hiperparámetros:")
for param, value in grid_gmm.best_params_.items():
    print(f"  {param}: {value}")

# Evaluación del mejor modelo en test
best_gmm = grid_gmm.best_estimator_
y_pred_prob_gmm = best_gmm.predict_proba(X_test)[:, 1]
y_pred_gmm = best_gmm.predict(X_test)

roc_auc_test = roc_auc_score(y_test, y_pred_prob_gmm)
print(f"\nROC-AUC en TEST: {roc_auc_test:.4f}")

print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred_gmm, target_names=['No compra (0)', 'Com

print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred_gmm))
```

```
================================================================================
TOP 3 PIPELINES GMM SEGÚN BALANCED_ACCURACY (CV)
================================================================================

================================================================================
PIPELINE GMM #1
================================================================================
HIPERPARÁMETROS:
  gmm__covariance_type: spherical
  gmm__init_params: random
  gmm__max_iter: 200
  gmm__n_components: 2
  gmm__n_init: 2
  smote__sampling_strategy: 0.3

BALANCED_ACCURACY:
  Train: 0.7091
  Validation (CV): 0.7221 (±0.0284)

TEST:
  ROC-AUC: 0.7772
  F1-Score: 0.4503


================================================================================
PIPELINE GMM #2
================================================================================
HIPERPARÁMETROS:
  gmm__covariance_type: spherical
  gmm__init_params: random
  gmm__max_iter: 100
  gmm__n_components: 2
  gmm__n_init: 2
  smote__sampling_strategy: 0.3

BALANCED_ACCURACY:
  Train: 0.7091
  Validation (CV): 0.7221 (±0.0284)

TEST:
  ROC-AUC: 0.7772
  F1-Score: 0.4503


================================================================================
PIPELINE GMM #3
================================================================================
HIPERPARÁMETROS:
  gmm__covariance_type: spherical
  gmm__init_params: random
  gmm__max_iter: 100
  gmm__n_components: 2
  gmm__n_init: 1
  smote__sampling_strategy: 0.3

BALANCED_ACCURACY:
```

```
  Train: 0.7088
  Validation (CV): 0.7219 (±0.0284)

TEST:
  ROC-AUC: 0.7773
  F1-Score: 0.4503


============================================================
MEJOR MODELO GMM (GridSearchCV)
============================================================
Mejor balanced_accuracy (CV): 0.7221
Mejores hiperparámetros:
  gmm__covariance_type: spherical
  gmm__init_params: random
  gmm__max_iter: 100
  gmm__n_components: 2
  gmm__n_init: 2
  smote__sampling_strategy: 0.3

ROC-AUC en TEST: 0.7772

Reporte de clasificación:
              precision    recall  f1-score   support

No compra (0)       0.92      0.77      0.84      2084
   Compra (1)       0.34      0.65      0.45       382

     accuracy                           0.75      2466
    macro avg       0.63      0.71      0.65      2466
 weighted avg       0.83      0.75      0.78      2466

Matriz de confusión:
[[1609  475]
 [ 133  249]]
```

Intervalos de confianza

In [124…

```python
# === INTERVALOS DE CONFIANZA BOOTSTRAP para test ===
from sklearn.utils import resample

def bootstrap_ci(y_true, y_pred, y_prob, n_bootstrap=1000, alpha=0.05):
    """Calcula IC 95% por bootstrap para ROC-AUC y F1."""
    rng = np.random.RandomState(42)
    aucs, f1s = [], []
    for _ in range(n_bootstrap):
        idx = resample(range(len(y_true)), random_state=rng)
        if len(np.unique(np.array(y_true)[idx])) < 2:
            continue
        aucs.append(roc_auc_score(np.array(y_true)[idx], np.array(y_prob)[idx]))
        f1s.append(f1_score(np.array(y_true)[idx], np.array(y_pred)[idx]))
    lo, hi = alpha/2, 1 - alpha/2
    return {
        'ROC-AUC': (np.percentile(aucs, lo*100), np.mean(aucs), np.percentile(aucs,
        'F1':      (np.percentile(f1s,  lo*100), np.mean(f1s),  np.percentile(f1s,
```

```
    }

ci = bootstrap_ci(y_test, y_pred_gmm, y_pred_prob_gmm)
print(f"ROC-AUC test: {ci['ROC-AUC'][1]:.4f} "
      f"(IC 95%: [{ci['ROC-AUC'][0]:.4f}, {ci['ROC-AUC'][2]:.4f}])")
print(f"F1 test:      {ci['F1'][1]:.4f} "
      f"(IC 95%: [{ci['F1'][0]:.4f}, {ci['F1'][2]:.4f}])")
```

```
ROC-AUC test: 0.7772 (IC 95%: [0.7508, 0.8015])
F1 test:      0.4500 (IC 95%: [0.4152, 0.4882])
```

In [125…

```
scores_gmm = cross_val_score(
    grid_gmm.best_estimator_,
    X_train,
    y_train,
    cv=cv_strategy,
    scoring='roc_auc'
)
print(f"ROC-AUC por fold Cross-Validation:")
print(np.round(scores_gmm, 4))
print(f"Media:  {scores_gmm.mean():.4f}")
print(f"Std:    {scores_gmm.std():.4f}")
```

```
ROC-AUC por fold Cross-Validation:
[0.7895 0.7758 0.809  0.8162 0.7845 0.8402 0.8556 0.8248 0.8448 0.774 ]
Media:  0.8114
Std:    0.0282
```

Curva de aprendizaje

In [126…

```
best_model = grid_gmm.best_estimator_

print("\n===== MEJOR MODELO =====\n")
print(grid_gmm.best_params_)
print(f"Best Validation ROC_AUC: {grid_gmm.best_score_:.4f}")

train_sizes, train_scores, val_scores = learning_curve(
    estimator=best_model,
    X=X_train,
    y=y_train,
    cv=cv_strategy,
    scoring='roc_auc',
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10)
)

# medias
train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)

val_mean = val_scores.mean(axis=1)
val_std = val_scores.std(axis=1)

plt.figure(figsize=(10, 6))

plt.plot(
```

```
    train_sizes,
    train_mean,
    marker='o',
    label='Train ROC_AUC'
)

plt.plot(
    train_sizes,
    val_mean,
    marker='o',
    label='Validation ROC_AUC'
)

plt.fill_between(
    train_sizes,
    train_mean - train_std,
    train_mean + train_std,
    alpha=0.2
)

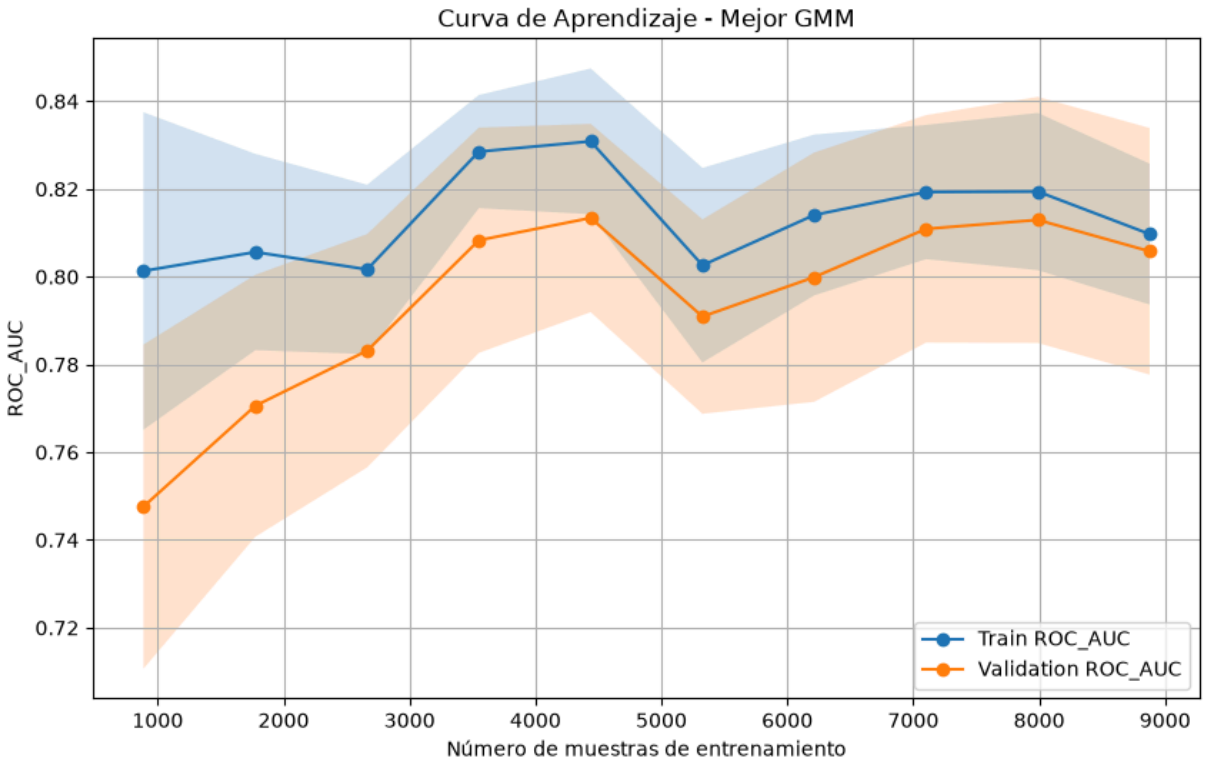
plt.fill_between(
    train_sizes,
    val_mean - val_std,
    val_mean + val_std,
    alpha=0.2
)

plt.xlabel('Número de muestras de entrenamiento')
plt.ylabel('ROC_AUC')
plt.title('Curva de Aprendizaje - Mejor GMM')
plt.legend()
plt.grid(True)

plt.show()
```

```
===== MEJOR MODELO =====

{'gmm__covariance_type': 'spherical', 'gmm__init_params': 'random', 'gmm__max_iter':
100, 'gmm__n_components': 2, 'gmm__n_init': 2, 'smote__sampling_strategy': 0.3}
Best Validation ROC_AUC: 0.7221
```



Matriz de confusión

In [127…

```
y_test_pred = best_model.predict(X_test)

cm = confusion_matrix(
    y_test,
    y_test_pred,
    normalize='true'
)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

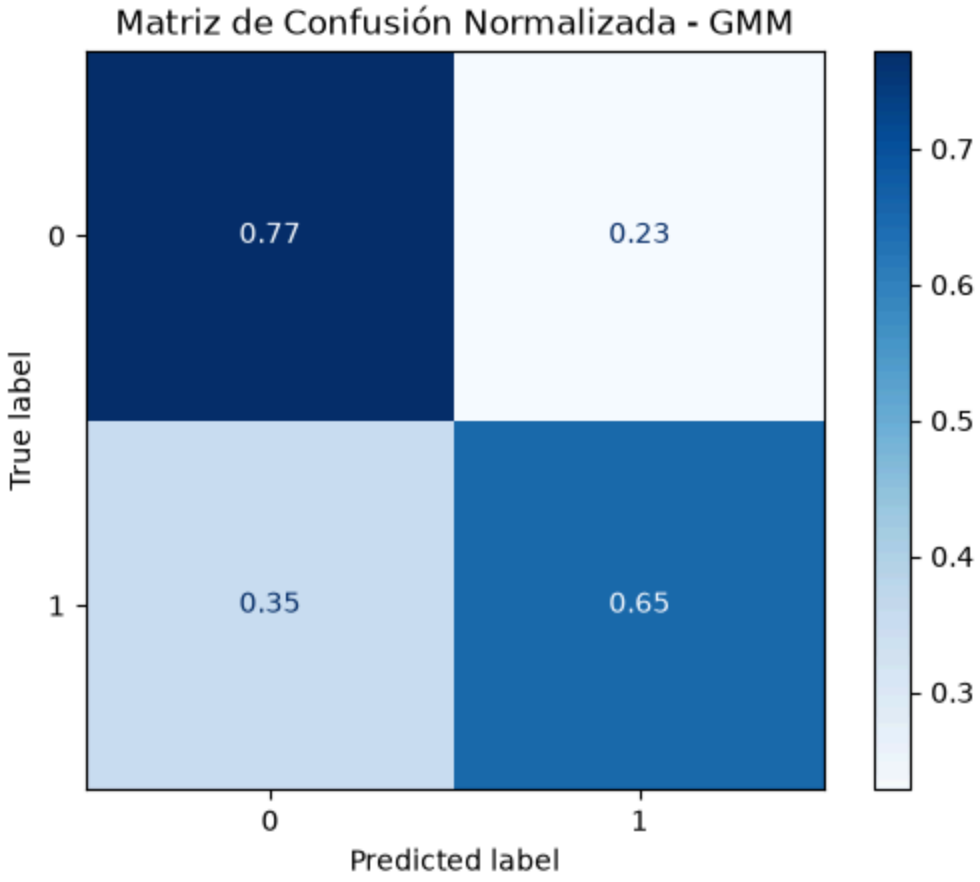
plt.figure(figsize=(6, 6))

disp.plot(
    cmap='Blues',
    values_format='.2f'
)

plt.title('Matriz de Confusión Normalizada - GMM')

plt.show()
```

```
<Figure size 600x600 with 0 Axes>
```



Reducción de Dimensión

Debe haber ejecutado el notebook principal hasta obtener X_train, X_test, y_train, y_test, best_rf y best_xgb.

In [128…

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import warnings
warnings.filterwarnings('ignore')

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, cross_val_score, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    roc_auc_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay,
    f1_score, precision_score, recall_score, accuracy_score
)
from xgboost import XGBClassifier
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
```

```
# Instalar UMAP si no está disponible
try:
    import umap
    print("UMAP ya está instalado.")
except ImportError:
    import subprocess, sys
    subprocess.check_call([sys.executable, "-m", "pip", "install", "umap-learn", "-
    import umap
    print("UMAP instalado correctamente.")

print("Librerías cargadas correctamente.")
```

```
UMAP ya está instalado.
Librerías cargadas correctamente.
```

In [129…

```
print(f"X_train: {X_train.shape}")
print(f"X_test:  {X_test.shape}")
print(f"Variables originales: {X_train.shape[1]}")
```

```
X_train: (9864, 74)
X_test:  (2466, 74)
Variables originales: 74
```

Análisis Individual de Variables

In [130…

```
# ── Correlación de Pearson (variables numéricas) ──────────────────────────
y_train_flat = np.array(y_train).ravel()
y_series = pd.Series(y_train_flat, index=X_train.index, name='Revenue')
X_train_df = pd.DataFrame(X_train, columns=X_train.columns if hasattr(X_train, 'col

pearson_corr = X_train_df.corrwith(y_series).abs().sort_values(ascending=False)

print("=" * 55)
print("CORRELACIÓN DE PEARSON |r| con Revenue (descendente)")
print("=" * 55)
print(pearson_corr.to_string())
```

```
=========================================================
CORRELACIÓN DE PEARSON |r| con Revenue (descendente)
=========================================================
PageValues                          0.503271
ExitRates                           0.212674
ProductRelated                      0.155596
BounceRates                         0.154210
ProductRelated_Duration             0.145974
Month_Nov                           0.145156
Administrative                      0.142691
TrafficType_2                       0.127040
VisitorType_New_Visitor             0.101738
VisitorType_Returning_Visitor       0.100726
Administrative_Duration             0.099236
Informational                       0.097372
SpecialDay                          0.087851
TrafficType_3                       0.084708
Month_May                           0.079300
TrafficType_1                       0.073576
OperatingSystems_3                  0.069191
Informational_Duration              0.068662
OperatingSystems_2                  0.063476
TrafficType_13                      0.062969
Month_Mar                           0.060998
TrafficType_8                       0.056110
Month_Feb                           0.047793
Month_Oct                           0.039634
Weekend                             0.034779
TrafficType_20                      0.028911
Browser_3                           0.027132
Month_Sep                           0.026729
TrafficType_5                       0.026377
TrafficType_10                      0.026203
Month_Dec                           0.025126
TrafficType_15                      0.024786
Month_June                          0.022896
TrafficType_7                       0.021619
TrafficType_6                       0.021438
Browser_12                          0.020182
Browser_10                          0.019581
Browser_13                          0.019184
OperatingSystems_1                  0.016650
Browser_7                           0.016594
Browser_4                           0.016233
Browser_5                           0.015842
Browser_6                           0.013411
Region_3                            0.013251
Region_1                            0.013209
TrafficType_18                      0.012928
OperatingSystems_8                  0.012406
Month_Aug                           0.012399
Browser_8                           0.012323
OperatingSystems_4                  0.011244
Month_Jul                           0.011030
Region_6                            0.009624
Browser_1                           0.009181
```

```
TrafficType_16                  0.008616
Region_4                        0.008565
TrafficType_19                  0.007814
Region_2                        0.007749
Region_5                        0.006989
Region_8                        0.006382
Region_9                        0.005749
TrafficType_11                  0.005705
TrafficType_9                   0.005366
OperatingSystems_7              0.005308
VisitorType_Other               0.005042
TrafficType_17                  0.004308
TrafficType_12                  0.004308
Browser_9                       0.004308
OperatingSystems_6              0.004258
TrafficType_4                   0.002946
Browser_11                      0.002821
OperatingSystems_5              0.002821
Browser_2                       0.002385
Region_7                        0.001459
TrafficType_14                  0.001155
```

In [131…

```python
# ── Information Value (IV) ──────────────────────────────────────────
def compute_iv(X_df, y, n_bins=10):
    """
    Calcula el Information Value para cada variable.
    Variables continuas se discretizan en cuantiles.
    """
    iv_dict = {}
    y = np.array(y)
    total_pos = y.sum()
    total_neg = len(y) - total_pos

    for col in X_df.columns:
        x = X_df[col].values
        n_unique = len(np.unique(x))

        # Discretizar variables continuas en cuantiles
        if n_unique > n_bins:
            try:
                bins = pd.qcut(x, q=n_bins, duplicates='drop', labels=False)
            except Exception:
                bins = pd.cut(x, bins=n_bins, labels=False)
        else:
            bins = x

        iv = 0.0
        for b in np.unique(bins):
            mask = (bins == b)
            n_pos = y[mask].sum()
            n_neg = mask.sum() - n_pos

            dist_pos = (n_pos + 0.5) / (total_pos + 0.5)
            dist_neg = (n_neg + 0.5) / (total_neg + 0.5)
            woe = np.log(dist_pos / dist_neg)
            iv += (dist_pos - dist_neg) * woe
```

```
        iv_dict[col] = iv

    return pd.Series(iv_dict).sort_values(ascending=False)


iv_series = compute_iv(X_train_df, y_train)

print("=" * 55)
print("INFORMATION VALUE (IV) por variable (descendente)")
print("=" * 55)
print(iv_series.to_string())
print("\nEscala de referencia IV:")
print("  < 0.02 → nulo/inútil | 0.02–0.1 → débil | 0.1–0.3 → medio | >0.3 → fuerte"
```

```
=======================================================
INFORMATION VALUE (IV) por variable (descendente)
=======================================================
PageValues                          2.428687
ExitRates                           0.863661
BounceRates                         0.522602
ProductRelated_Duration             0.495924
ProductRelated                      0.424999
Administrative_Duration             0.242291
Administrative                      0.162155
Month_Nov                           0.143214
TrafficType_2                       0.115694
SpecialDay                          0.105208
VisitorType_New_Visitor             0.067912
VisitorType_Returning_Visitor       0.067033
TrafficType_3                       0.066187
Informational_Duration              0.056979
Informational                       0.053792
Month_May                           0.052790
TrafficType_1                       0.047150
TrafficType_13                      0.041582
OperatingSystems_3                  0.040953
Month_Feb                           0.037275
Month_Mar                           0.032341
OperatingSystems_2                  0.031233
TrafficType_8                       0.019470
Month_Oct                           0.010654
TrafficType_15                      0.009266
Weekend                             0.008953
Browser_3                           0.007778
TrafficType_20                      0.005617
Month_Sep                           0.005056
Month_Dec                           0.005004
TrafficType_10                      0.004872
TrafficType_5                       0.004826
Month_June                          0.004445
TrafficType_6                       0.003753
TrafficType_7                       0.003109
Browser_10                          0.002754
Browser_13                          0.002579
Browser_12                          0.002538
Browser_7                           0.002324
OperatingSystems_1                  0.002132
Browser_4                           0.001989
Browser_5                           0.001893
Browser_6                           0.001351
Region_3                            0.001338
Region_1                            0.001335
OperatingSystems_8                  0.001218
Month_Aug                           0.001195
Browser_8                           0.001108
TrafficType_18                      0.001013
OperatingSystems_4                  0.000994
Month_Jul                           0.000894
TrafficType_16                      0.000811
Region_6                            0.000679
```

```
Browser_1                       0.000632
Region_4                        0.000538
Region_2                        0.000481
OperatingSystems_7              0.000479
Region_5                        0.000417
TrafficType_11                  0.000297
Region_9                        0.000285
Region_8                        0.000272
VisitorType_Other               0.000268
Browser_11                      0.000266
OperatingSystems_5              0.000266
TrafficType_19                  0.000219
TrafficType_9                   0.000108
TrafficType_14                  0.000099
TrafficType_12                  0.000089
TrafficType_17                  0.000089
Browser_9                       0.000089
TrafficType_4                   0.000055
Browser_2                       0.000046
OperatingSystems_6              0.000028
Region_7                        0.000025

Escala de referencia IV:
  < 0.02 → nulo/inútil | 0.02–0.1 → débil | 0.1–0.3 → medio | >0.3 → fuerte
```

Tabla de candidatas a eliminación

In [132…

```python
summary_df = pd.DataFrame({
    'Pearson_abs': pearson_corr,
    'IV': iv_series
}).dropna()

# Umbral de eliminación
PEARSON_THRESHOLD = 0.05
IV_THRESHOLD = 0.02

summary_df['Candidata_eliminacion'] = (
    (summary_df['Pearson_abs'] < PEARSON_THRESHOLD) &
    (summary_df['IV'] < IV_THRESHOLD)
)

candidatas = summary_df[summary_df['Candidata_eliminacion']].index.tolist()

print("=" * 60)
print("TABLA RESUMEN – Análisis Individual de Variables")
print("=" * 60)
print(summary_df.sort_values('IV', ascending=False).to_string())
print(f"\n>>> Candidatas a eliminación ({len(candidatas)} variables):")
for v in candidatas:
    print(f"    - {v}")
```

```
============================================================
TABLA RESUMEN – Análisis Individual de Variables
============================================================
                               Pearson_abs        IV  Candidata_eliminacion
PageValues                        0.503271  2.428687                  False
ExitRates                         0.212674  0.863661                  False
BounceRates                       0.154210  0.522602                  False
ProductRelated_Duration           0.145974  0.495924                  False
ProductRelated                    0.155596  0.424999                  False
Administrative_Duration           0.099236  0.242291                  False
Administrative                    0.142691  0.162155                  False
Month_Nov                         0.145156  0.143214                  False
TrafficType_2                     0.127040  0.115694                  False
SpecialDay                        0.087851  0.105208                  False
VisitorType_New_Visitor           0.101738  0.067912                  False
VisitorType_Returning_Visitor     0.100726  0.067033                  False
TrafficType_3                     0.084708  0.066187                  False
Informational_Duration            0.068662  0.056979                  False
Informational                     0.097372  0.053792                  False
Month_May                         0.079300  0.052790                  False
TrafficType_1                     0.073576  0.047150                  False
TrafficType_13                    0.062969  0.041582                  False
OperatingSystems_3                0.069191  0.040953                  False
Month_Feb                         0.047793  0.037275                  False
Month_Mar                         0.060998  0.032341                  False
OperatingSystems_2                0.063476  0.031233                  False
TrafficType_8                     0.056110  0.019470                  False
Month_Oct                         0.039634  0.010654                   True
TrafficType_15                    0.024786  0.009266                   True
Weekend                           0.034779  0.008953                   True
Browser_3                         0.027132  0.007778                   True
TrafficType_20                    0.028911  0.005617                   True
Month_Sep                         0.026729  0.005056                   True
Month_Dec                         0.025126  0.005004                   True
TrafficType_10                    0.026203  0.004872                   True
TrafficType_5                     0.026377  0.004826                   True
Month_June                        0.022896  0.004445                   True
TrafficType_6                     0.021438  0.003753                   True
TrafficType_7                     0.021619  0.003109                   True
Browser_10                        0.019581  0.002754                   True
Browser_13                        0.019184  0.002579                   True
Browser_12                        0.020182  0.002538                   True
Browser_7                         0.016594  0.002324                   True
OperatingSystems_1                0.016650  0.002132                   True
Browser_4                         0.016233  0.001989                   True
Browser_5                         0.015842  0.001893                   True
Browser_6                         0.013411  0.001351                   True
Region_3                          0.013251  0.001338                   True
Region_1                          0.013209  0.001335                   True
OperatingSystems_8                0.012406  0.001218                   True
Month_Aug                         0.012399  0.001195                   True
Browser_8                         0.012323  0.001108                   True
TrafficType_18                    0.012928  0.001013                   True
OperatingSystems_4                0.011244  0.000994                   True
Month_Jul                         0.011030  0.000894                   True
TrafficType_16                    0.008616  0.000811                   True
```

```
Region_6                        0.009624  0.000679                True
Browser_1                       0.009181  0.000632                True
Region_4                        0.008565  0.000538                True
Region_2                        0.007749  0.000481                True
OperatingSystems_7              0.005308  0.000479                True
Region_5                        0.006989  0.000417                True
TrafficType_11                  0.005705  0.000297                True
Region_9                        0.005749  0.000285                True
Region_8                        0.006382  0.000272                True
VisitorType_Other               0.005042  0.000268                True
OperatingSystems_5              0.002821  0.000266                True
Browser_11                      0.002821  0.000266                True
TrafficType_19                  0.007814  0.000219                True
TrafficType_9                   0.005366  0.000108                True
TrafficType_14                  0.001155  0.000099                True
TrafficType_12                  0.004308  0.000089                True
TrafficType_17                  0.004308  0.000089                True
Browser_9                       0.004308  0.000089                True
TrafficType_4                   0.002946  0.000055                True
Browser_2                       0.002385  0.000046                True
OperatingSystems_6              0.004258  0.000028                True
Region_7                        0.001459  0.000025                True

>>> Candidatas a eliminación (51 variables):
    - Browser_1
    - Browser_10
    - Browser_11
    - Browser_12
    - Browser_13
    - Browser_2
    - Browser_3
    - Browser_4
    - Browser_5
    - Browser_6
    - Browser_7
    - Browser_8
    - Browser_9
    - Month_Aug
    - Month_Dec
    - Month_Jul
    - Month_June
    - Month_Oct
    - Month_Sep
    - OperatingSystems_1
    - OperatingSystems_4
    - OperatingSystems_5
    - OperatingSystems_6
    - OperatingSystems_7
    - OperatingSystems_8
    - Region_1
    - Region_2
    - Region_3
    - Region_4
    - Region_5
    - Region_6
    - Region_7
```

- Region_8
- Region_9
- TrafficType_10
- TrafficType_11
- TrafficType_12
- TrafficType_14
- TrafficType_15
- TrafficType_16
- TrafficType_17
- TrafficType_18
- TrafficType_19
- TrafficType_20
- TrafficType_4
- TrafficType_5
- TrafficType_6
- TrafficType_7
- TrafficType_9
- VisitorType_Other
- Weekend

Visualización: heatmap de correlación (top 20) y barras IV

In [133…

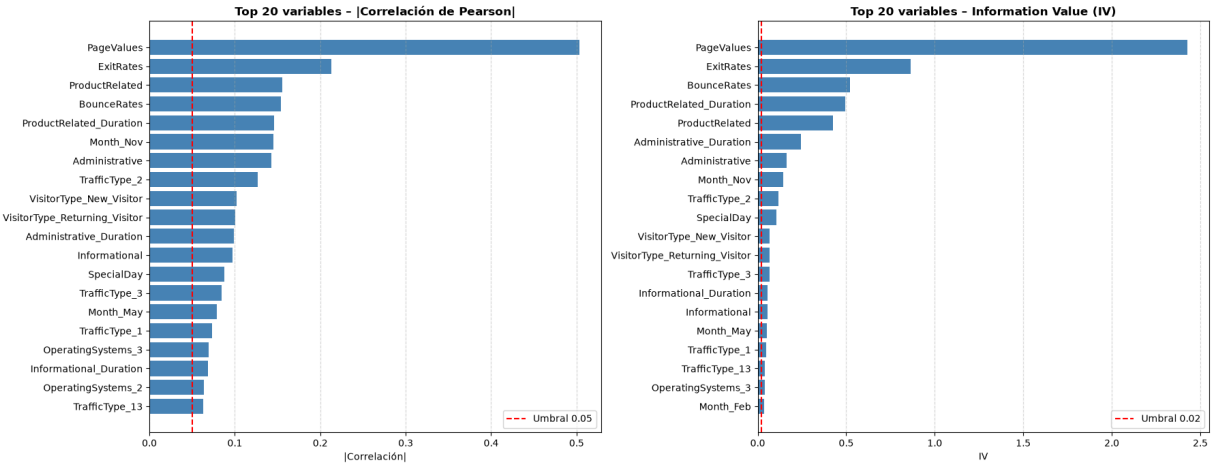
```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# Barras – Pearson
top20_pearson = pearson_corr.head(20)
colors_p = ['steelblue' if v >= PEARSON_THRESHOLD else 'salmon' for v in top20_pear
axes[0].barh(top20_pearson.index[::-1], top20_pearson.values[::-1], color=colors_p[
axes[0].axvline(PEARSON_THRESHOLD, color='red', linestyle='--', linewidth=1.5, labe
axes[0].set_title('Top 20 variables – |Correlación de Pearson|', fontweight='bold')
axes[0].set_xlabel('|Correlación|')
axes[0].legend()
axes[0].grid(True, axis='x', linestyle='--', alpha=0.5)

# Barras – IV
top20_iv = iv_series.head(20)
colors_iv = ['steelblue' if v >= IV_THRESHOLD else 'salmon' for v in top20_iv]
axes[1].barh(top20_iv.index[::-1], top20_iv.values[::-1], color=colors_iv[::-1])
axes[1].axvline(IV_THRESHOLD, color='red', linestyle='--', linewidth=1.5, label=f'U
axes[1].set_title('Top 20 variables – Information Value (IV)', fontweight='bold')
axes[1].set_xlabel('IV')
axes[1].legend()
axes[1].grid(True, axis='x', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig('5_1_analisis_variables.png', dpi=150, bbox_inches='tight')
plt.show()
print("Gráfica guardada en: 5_1_analisis_variables.png")
```




```
Gráfica guardada en: 5_1_analisis_variables.png
```

Extracción de Características Lineal – PCA

Análisis de varianza explicada por PCA

In [134…

```python
scaler_pca = StandardScaler()
X_train_scaled = scaler_pca.fit_transform(X_train)
X_test_scaled  = scaler_pca.transform(X_test)

n_vars = X_train_scaled.shape[1]
pca_full = PCA(n_components=n_vars, random_state=42)
pca_full.fit(X_train_scaled)

varianza_acum = np.cumsum(pca_full.explained_variance_ratio_)

# Número de componentes para ≥ 95% de varianza
VARIANZA_OBJETIVO = 0.95
n_componentes_pca = int(np.argmax(varianza_acum >= VARIANZA_OBJETIVO)) + 1
reduccion_pca = round((1 - n_componentes_pca / n_vars) * 100, 1)

print(f"Variables originales:        {n_vars}")
print(f"Componentes PCA (≥95% var):  {n_componentes_pca}")
print(f"Varianza explicada:          {varianza_acum[n_componentes_pca-1]*100:.2f}%"
print(f"Porcentaje de reducción:     {reduccion_pca}%")
```

```
Variables originales:        74
Componentes PCA (≥95% var):  57
Varianza explicada:          95.34%
Porcentaje de reducción:     23.0%
```

Gráfica de varianza explicada

In [135…

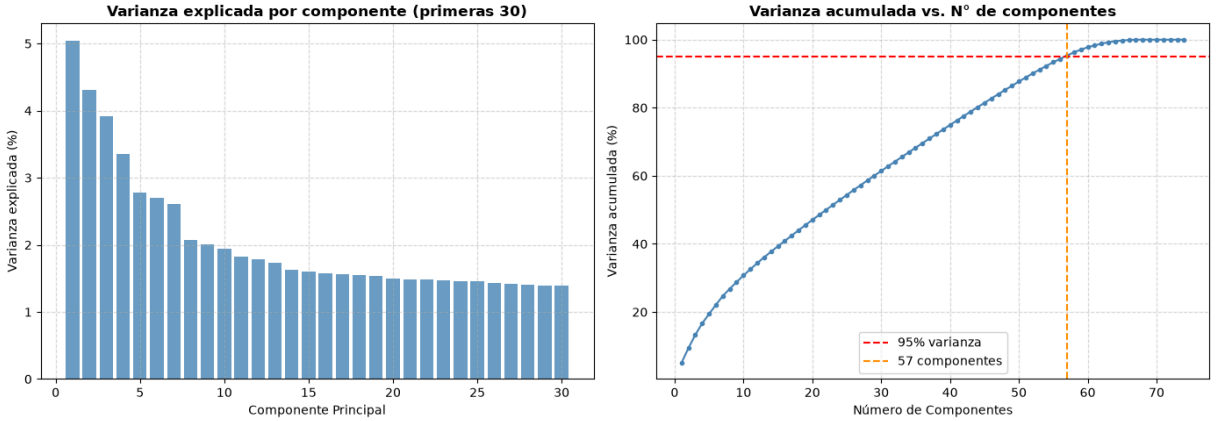
```python
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(range(1, min(31, n_vars+1)),
            pca_full.explained_variance_ratio_[:30] * 100,
            color='steelblue', alpha=0.8)
axes[0].set_title('Varianza explicada por componente (primeras 30)', fontweight='bo
```

```
axes[0].set_xlabel('Componente Principal')
axes[0].set_ylabel('Varianza explicada (%)')
axes[0].grid(True, linestyle='--', alpha=0.5)

axes[1].plot(range(1, n_vars+1), varianza_acum * 100, 'o-', color='steelblue', mark
axes[1].axhline(95, color='red', linestyle='--', label='95% varianza')
axes[1].axvline(n_componentes_pca, color='darkorange', linestyle='--',
                label=f'{n_componentes_pca} componentes')
axes[1].set_title('Varianza acumulada vs. N° de componentes', fontweight='bold')
axes[1].set_xlabel('Número de Componentes')
axes[1].set_ylabel('Varianza acumulada (%)')
axes[1].legend()
axes[1].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig('5_2_pca_varianza.png', dpi=150, bbox_inches='tight')
plt.show()
print("Gráfica guardada en: 5_2_pca_varianza.png")
```




```
Gráfica guardada en: 5_2_pca_varianza.png
```

Transformación PCA con n componentes seleccionados

In [136…

```
pca = PCA(n_components=n_componentes_pca, random_state=42)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca  = pca.transform(X_test_scaled)

print(f"Shape train PCA: {X_train_pca.shape}")
print(f"Shape test  PCA: {X_test_pca.shape}")
```

```
Shape train PCA: (9864, 57)
Shape test  PCA: (2466, 57)
```

CV StratifiedKFold reutilizado

In [137…

```
from sklearn.metrics import average_precision_score
cv_strategy = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

def evaluar_modelo(nombre, modelo, X_tr, y_tr, X_te, y_te, cv):
    modelo.fit(X_tr, y_tr)
```

```
    y_pred       = modelo.predict(X_te)
    y_pred_proba = modelo.predict_proba(X_te)[:, 1]
    cv_auc = cross_val_score(modelo, X_tr, y_tr, cv=cv, scoring='roc_auc', n_jobs=-
    return {
        'Modelo': nombre,
        'ROC-AUC CV (media)': round(cv_auc.mean(), 4),
        'ROC-AUC CV (std)': round(cv_auc.std(), 4),
        'ROC-AUC Test': round(roc_auc_score(y_te, y_pred_proba), 4),
        'auPR Test': round(average_precision_score(y_te, y_pred_proba), 4),
        'Accuracy Test': round(accuracy_score(y_te, y_pred), 4),
        'F1 Test': round(f1_score(y_te, y_pred), 4),
        'Precision Test': round(precision_score(y_te, y_pred), 4),
        'Recall Test': round(recall_score(y_te, y_pred), 4),
    }
```

RF con PCA

In [138…

```
# Se extraen los mejores hiperparámetros del RF (sin reducción)
# y se re-entrena sobre los componentes PCA.

best_params_rf = grid_rf.best_params_   # del notebook principal

rf_pca = ImbPipeline([
    ('scaler', StandardScaler()),

    ('pca', PCA(
        n_components=n_componentes_pca,
        random_state=42
    )),

    ('smote', SMOTE(
        random_state=42,
        sampling_strategy=best_params_rf.get(
            'smote__sampling_strategy',0.3
        ),
        k_neighbors=best_params_rf.get(
            'smote__k_neighbors',5
        )
    )),

    ('classifier', RandomForestClassifier(
        n_estimators=best_params_rf.get(
            'classifier__n_estimators',200
        ),
        max_depth=best_params_rf.get(
            'classifier__max_depth',10
        ),
        min_samples_leaf=best_params_rf.get(
            'classifier__min_samples_leaf',1
        ),
        max_features=best_params_rf.get(
            'classifier__max_features','sqrt'
        ),
        class_weight='balanced',
```

```
        random_state=42,
        n_jobs=-1
    ))
])
resultado_rf_pca = evaluar_modelo(
    'RF + PCA',
    rf_pca,
    X_train,
    y_train,
    X_test,
    y_test,
    cv_strategy
)
print(resultado_rf_pca)
```

```
{'Modelo': 'RF + PCA', 'ROC-AUC CV (media)': np.float64(0.8645), 'ROC-AUC CV (std)': np.float64(0.0198), 'ROC-AUC Test': 0.8435, 'auPR Test': 0.5624, 'Accuracy Test': 0.8305, 'F1 Test': 0.5261, 'Precision Test': 0.464, 'Recall Test': 0.6073}
```

XGBoost con PCA

In [139…

```
best_params_xgb = grid_xgb.best_params_   # del notebook principal

xgb_pca = ImbPipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(
        n_components=n_componentes_pca,
        random_state=42
    )),
    ('smote', SMOTE(
        random_state=42,
        sampling_strategy=best_params_xgb.get('smote__sampling_strategy', 0.3),
        k_neighbors=best_params_xgb.get('smote__k_neighbors', 5)
    )),
    ('classifier', XGBClassifier(
        n_estimators=best_params_xgb.get('classifier__n_estimators', 200),
        max_depth=best_params_xgb.get('classifier__max_depth', 6),
        learning_rate=best_params_xgb.get('classifier__learning_rate', 0.1),
        subsample=best_params_xgb.get('classifier__subsample', 0.8),
        colsample_bytree=best_params_xgb.get('classifier__colsample_bytree', 0.8),
        eval_metric='logloss',
        verbosity=0,
        random_state=42,
        n_jobs=-1
    ))
])

resultado_xgb_pca = evaluar_modelo(
    f'XGBoost + PCA ({n_componentes_pca} comp)',
    xgb_pca, X_train, y_train, X_test, y_test, cv_strategy
)
print(resultado_xgb_pca)
```

```
{'Modelo': 'XGBoost + PCA (57 comp)', 'ROC-AUC CV (media)': np.float64(0.8749), 'ROC
-AUC CV (std)': np.float64(0.0184), 'ROC-AUC Test': 0.858, 'auPR Test': 0.5913, 'Acc
uracy Test': 0.8808, 'F1 Test': 0.5532, 'Precision Test': 0.6594, 'Recall Test': 0.4
764}
```

Resultados baseline (sin reducción)

In [140…

```python
# Se recalculan con la misma función para garantizar comparabilidad

resultado_rf_base = evaluar_modelo(
    f'RF baseline ({n_vars} vars)',
    best_rf, X_train, y_train, X_test, y_test, cv_strategy
)

resultado_xgb_base = evaluar_modelo(
    f'XGBoost baseline ({n_vars} vars)',
    best_xgb, X_train, y_train, X_test, y_test, cv_strategy
)

print(resultado_rf_base)
print(resultado_xgb_base)
```

```
{'Modelo': 'RF baseline (74 vars)', 'ROC-AUC CV (media)': np.float64(0.924), 'ROC-AU
C CV (std)': np.float64(0.0112), 'ROC-AUC Test': 0.9127, 'auPR Test': 0.687, 'Accura
cy Test': 0.8617, 'F1 Test': 0.6444, 'Precision Test': 0.5355, 'Recall Test': 0.808
9}
{'Modelo': 'XGBoost baseline (74 vars)', 'ROC-AUC CV (media)': np.float64(0.934), 'R
OC-AUC CV (std)': np.float64(0.0077), 'ROC-AUC Test': 0.9308, 'auPR Test': 0.7405,
'Accuracy Test': 0.8978, 'F1 Test': 0.6753, 'Precision Test': 0.665, 'Recall Test':
0.6859}
```

Tabla comparativa PCA

In [141…

```python
tabla_pca = pd.DataFrame([
    resultado_rf_base,
    resultado_rf_pca,
    resultado_xgb_base,
    resultado_xgb_pca,
])

tabla_pca['Reducción (%)'] = [
    '0%',
    f'{reduccion_pca}%',
    '0%',
    f'{reduccion_pca}%',
]

tabla_pca['N° dimensiones'] = [
    n_vars, n_componentes_pca,
    n_vars, n_componentes_pca
]

print("\n" + "=" * 90)
print("TABLA COMPARATIVA – Resultados con PCA vs Baseline")
```

```
print("=" * 90)
print(tabla_pca.to_string(index=False))
```

```
==========================================================================================
======
TABLA COMPARATIVA – Resultados con PCA vs Baseline
==========================================================================================
======
                   Modelo  ROC-AUC CV (media)  ROC-AUC CV (std)  ROC-AUC Test  auPR
Test  Accuracy Test  F1 Test  Precision Test  Recall Test Reducción (%)  N° dimensio
nes
     RF baseline (74 vars)              0.9240            0.0112        0.9127
0.6870         0.8617   0.6444          0.5355       0.8089            0%
74
                  RF + PCA              0.8645            0.0198        0.8435
0.5624         0.8305   0.5261          0.4640       0.6073         23.0%
57
XGBoost baseline (74 vars)              0.9340            0.0077        0.9308
0.7405         0.8978   0.6753          0.6650       0.6859            0%
74
   XGBoost + PCA (57 comp)              0.8749            0.0184        0.8580
0.5913         0.8808   0.5532          0.6594       0.4764         23.0%
57
```

Extracción de Características No Lineal – UMAP

Se evalúa el desempeño (ROC-AUC en validación cruzada) de ambos modelos para n_components ∈ {2, 5, 10, 15, 20}. Se selecciona el número de componentes que maximice el ROC-AUC en CV sobre el conjunto de entrenamiento, buscando el mejor balance entre compresión y capacidad discriminativa.

Búsqueda del número óptimo de componentes UMAP

In [142…

```
from umap import UMAP

UMAP_COMPONENTES_EVAL = [2,5,10,15,20]

resultados_busqueda_umap = []

for n_comp in UMAP_COMPONENTES_EVAL:

    print(f"\nEvaluando UMAP con {n_comp} componentes")

    rf_pipe = ImbPipeline([

        ('scaler', StandardScaler()),

        ('umap', UMAP(
            n_components=n_comp,
            random_state=42,
            n_jobs=1
```

```
    )),

    ('smote', SMOTE(
        random_state=42,
        sampling_strategy=0.3
    )),

    ('classifier', RandomForestClassifier(
        n_estimators=200,
        max_depth=10,
        class_weight='balanced',
        random_state=42,
        n_jobs=-1
    ))
])

auc_rf = cross_val_score(
    rf_pipe,
    X_train,
    y_train,
    cv=cv_strategy,
    scoring='roc_auc',
    n_jobs=-1
).mean()

xgb_pipe = ImbPipeline([

    ('scaler', StandardScaler()),

    ('umap', UMAP(
        n_components=n_comp,
        random_state=42,
        n_jobs=1
    )),

    ('smote', SMOTE(
        random_state=42,
        sampling_strategy=0.3
    )),

    ('classifier', XGBClassifier(
        n_estimators=200,
        max_depth=6,
        learning_rate=0.1,
        eval_metric='logloss',
        verbosity=0,
        random_state=42,
        n_jobs=-1
    ))
])

auc_xgb = cross_val_score(
    xgb_pipe,
    X_train,
    y_train,
    cv=cv_strategy,
```

```
        scoring='roc_auc',
        n_jobs=-1
    ).mean()

    resultados_busqueda_umap.append({
        'n_components': n_comp,
        'ROC-AUC CV RF': round(auc_rf,4),
        'ROC-AUC CV XGB': round(auc_xgb,4)
    })
```

```
Evaluando UMAP con 2 componentes

Evaluando UMAP con 5 componentes

Evaluando UMAP con 10 componentes

Evaluando UMAP con 15 componentes

Evaluando UMAP con 20 componentes
```

Selección del n_components óptimo

Se toma el n_components que maximiza el ROC-AUC medio (RF + XGB)

In [143…

```
df_busqueda_umap = pd.DataFrame(resultados_busqueda_umap)
print(df_busqueda_umap)
```

```
   n_components  ROC-AUC CV RF  ROC-AUC CV XGB
0             2         0.5608          0.5579
1             5         0.5384          0.5450
2            10         0.5435          0.5548
3            15         0.5552          0.5512
4            20         0.5500          0.5451
```

In [144…

```
print("\n" + "=" * 50)
print("BÚSQUEDA DE COMPONENTES UMAP")
print("=" * 50)
print(df_busqueda_umap.to_string(index=False))
```

```
==================================================
BÚSQUEDA DE COMPONENTES UMAP
==================================================
 n_components  ROC-AUC CV RF  ROC-AUC CV XGB
            2         0.5608          0.5579
            5         0.5384          0.5450
           10         0.5435          0.5548
           15         0.5552          0.5512
           20         0.5500          0.5451
```

In [145…

```
df_busqueda_umap['ROC-AUC CV medio'] = (
    df_busqueda_umap['ROC-AUC CV RF'] + df_busqueda_umap['ROC-AUC CV XGB']
) / 2

n_componentes_umap = df_busqueda_umap.loc[
    df_busqueda_umap['ROC-AUC CV medio'].idxmax(), 'n_components'
```

```
]
reduccion_umap = round((1 - n_componentes_umap / n_vars) * 100, 1)

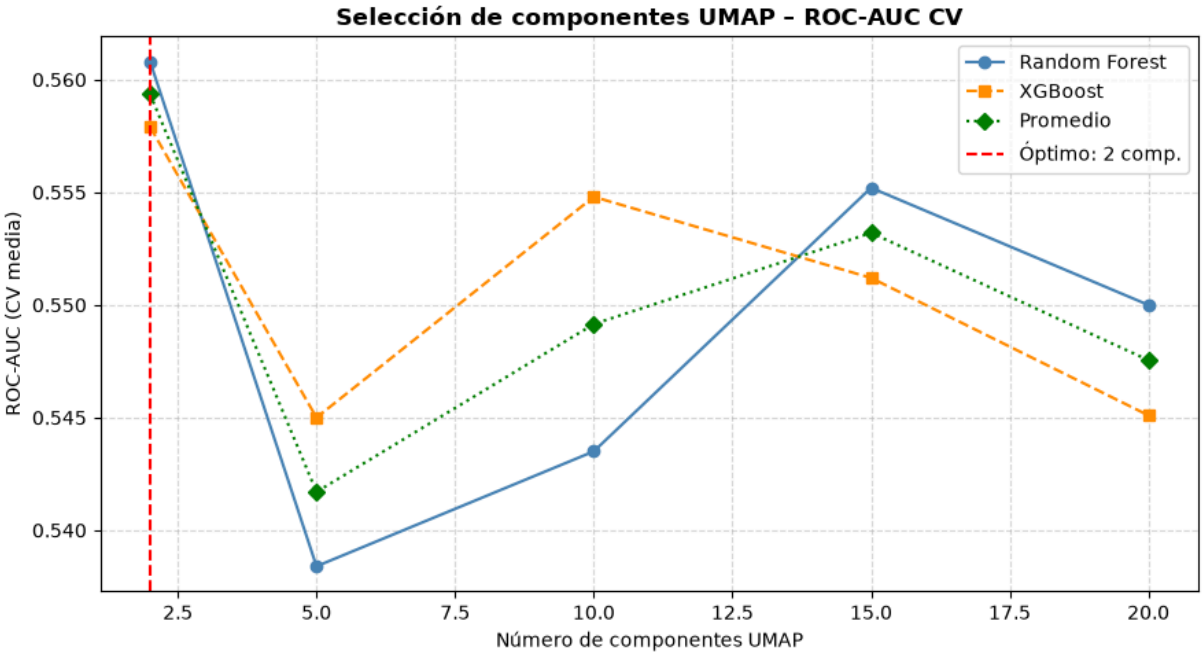
print(f"Número óptimo de componentes UMAP: {n_componentes_umap}")
print(f"Porcentaje de reducción:           {reduccion_umap}%")
```

```
Número óptimo de componentes UMAP: 2
Porcentaje de reducción:           97.3%
```

Búsqueda UMAP

In [146…

```
fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(df_busqueda_umap['n_components'], df_busqueda_umap['ROC-AUC CV RF'],
        'o-', color='steelblue', label='Random Forest')
ax.plot(df_busqueda_umap['n_components'], df_busqueda_umap['ROC-AUC CV XGB'],
        's--', color='darkorange', label='XGBoost')
ax.plot(df_busqueda_umap['n_components'], df_busqueda_umap['ROC-AUC CV medio'],
        'D:', color='green', label='Promedio')
ax.axvline(n_componentes_umap, color='red', linestyle='--',
           label=f'Óptimo: {n_componentes_umap} comp.')
ax.set_title('Selección de componentes UMAP – ROC-AUC CV', fontweight='bold')
ax.set_xlabel('Número de componentes UMAP')
ax.set_ylabel('ROC-AUC (CV media)')
ax.legend()
ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('5_3_umap_busqueda.png', dpi=150, bbox_inches='tight')
plt.show()
print("Gráfica guardada en: 5_3_umap_busqueda.png")
```




```
Gráfica guardada en: 5_3_umap_busqueda.png
```

Transformación UMAP con n_componentes óptimos

In [147…

```
reducer_final = UMAP(n_components=n_componentes_umap, random_state=42, n_jobs=1)
X_train_umap = reducer_final.fit_transform(X_train_scaled)
X_test_umap  = reducer_final.transform(X_test_scaled)

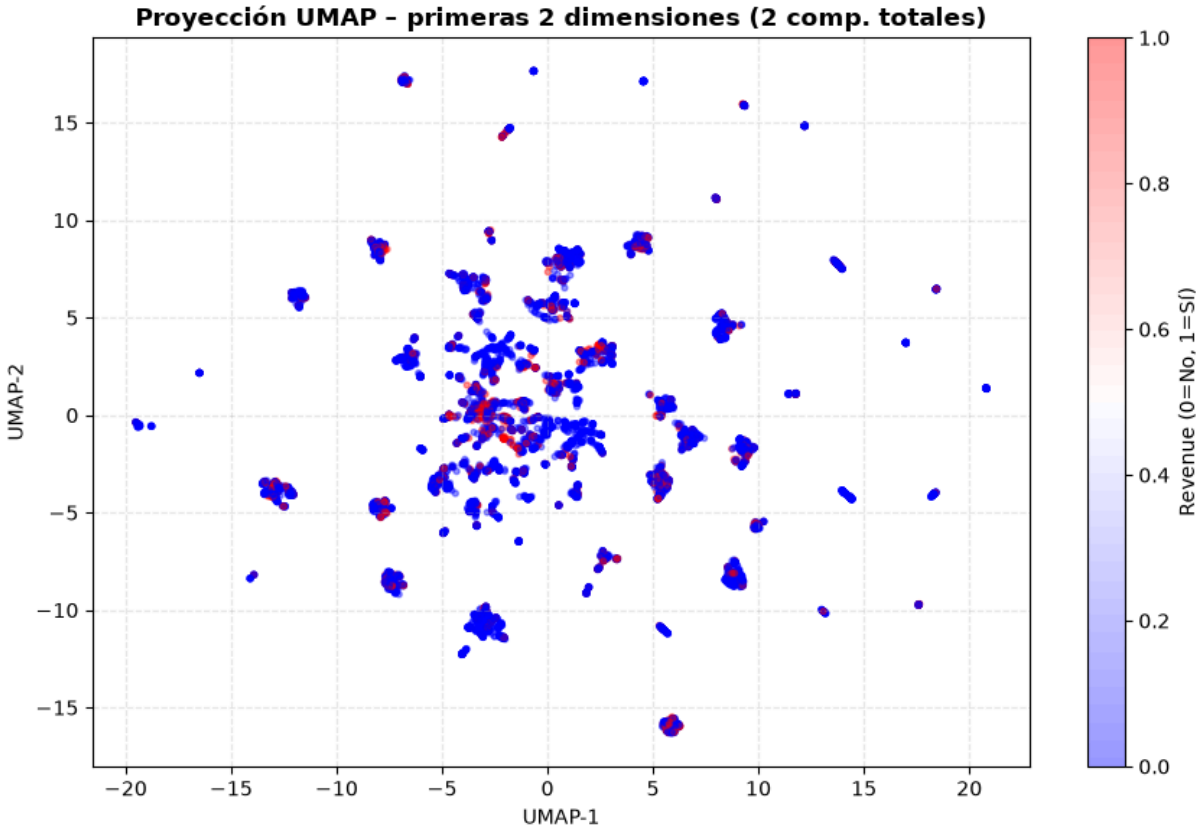
print(f"Shape train UMAP: {X_train_umap.shape}")
print(f"Shape test  UMAP: {X_test_umap.shape}")
```

```
Shape train UMAP: (9864, 2)
Shape test  UMAP: (2466, 2)
```

Visualización 2D del espacio UMAP (si n_comp ≥ 2)

In [148…

```
if n_componentes_umap >= 2:
    fig, ax = plt.subplots(figsize=(9, 6))
    scatter = ax.scatter(
        X_train_umap[:, 0], X_train_umap[:, 1],
        c=np.array(y_train).ravel(), cmap='bwr', alpha=0.4, s=8
    )
    plt.colorbar(scatter, ax=ax, label='Revenue (0=No, 1=Sí)')
    ax.set_title(f'Proyección UMAP – primeras 2 dimensiones ({n_componentes_umap} c
                 fontweight='bold')
    ax.set_xlabel('UMAP-1')
    ax.set_ylabel('UMAP-2')
    ax.grid(True, linestyle='--', alpha=0.3)
    plt.tight_layout()
    plt.savefig('5_3_umap_2d.png', dpi=150, bbox_inches='tight')
    plt.show()
    print("Gráfica guardada en: 5_3_umap_2d.png")
```




```
Gráfica guardada en: 5_3_umap_2d.png
```

RF con UMAP

In [149…

```
from umap import UMAP

rf_umap = ImbPipeline([
    ('scaler', StandardScaler()),

    ('umap', UMAP(
        n_components=n_componentes_umap,
        random_state=42,
        n_jobs=1
    )),

    ('smote', SMOTE(
        random_state=42,
        sampling_strategy=best_params_rf.get('smote__sampling_strategy', 0.3),
        k_neighbors=best_params_rf.get('smote__k_neighbors', 5)
    )),

    ('classifier', RandomForestClassifier(
        n_estimators=best_params_rf.get('classifier__n_estimators', 200),
        max_depth=best_params_rf.get('classifier__max_depth', 10),
        min_samples_leaf=best_params_rf.get('classifier__min_samples_leaf', 1),
        max_features=best_params_rf.get('classifier__max_features', 'sqrt'),
        class_weight='balanced',
        random_state=42,
        n_jobs=-1
    ))
])

resultado_rf_umap = evaluar_modelo(
    f'RF + UMAP ({n_componentes_umap} comp)',
    rf_umap,
    X_train,
    y_train,
    X_test,
    y_test,
    cv_strategy
)

print(resultado_rf_umap)
```

```
{'Modelo': 'RF + UMAP (2 comp)', 'ROC-AUC CV (media)': np.float64(0.5583), 'ROC-AUC
CV (std)': np.float64(0.0302), 'ROC-AUC Test': 0.5942, 'auPR Test': 0.2774, 'Accurac
y Test': 0.8475, 'F1 Test': 0.245, 'Precision Test': 0.5259, 'Recall Test': 0.1597}
```

XGBoost con UMAP

In [150…

```
xgb_umap = ImbPipeline([
    ('scaler', StandardScaler()),

    ('umap', UMAP(
        n_components=n_componentes_umap,
        random_state=42,
```

```
        n_jobs=1
    )),

    ('smote', SMOTE(
        random_state=42,
        sampling_strategy=best_params_xgb.get('smote__sampling_strategy', 0.3),
        k_neighbors=best_params_xgb.get('smote__k_neighbors', 5)
    )),

    ('classifier', XGBClassifier(
        n_estimators=best_params_xgb.get('classifier__n_estimators', 200),
        max_depth=best_params_xgb.get('classifier__max_depth', 6),
        learning_rate=best_params_xgb.get('classifier__learning_rate', 0.1),
        subsample=best_params_xgb.get('classifier__subsample', 0.8),
        colsample_bytree=best_params_xgb.get('classifier__colsample_bytree', 0.8),
        eval_metric='logloss',
        verbosity=0,
        random_state=42,
        n_jobs=-1
    ))
])

resultado_xgb_umap = evaluar_modelo(
    f'XGBoost + UMAP ({n_componentes_umap} comp)',
    xgb_umap,
    X_train,
    y_train,
    X_test,
    y_test,
    cv_strategy
)

print(resultado_xgb_umap)
```

```
{'Modelo': 'XGBoost + UMAP (2 comp)', 'ROC-AUC CV (media)': np.float64(0.5715), 'ROC
-AUC CV (std)': np.float64(0.0361), 'ROC-AUC Test': 0.598, 'auPR Test': 0.2826, 'Acc
uracy Test': 0.839, 'F1 Test': 0.2321, 'Precision Test': 0.4444, 'Recall Test': 0.15
71}
```

Tabla comparativa

In [151…

```
tabla_final = pd.DataFrame([
    resultado_rf_base,
    resultado_rf_pca,
    resultado_rf_umap,
    resultado_xgb_base,
    resultado_xgb_pca,
    resultado_xgb_umap,
])

tabla_final['N° dimensiones'] = [
    n_vars, n_componentes_pca, n_componentes_umap,
    n_vars, n_componentes_pca, n_componentes_umap
]
tabla_final['Reducción (%)'] = [
```

```
    '0%', f'{reduccion_pca}%', f'{reduccion_umap}%',
    '0%', f'{reduccion_pca}%', f'{reduccion_umap}%',
]

print("\n" + "=" * 110)
print("TABLA COMPARATIVA COMPLETA – Baseline vs PCA vs UMAP")
print("=" * 110)
print(tabla_final.to_string(index=False))
```

```
==============================================================================================
==========================
TABLA COMPARATIVA COMPLETA – Baseline vs PCA vs UMAP
==============================================================================================
==========================
                    Modelo  ROC-AUC CV (media)  ROC-AUC CV (std)  ROC-AUC Test  auPR
Test  Accuracy Test  F1 Test  Precision Test  Recall Test  N° dimensiones Reducción
(%)
     RF baseline (74 vars)              0.9240            0.0112        0.9127
0.6870         0.8617   0.6444          0.5355       0.8089               74
0%
                  RF + PCA              0.8645            0.0198        0.8435
0.5624         0.8305   0.5261          0.4640       0.6073               57
23.0%
        RF + UMAP (2 comp)              0.5583            0.0302        0.5942
0.2774         0.8475   0.2450          0.5259       0.1597                2
97.3%
XGBoost baseline (74 vars)              0.9340            0.0077        0.9308
0.7405         0.8978   0.6753          0.6650       0.6859               74
0%
   XGBoost + PCA (57 comp)              0.8749            0.0184        0.8580
0.5913         0.8808   0.5532          0.6594       0.4764               57
23.0%
   XGBoost + UMAP (2 comp)              0.5715            0.0361        0.5980
0.2826         0.8390   0.2321          0.4444       0.1571                2
97.3%
```

Gráfica comparativa de ROC-AUC Test

In [152…

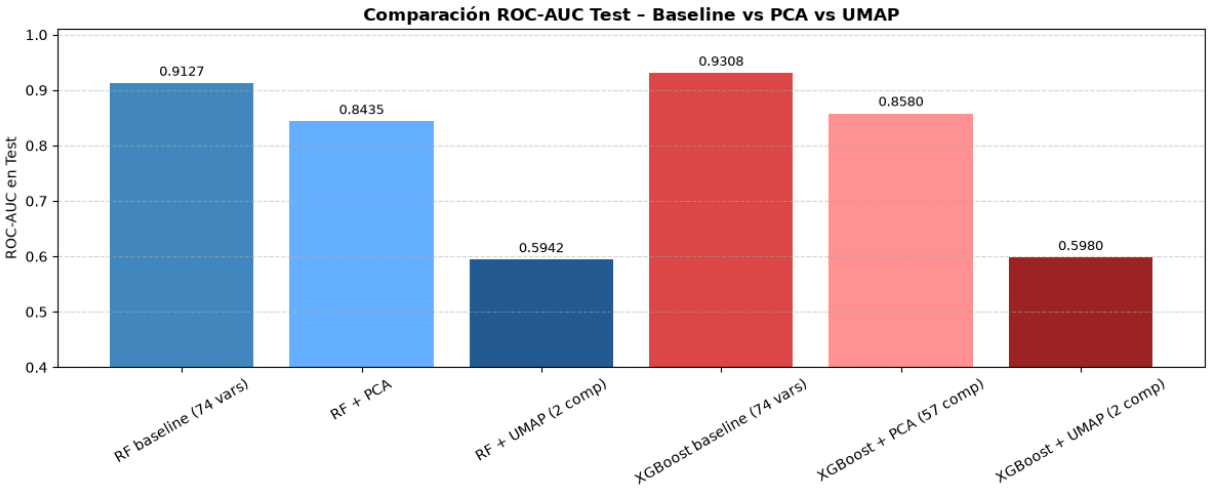
```
fig, ax = plt.subplots(figsize=(12, 5))

modelos   = tabla_final['Modelo'].tolist()
auc_tests = tabla_final['ROC-AUC Test'].tolist()
colores   = ['#1f77b4', '#4da6ff', '#003f7f',
             '#d62728', '#ff7f7f', '#8c0000']

bars = ax.bar(modelos, auc_tests, color=colores, alpha=0.85)
ax.bar_label(bars, fmt='%.4f', padding=3, fontsize=9)
ax.set_ylim(0.40, 1.01)
ax.set_ylabel('ROC-AUC en Test')
ax.set_title('Comparación ROC-AUC Test – Baseline vs PCA vs UMAP', fontweight='bold
ax.tick_params(axis='x', rotation=30)
ax.grid(True, axis='y', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig('5_comparativa_reduccion.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
print("Gráfica guardada en: 5_comparativa_reduccion.png")
```




```
Gráfica guardada en: 5_comparativa_reduccion.png
```

Discusión y Conclusiones

Análisis individual de variables

El análisis de correlación de Pearson e Information Value mostró que **PageValues** es, con amplia diferencia, la variable más discriminativa de `Revenue` (IV = 2,43; |r| = 0,50). Le siguen `ExitRates` (IV 0,86), `BounceRates` (0,52), `ProductRelated_Duration` (0,50) y `ProductRelated` (0,42), con IV fuerte. Con los criterios |Pearson| < 0,05 e IV < 0,02 resultaron **51 candidatas a eliminación**, todas indicadoras de variables nominales (`Browser_` , `Region_` , `TrafficType_` , `OperatingSystems_` , algunos meses, `VisitorType_Other` y `Weekend`). No se eliminaron: los modelos de árboles pueden aprovechar relaciones no lineales que este análisis univariado no captura.

Reducción con PCA

Con el criterio de varianza acumulada ≥ 95 %, PCA conservó **57** componentes de 74 variables (reducción de **23,0 %**). El ROC-AUC en test pasó de **0,912** (RF) y **0,930** (XGBoost) en el baseline a **0,839** y **0,856** con PCA; el F1 pasó de **0,640** y **0,667** a **0,531** y **0,543**. UMAP (2 componentes, reducción de 97,3 %) llevó el ROC-AUC a **0,538** (RF) y **0,593** (XGBoost), cercano al azar. Ninguna reducción mejora el desempeño; se recomienda conservar las 74 variables.

Comparación con la literatura

Abdullah-All-Tanvir et al. reportan un AUROC de 0,937 y un auPR de 0,754 con XGBoost (partición 70/30), y Alamuri y Bondalapu un ROC-AUC de 0,886 con SVM y 0,882 con

Random Forest (70/30 y validación cruzada de 10 pliegues). Aquí el ROC-AUC en test fue **0,930** (XGBoost) y **0,912** (Random Forest), con auPR de **0,734** en XGBoost, validación cruzada estratificada de 10 pliegues y preprocesamiento dentro del pipeline. Las particiones y el tratamiento del sobremuestreo difieren entre estudios, por lo que la comparación es solo orientativa.

In [153…

```
from sklearn.metrics import (roc_auc_score, f1_score, balanced_accuracy_score,
                             average_precision_score, precision_score, recall_score
from sklearn.model_selection import cross_val_score
import numpy as np, pandas as pd

def ic_bootstrap(y_true, y_pred, y_prob, n=1000, seed=42):
    rng = np.random.RandomState(seed)
    y_true, y_pred, y_prob = map(np.asarray, (y_true, y_pred, y_prob))
    aucs, f1s = [], []
    for _ in range(n):
        idx = rng.randint(0, len(y_true), len(y_true))
        if len(np.unique(y_true[idx])) < 2:
            continue
        aucs.append(roc_auc_score(y_true[idx], y_prob[idx]))
        f1s.append(f1_score(y_true[idx], y_pred[idx]))
    return np.percentile(aucs, [2.5, 97.5]), np.percentile(f1s, [2.5, 97.5])

grids = {'Random Forest': grid_rf, 'XGBoost': grid_xgb, 'MLP': grid_mlp, 'SVM': gri
         'Reg. logistica': grid_lr, 'KNN': grid_knn, 'K-Means': grid_kmeans, 'GMM':

filas = []
for nombre, g in grids.items():
    print("\n==", nombre, "==")
    print("best_params_:", g.best_params_)
    m = g.best_estimator_
    prob = m.predict_proba(X_test)[:, 1]
    pred = m.predict(X_test)
    i = g.best_index_
    auc_cv = cross_val_score(m, X_train, y_train, cv=cv_strategy, scoring='roc_auc'
    ic_auc, ic_f1 = ic_bootstrap(y_test, pred, prob)
    filas.append({
        'Modelo': nombre,
        'BA train': round(balanced_accuracy_score(y_train, m.predict(X_train)), 4),
        'BA CV': f"{g.cv_results_['mean_test_score'][i]:.4f} ± {g.cv_results_['std_
        'AUC CV': f"{auc_cv.mean():.4f} ± {auc_cv.std():.4f}",
        'AUC test': round(roc_auc_score(y_test, prob), 4),
        'IC AUC': f"[{ic_auc[0]:.4f}, {ic_auc[1]:.4f}]",
        'auPR test': round(average_precision_score(y_test, prob), 4),
        'F1 test': round(f1_score(y_test, pred), 4),
        'IC F1': f"[{ic_f1[0]:.4f}, {ic_f1[1]:.4f}]",
        'Precision': round(precision_score(y_test, pred), 4),
        'Recall': round(recall_score(y_test, pred), 4),
    })

print("\n\n===== TABLA RESUMEN =====")
print(pd.DataFrame(filas).to_string(index=False))

print("\n\n===== REDUCCIÓN DE DIMENSIÓN =====")
```

```
print(tabla_final.to_string(index=False))

print("\n\n===== IV Y PEARSON (top 15 por IV) =====")
print(summary_df.sort_values('IV', ascending=False).head(15).to_string())
print("Candidatas a eliminación:", len(candidatas))
```

```
== Random Forest ==
best_params_: {'classifier__max_depth': 10, 'classifier__max_features': 'sqrt', 'cla
ssifier__min_samples_leaf': 4, 'classifier__n_estimators': 200, 'smote__k_neighbor
s': 7, 'smote__sampling_strategy': 0.22}

== XGBoost ==
best_params_: {'classifier__colsample_bytree': 0.8, 'classifier__learning_rate': 0.0
5, 'classifier__max_depth': 3, 'classifier__n_estimators': 200, 'classifier__subsamp
le': 0.8, 'smote__k_neighbors': 3, 'smote__sampling_strategy': 0.4}

== MLP ==
best_params_: {'classifier__activation': 'relu', 'classifier__alpha': 0.001, 'classi
fier__hidden_layer_sizes': (50,), 'classifier__learning_rate_init': 0.001, 'smote__k
_neighbors': 5, 'smote__sampling_strategy': 0.3}

== SVM ==
best_params_: {'smote__sampling_strategy': 0.3, 'svc__C': 1, 'svc__kernel': 'linea
r'}

== Reg. logistica ==
best_params_: {'classifier__C': 1, 'classifier__penalty': 'l2', 'classifier__solve
r': 'lbfgs', 'smote__k_neighbors': 3, 'smote__sampling_strategy': 0.4}

== KNN ==
best_params_: {'classifier__metric': 'euclidean', 'classifier__n_neighbors': 11, 'cl
assifier__weights': 'distance', 'smote__sampling_strategy': 0.4}

== K-Means ==
best_params_: {'classifier__n_clusters': 4, 'smote__k_neighbors': 7, 'smote__samplin
g_strategy': 0.4}

== GMM ==
best_params_: {'gmm__covariance_type': 'spherical', 'gmm__init_params': 'random', 'g
mm__max_iter': 100, 'gmm__n_components': 2, 'gmm__n_init': 2, 'smote__sampling_strat
egy': 0.3}


===== TABLA RESUMEN =====
        Modelo  BA train           BA CV          AUC CV  AUC test           IC AUC
auPR test  F1 test            IC F1  Precision  Recall
 Random Forest    0.8872 0.8552 ± 0.0134 0.9240 ± 0.0112    0.9127 [0.8974, 0.9271]
0.6870   0.6444 [0.6075, 0.6771]     0.5355  0.8089
       XGBoost    0.8360 0.8203 ± 0.0180 0.9340 ± 0.0077    0.9308 [0.9190, 0.9420]
0.7405   0.6753 [0.6354, 0.7107]     0.6650  0.6859
          MLP    0.9533 0.7506 ± 0.0207 0.8529 ± 0.0186    0.8458 [0.8233, 0.8662]
0.5637   0.5459 [0.5015, 0.5865]     0.5642  0.5288
          SVM    0.7509 0.7479 ± 0.0208 0.8989 ± 0.0099    0.8905 [0.8715, 0.9087]
0.6241   0.5723 [0.5254, 0.6163]     0.6644  0.5026
Reg. logistica    0.7649 0.7599 ± 0.0227 0.8971 ± 0.0127    0.8852 [0.8667, 0.9041]
0.6217   0.6169 [0.5731, 0.6597]     0.6825  0.5628
          KNN    1.0000 0.6537 ± 0.0190 0.7488 ± 0.0201    0.7475 [0.7192, 0.7747]
0.4000   0.4185 [0.3753, 0.4617]     0.4236  0.4136
       K-Means    0.5000 0.5003 ± 0.0010 0.5031 ± 0.0094    0.5000 [0.5000, 0.5000]
0.1549   0.0000 [0.0000, 0.0000]     0.0000  0.0000
          GMM    0.7091 0.7221 ± 0.0284 0.8114 ± 0.0282    0.7772 [0.7508, 0.8015]
0.3688   0.4503 [0.4152, 0.4882]     0.3439  0.6518
```

```
===== REDUCCIÓN DE DIMENSIÓN =====
                    Modelo  ROC-AUC CV (media)  ROC-AUC CV (std)  ROC-AUC Test  auPR
Test  Accuracy Test  F1 Test  Precision Test  Recall Test  N° dimensiones Reducción
(%)
     RF baseline (74 vars)              0.9240            0.0112        0.9127
0.6870         0.8617   0.6444          0.5355       0.8089               74
0%
                  RF + PCA              0.8645            0.0198        0.8435
0.5624         0.8305   0.5261          0.4640       0.6073               57
23.0%
        RF + UMAP (2 comp)              0.5583            0.0302        0.5942
0.2774         0.8475   0.2450          0.5259       0.1597                2
97.3%
XGBoost baseline (74 vars)              0.9340            0.0077        0.9308
0.7405         0.8978   0.6753          0.6650       0.6859               74
0%
   XGBoost + PCA (57 comp)              0.8749            0.0184        0.8580
0.5913         0.8808   0.5532          0.6594       0.4764               57
23.0%
   XGBoost + UMAP (2 comp)              0.5715            0.0361        0.5980
0.2826         0.8390   0.2321          0.4444       0.1571                2
97.3%


===== IV Y PEARSON (top 15 por IV) =====
                               Pearson_abs        IV  Candidata_eliminacion
PageValues                        0.503271  2.428687                  False
ExitRates                         0.212674  0.863661                  False
BounceRates                       0.154210  0.522602                  False
ProductRelated_Duration           0.145974  0.495924                  False
ProductRelated                    0.155596  0.424999                  False
Administrative_Duration           0.099236  0.242291                  False
Administrative                    0.142691  0.162155                  False
Month_Nov                         0.145156  0.143214                  False
TrafficType_2                     0.127040  0.115694                  False
SpecialDay                        0.087851  0.105208                  False
VisitorType_New_Visitor           0.101738  0.067912                  False
VisitorType_Returning_Visitor     0.100726  0.067033                  False
TrafficType_3                     0.084708  0.066187                  False
Informational_Duration            0.068662  0.056979                  False
Informational                     0.097372  0.053792                  False
Candidatas a eliminación: 51
```